

Bloom's Taxonomy Learning Outcome Classification

A Complete Machine Learning Pipeline: Traditional → ML → Deep Learning

Research Overview

This notebook presents a comprehensive classification experiment on **Bloom's Taxonomy** cognitive levels applied to academic learning outcomes. The dataset contains 21,380 learning outcome statements that must be classified into six cognitive categories:

Level	Category	Description
1	Remember	Recall facts and basic concepts
2	Understand	Explain ideas or concepts
3	Apply	Use information in new situations
4	Analyze	Draw connections among ideas
5	Evaluate	Justify a decision or course of action
6	Create	Produce new or original work

Experimental Pipeline

We follow the **standard researcher pipeline**:

1. **Data Loading & Initial Inspection**
2. **Exploratory Data Analysis (EDA)**
3. **Data Preprocessing & Feature Engineering**
4. **Train/Validation/Test Split**
5. **Traditional ML Models** (Naive Bayes, Logistic Regression, SVM, KNN, Decision Tree, Random Forest, Gradient Boosting, XGBoost)
6. **Deep Learning Models** (BiLSTM, CNN-Text, BERT Fine-tuning)
7. **Comprehensive Evaluation** (Accuracy, Precision, Recall, F1, ROC-AUC, MCC, Per-class metrics)
8. **Model Comparison & Statistical Analysis**
9. **Error Analysis**

10. Conclusions & Recommendations

Note on Task Framing: The dataset contains both single-label (87.8%) and multi-label (12.2%) samples. We address both framings: (a) single-label classification on the primary label subset, and (b) multi-label classification on the full dataset using the binary relevance strategy.

Section 1: Environment Setup & Library Imports

We import all necessary libraries upfront. The stack covers:

- **Data manipulation:** pandas , numpy
- **NLP preprocessing:** nltk , re , sklearn
- **Traditional ML:** scikit-learn
- **Gradient Boosting:** xgboost , lightgbm
- **Deep Learning:** tensorflow / keras
- **Visualization:** matplotlib , seaborn
- **Transformers:** transformers (HuggingFace)

```
In [6]: # =====  
# SECTION 1: Environment Setup & Library Imports  
# =====  
  
# --- Standard Libraries ---  
import os  
import re  
import time  
import warnings  
import numpy as np  
import pandas as pd  
from collections import Counter  
warnings.filterwarnings('ignore')  
  
# --- NLP ---  
import nltk  
from nltk.corpus import stopwords  
from nltk.stem import WordNetLemmatizer  
from nltk.tokenize import word_tokenize  
nltk.download('punkt', quiet=True)  
nltk.download('punkt_tab', quiet=True)  
nltk.download('stopwords', quiet=True)
```

```

nltk.download('wordnet', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

# --- Scikit-Learn: Preprocessing & Feature Extraction ---
from sklearn.preprocessing import LabelEncoder, label_binarize
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline

# --- Scikit-Learn: Traditional ML Models ---
from sklearn.naive_bayes import MultinomialNB, ComplementNB
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.svm import LinearSVC, SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    ExtraTreesClassifier, AdaBoostClassifier, VotingClassifier
)
from sklearn.multiclass import OneVsRestClassifier
from sklearn.multioutput import MultiOutputClassifier

# --- Scikit-Learn: Evaluation Metrics ---
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score,
    precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    roc_auc_score, average_precision_score,
    matthews_corrcoef, cohen_kappa_score,
    roc_curve, precision_recall_curve,
    hamming_loss, jaccard_score
)

# --- XGBoost & LightGBM ---
try:
    import xgboost as xgb
    XGB_AVAILABLE = True
    print("XGBoost available:", xgb.__version__)
except ImportError:
    XGB_AVAILABLE = False
    print("XGBoost not installed – skipping")

try:
    import lightgbm as lgb
    LGB_AVAILABLE = True
    print("LightGBM available:", lgb.__version__)

```

```

except ImportError:
    LGB_AVAILABLE = False
    print("LightGBM not installed – skipping")

# --- Deep Learning (TensorFlow / Keras) ---
try:
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers, Model
    from tensorflow.keras.preprocessing.text import Tokenizer
    from tensorflow.keras.preprocessing.sequence import pad_sequences
    from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
    from tensorflow.keras.regularizers import l2
    TF_AVAILABLE = True
    print(f"TensorFlow available: {tf.__version__}")
    # Suppress TF verbose logging
    os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
except ImportError:
    TF_AVAILABLE = False
    print("TensorFlow not installed – DL models will be skipped")

# --- Transformers (BERT) ---
try:
    from transformers import (
        AutoTokenizer, AutoModelForSequenceClassification,
        TrainingArguments, Trainer
    )
    import torch
    BERT_AVAILABLE = True
    print(f"Transformers + PyTorch available")
except ImportError:
    BERT_AVAILABLE = False
    print("Transformers/PyTorch not installed – BERT will be skipped")

# --- Visualization ---
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from matplotlib.gridspec import GridSpec

# --- Plot Aesthetics ---
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})

# --- Reproducibility ---
RANDOM_STATE = 42

```

```

np.random.seed(RANDOM_STATE)

# --- Dataset Path ---
#kaggle DATA_PATH = 'content/sample_full.csv'
DATA_PATH = '/kaggle/input/datasets/daniyalsperspective/mon-clo-lo-dataset-full/sample_full.csv'
LABEL_COLS = ['Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate', 'Create']

print("\n✅ All imports complete.")
print(f"NumPy: {np.__version__} | Pandas: {pd.__version__}")

```

XGBoost available: 3.2.0
 LightGBM available: 4.6.0
 TensorFlow available: 2.19.0
 Transformers + PyTorch available

✅ All imports complete.
 NumPy: 2.0.2 | Pandas: 2.3.3

Section 2: Data Loading & Initial Inspection

We load the dataset and perform an initial sanity check to understand:

- Shape (rows × columns)
- Column types and null values
- Label distribution across the six Bloom's levels
- Whether samples are single-label or multi-label

This informs how we will **frame the classification task**.

```

In [7]: DATA_PATH = '/kaggle/input/datasets/daniyalsperspective/mon-clo-lo-dataset-full/sample_full.csv'
        LABEL_COLS = ['Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate', 'Create']

```

```

In [8]: # =====
        # SECTION 2: Data Loading & Initial Inspection
        # =====

        df_raw = pd.read_csv(DATA_PATH)
        print(f"Dataset shape: {df_raw.shape}")
        print(f"Columns: {list(df_raw.columns)}")
        print()

        # --- Basic Info ---

```

```

print("=" * 60)
print("DATA TYPES & NULL COUNTS")
print("=" * 60)
info_df = pd.DataFrame({
    'dtype': df_raw.dtypes,
    'null_count': df_raw.isnull().sum(),
    'null_pct': (df_raw.isnull().sum() / len(df_raw) * 100).round(2)
})
print(info_df)
print()

# --- Label Distribution ---
print("=" * 60)
print("LABEL DISTRIBUTION (NaN = not assigned, 1.0 = assigned)")
print("=" * 60)
label_counts = df_raw[LABEL_COLS].sum()
label_pct = (label_counts / len(df_raw) * 100).round(2)
label_info = pd.DataFrame({'count': label_counts, 'pct_of_dataset': label_pct})
print(label_info)
print()

# --- Single vs Multi-Label ---
df_raw['label_count'] = df_raw[LABEL_COLS].notna().sum(axis=1)
print("=" * 60)
print("SINGLE vs MULTI-LABEL DISTRIBUTION")
print("=" * 60)
print(df_raw['label_count'].value_counts().sort_index())

n_single = (df_raw['label_count'] == 1).sum()
n_multi = (df_raw['label_count'] > 1).sum()
print(f"\nSingle-label: {n_single} ({n_single/len(df_raw)*100:.1f}%)")
print(f"Multi-label: {n_multi} ({n_multi/len(df_raw)*100:.1f}%)")

# --- Sample Rows ---
print("\nSample rows:")
df_raw.head(5)

```

Dataset shape: (21380, 7)

Columns: ['Learning_outcome', 'Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate', 'Create']

=====

DATA TYPES & NULL COUNTS

=====

	dtype	null_count	null_pct
Learning_outcome	object	0	0.00
Remember	float64	20195	94.46
Understand	float64	15555	72.75
Apply	float64	15299	71.56
Analyze	float64	17921	83.82
Evaluate	float64	17546	82.07
Create	float64	17493	81.82

=====

LABEL DISTRIBUTION (NaN = not assigned, 1.0 = assigned)

=====

	count	pct_of_dataset
Remember	1185.0	5.54
Understand	5825.0	27.25
Apply	6081.0	28.44
Analyze	3459.0	16.18
Evaluate	3834.0	17.93
Create	3887.0	18.18

=====

SINGLE vs MULTI-LABEL DISTRIBUTION

=====

label_count

1 18773

2 2325

3 280

4 2

Name: count, dtype: int64

Single-label: 18773 (87.8%)

Multi-label: 2607 (12.2%)

Sample rows:

Out[8]:

		Learning_outcome	Remember	Understand	Apply	Analyze	Evaluate	Create	label_count
0	Analyze the health economic implications of e...		NaN	NaN	NaN	1.0	NaN	NaN	1
1	Apply research skills to operate effectively ...		NaN	NaN	1.0	NaN	NaN	NaN	1
2	Assess and synthesise diverse information abo...		NaN	NaN	NaN	NaN	1.0	1.0	2
3	Describe the general characteristics of the m...		NaN	1.0	NaN	NaN	NaN	NaN	1
4	Evaluate the different models of perioperativ...		NaN	NaN	NaN	NaN	1.0	NaN	1

Section 3: Exploratory Data Analysis (EDA)

EDA is a critical research step that reveals patterns, imbalances, and characteristics of the data that inform all modelling decisions. We examine:

1. **Class distribution** — Is the dataset balanced?
2. **Text statistics** — Word count distributions, sentence length
3. **Label co-occurrence** — Which levels appear together?
4. **Vocabulary analysis** — Top N-grams per class
5. **Word clouds** per Bloom's level (optional)

In [9]:

```
# =====
# SECTION 3: Exploratory Data Analysis (EDA)
# =====

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle("Exploratory Data Analysis – Bloom's Taxonomy Dataset", fontsize=15, fontweight='bold')

# --- Plot 1: Label Distribution ---
ax1 = axes[0, 0]
colors = sns.color_palette('muted', 6)
bars = ax1.bar(LABEL_COLS, label_counts.values, color=colors, edgecolor='white', linewidth=1.2)
ax1.set_title('Label Distribution (# of Samples per Class)', fontweight='bold')
ax1.set_ylabel('Count')
ax1.set_xlabel('Bloom\'s Taxonomy Level')
for bar, val in zip(bars, label_counts.values):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
             f'{int(val):,}', ha='center', va='bottom', fontsize=9)

# --- Plot 2: Single vs Multi-Label Pie ---
```



```

ax2 = axes[0, 1]
lc_vals = df_raw['label_count'].value_counts().sort_index()
ax2.pie(lc_vals.values, labels=[f'{i} Label(s)' for i in lc_vals.index],
        autopct='%1.1f%%', colors=sns.color_palette('pastel', len(lc_vals)),
        startangle=140, wedgeprops=dict(edgecolor='white'))
ax2.set_title('Label Multiplicity per Sample', fontweight='bold')

# --- Plot 3: Text Length Distribution ---
ax3 = axes[1, 0]
df_raw['text_len'] = df_raw['Learning_outcome'].str.split().str.len()
ax3.hist(df_raw['text_len'], bins=50, color='steelblue', alpha=0.8, edgecolor='white')
ax3.axvline(df_raw['text_len'].mean(), color='tomato', linestyle='--', linewidth=2,
            label=f'Mean: {df_raw["text_len"].mean():.1f}')
ax3.axvline(df_raw['text_len'].median(), color='gold', linestyle='--', linewidth=2,
            label=f'Median: {df_raw["text_len"].median():.1f}')
ax3.set_title('Word Count Distribution per Learning Outcome', fontweight='bold')
ax3.set_xlabel('Number of Words')
ax3.set_ylabel('Frequency')
ax3.legend()

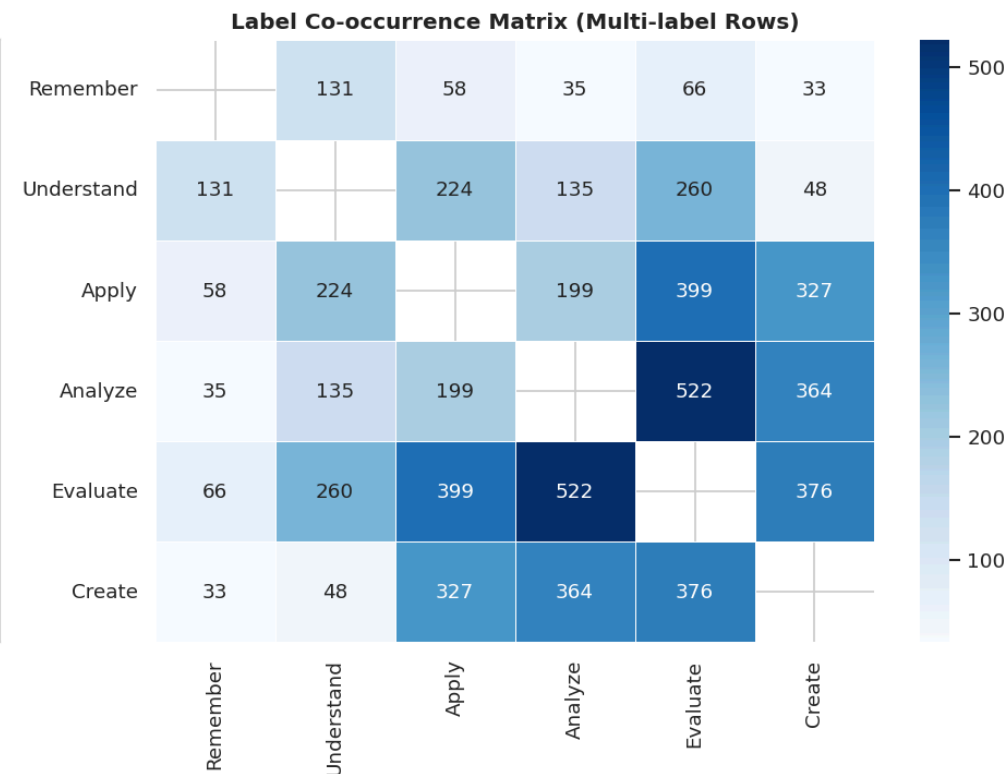
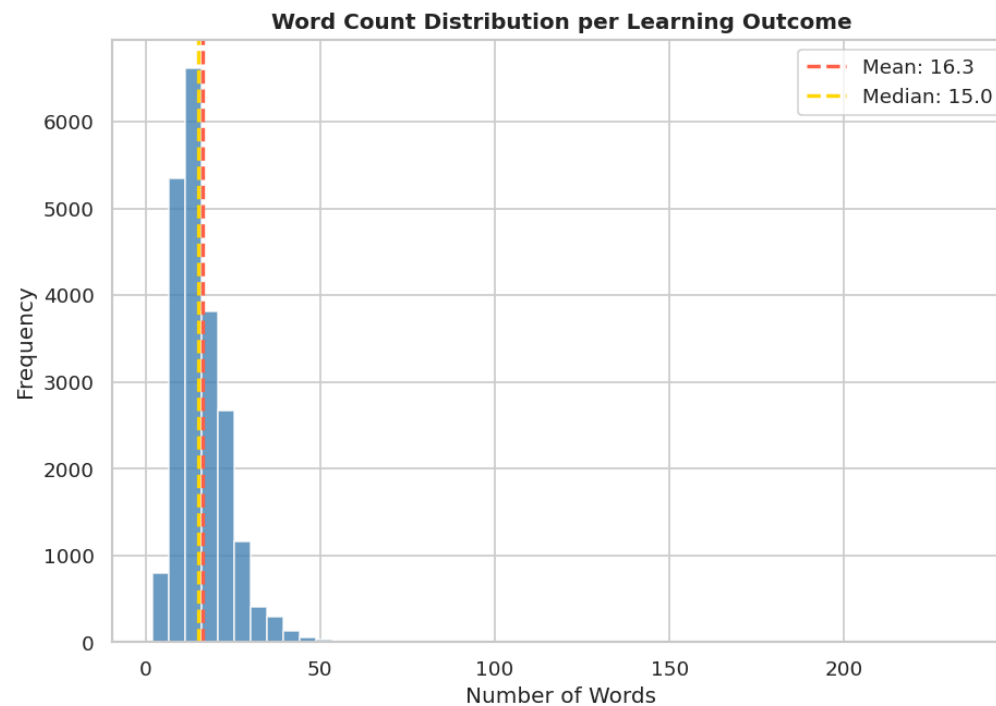
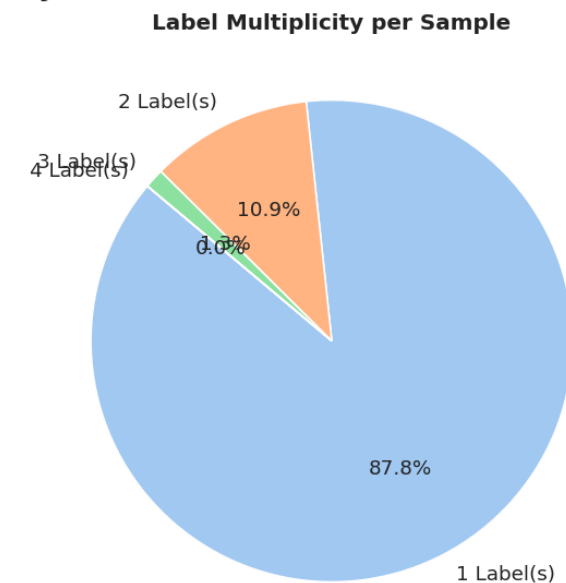
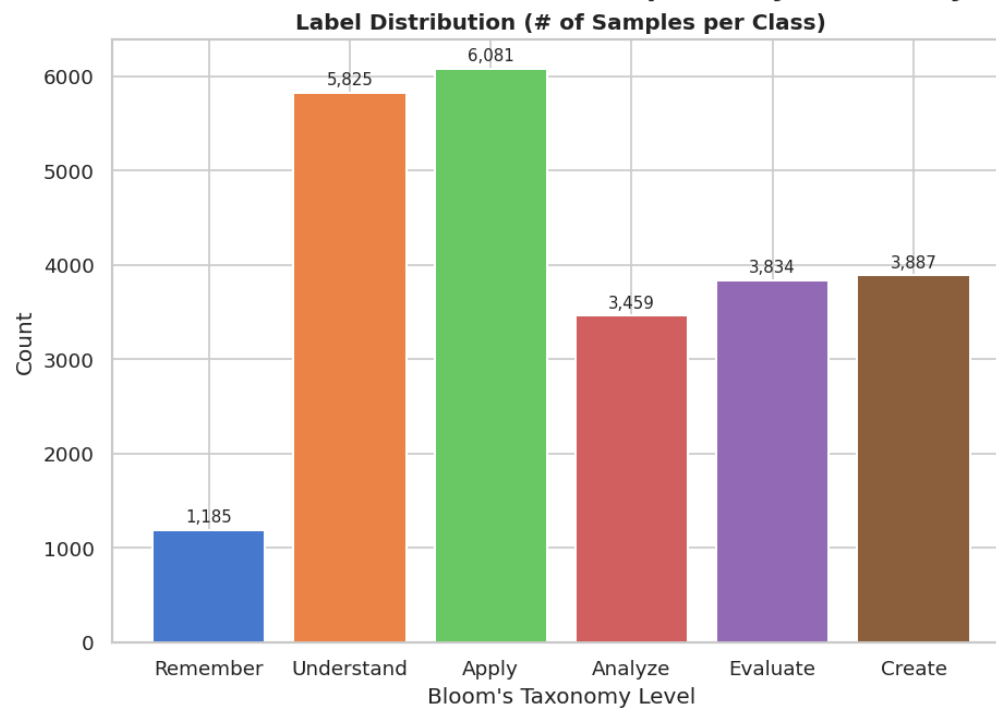
# --- Plot 4: Label Co-occurrence Heatmap ---
ax4 = axes[1, 1]
label_bin = df_raw[LABEL_COLS].fillna(0)
co_occur = label_bin.T.dot(label_bin)
mask = np.eye(len(LABEL_COLS), dtype=bool)
sns.heatmap(co_occur, annot=True, fmt='.0f', cmap='Blues', ax=ax4,
            mask=mask, linewidths=0.5,
            xticklabels=LABEL_COLS, yticklabels=LABEL_COLS)
ax4.set_title('Label Co-occurrence Matrix (Multi-label Rows)', fontweight='bold')

plt.tight_layout()
plt.show()

# --- Text Stats Summary ---
print("\nText Length Statistics:")
print(df_raw['text_len'].describe().round(2))

```

Exploratory Data Analysis — Bloom's Taxonomy Dataset



```
Text Length Statistics:
count    21380.00
mean      16.32
std       8.72
min       2.00
25%      11.00
50%      15.00
75%      20.00
max      236.00
Name: text_len, dtype: float64
```

```
In [11]: # --- Top Bigrams per Bloom's Level ---
```

```
# Build single-label subset for per-class bigram analysis
df_single = df_raw[df_raw['label_count'] == 1].copy()

def get_label_name(row):
    for col in LABEL_COLS:
        if row[col] == 1.0:
            return col
    return 'Unknown'

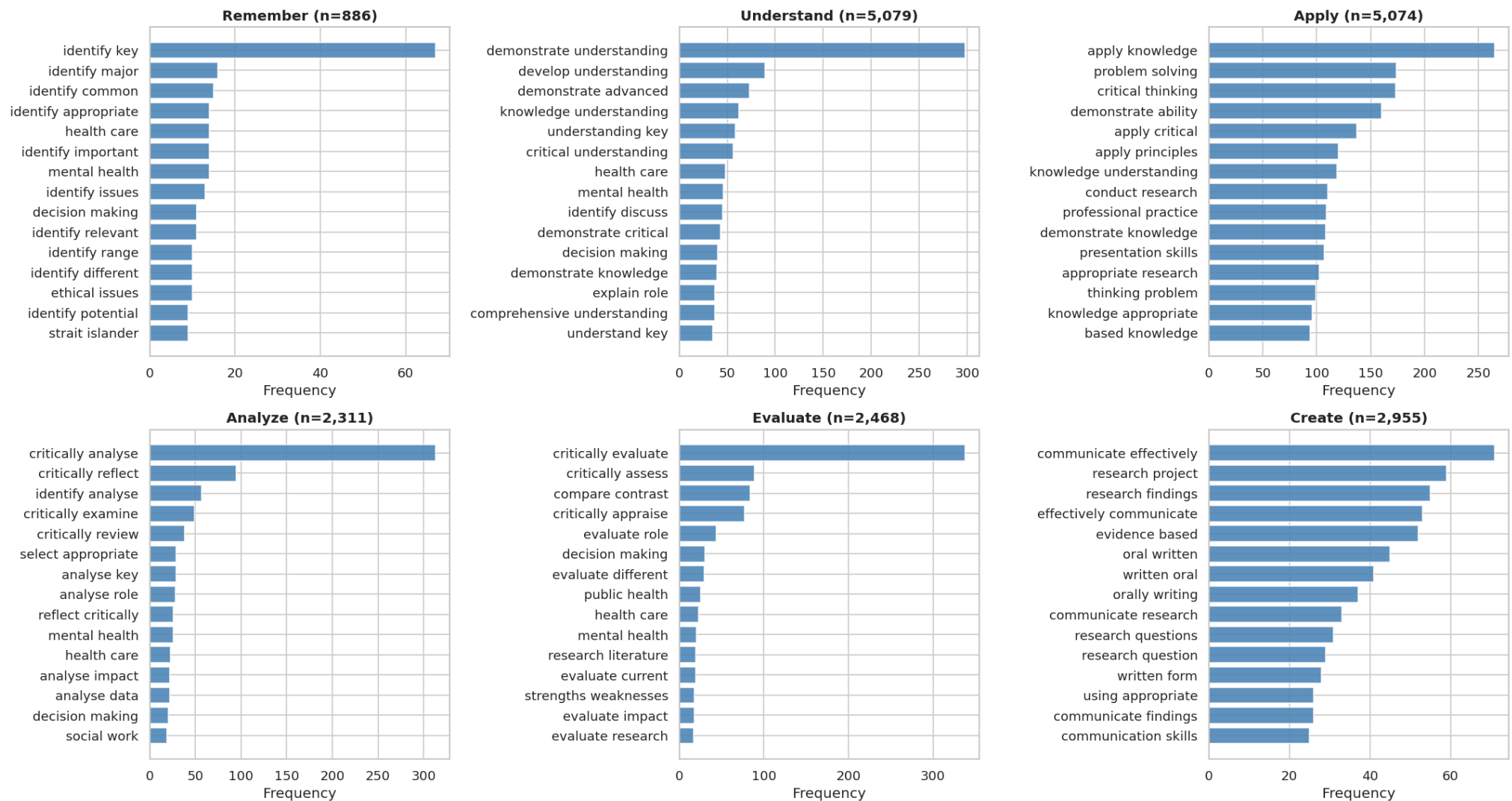
df_single['label'] = df_single.apply(get_label_name, axis=1)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Top 15 Bigrams per Bloom's Level (Single-label Samples)', fontsize=14, fontweight='bold')

for ax, level in zip(axes.flatten(), LABEL_COLS):
    subset = df_single[df_single['label'] == level]['Learning_outcome'].str.lower()
    cv = CountVectorizer(ngram_range=(2, 2), stop_words='english', max_features=15)
    cv.fit(subset)
    X = cv.transform(subset)
    freqs = np.asarray(X.sum(axis=0)).flatten()
    terms = cv.get_feature_names_out()
    top_idx = freqs.argsort()[::-1][:15]
    ax.barh(terms[top_idx][::-1], freqs[top_idx][::-1], color='steelblue', alpha=0.85)
    ax.set_title(f'{level} (n={len(subset):,})', fontweight='bold')
    ax.set_xlabel('Frequency')

plt.tight_layout()
plt.show()
```

Top 15 Bigrams per Bloom's Level (Single-label Samples)



Section 4: Data Preprocessing & Feature Engineering

Task Framing

Since 87.8% of samples have exactly one label, we define:

- **Primary task:** Multi-class single-label classification on the majority single-label subset (18,773 samples)
- **Secondary task:** Multi-label classification on the full dataset using Binary Relevance (one classifier per label)

Preprocessing Steps

1. Lowercasing
2. Remove punctuation & special characters
3. Remove extra whitespace
4. Lemmatization
5. Stop-word removal (selective — Bloom's verbs like `analyze` are kept)

Feature Representations

- **Bag of Words (BoW)** — CountVectorizer
- **TF-IDF (unigrams + bigrams)** — TfidfVectorizer
- **Word2Vec average embeddings** — dense feature vector
- **Padded sequences** — for deep learning models
- **BERT tokenization** — for transformer models

```
In [12]: # =====  
# SECTION 4: Data Preprocessing & Feature Engineering  
# =====  
  
# ----- 4.1 Text Cleaning -----  
  
STOP_WORDS = set(stopwords.words('english'))  
# Remove generic stop words but KEEP Bloom's taxonomy verbs  
BLOOMS_VERBS = {  
    'remember', 'recall', 'identify', 'describe', 'recognize',  
    'understand', 'explain', 'summarize', 'interpret', 'classify',  
    'apply', 'use', 'demonstrate', 'implement', 'execute',  
    'analyze', 'analyse', 'differentiate', 'organize', 'attribute',  
    'evaluate', 'critique', 'assess', 'judge', 'justify',  
    'create', 'design', 'construct', 'produce', 'develop'  
}  
CUSTOM_STOP = STOP_WORDS - BLOOMS_VERBS  
  
lemmatizer = WordNetLemmatizer()  
  
def clean_text(text: str, remove_stops: bool = True) -> str:  
    """Full NLP cleaning pipeline."""  
    text = str(text).lower().strip()  
    text = re.sub(r'^a-z\s', ' ', text)      # remove non-alpha  
    text = re.sub(r'\s+', ' ', text)        # collapse whitespace
```

```

tokens = word_tokenize(text)
if remove_stops:
    tokens = [t for t in tokens if t not in CUSTOM_STOP]
tokens = [lemmatizer.lemmatize(t) for t in tokens if len(t) > 1]
return ' '.join(tokens)

print("Cleaning text (this may take a minute)...")
t0 = time.time()
df_raw['text_clean'] = df_raw['Learning_outcome'].apply(clean_text)
print(f"Done in {time.time() - t0:.1f}s")

# Show before/after
print("\n--- Before ---")
print(df_raw['Learning_outcome'].iloc[2])
print("\n--- After ---")
print(df_raw['text_clean'].iloc[2])

```

Cleaning text (this may take a minute)...

Done in 5.6s

--- Before ---

Assess and synthesise diverse information about information and knowledge management technologies market and how to use implementation strategies to maximise their strengths and minimise their weaknesses.

--- After ---

assess synthesise diverse information information knowledge management technology market use implementation strategy maximise strength minimise weakness

In [13]: # ----- 4.2 Dataset Splits -----

```

# === Primary Task: Single-label classification ===
df_sl = df_raw[df_raw['label_count'] == 1].copy()
df_sl['label'] = df_sl.apply(get_label_name, axis=1)

le = LabelEncoder()
df_sl['label_enc'] = le.fit_transform(df_sl['label'])
CLASS_NAMES = le.classes_.tolist()
N_CLASSES = len(CLASS_NAMES)

print(f"Single-label dataset: {len(df_sl):,} samples | {N_CLASSES} classes")
print(f"Classes: {CLASS_NAMES}")
print("\nClass distribution:")
print(df_sl['label'].value_counts())

# Stratified 70/15/15 split
X_text = df_sl['text_clean'].values
y = df_sl['label_enc'].values

```

```

X_train_txt, X_temp_txt, y_train, y_temp = train_test_split(
    X_text, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y
)
X_val_txt, X_test_txt, y_val, y_test = train_test_split(
    X_temp_txt, y_temp, test_size=0.50, random_state=RANDOM_STATE, stratify=y_temp
)

print(f"\nTrain: {len(X_train_txt):,} | Val: {len(X_val_txt):,} | Test: {len(X_test_txt):,}")

# === Multi-Label: binary matrix ===
X_ml = df_raw['text_clean'].values
Y_ml = df_raw[LABEL_COLS].fillna(0).values.astype(int)

X_ml_train, X_ml_test, Y_ml_train, Y_ml_test = train_test_split(
    X_ml, Y_ml, test_size=0.20, random_state=RANDOM_STATE
)

```

Single-label dataset: 18,773 samples | 6 classes

Classes: ['Analyze', 'Apply', 'Create', 'Evaluate', 'Remember', 'Understand']

Class distribution:

label	
Understand	5079
Apply	5074
Create	2955
Evaluate	2468
Analyze	2311
Remember	886

Name: count, dtype: int64

Train: 13,141 | Val: 2,816 | Test: 2,816

```

In [14]: # ----- 4.3 Feature Extraction -----

# BoW
bow_vec = CountVectorizer(max_features=20000, ngram_range=(1, 2))
X_train_bow = bow_vec.fit_transform(X_train_txt)
X_val_bow   = bow_vec.transform(X_val_txt)
X_test_bow  = bow_vec.transform(X_test_txt)

# TF-IDF
tfidf_vec = TfidfVectorizer(max_features=20000, ngram_range=(1, 2),
                             sublinear_tf=True, min_df=2)
X_train_tfidf = tfidf_vec.fit_transform(X_train_txt)
X_val_tfidf   = tfidf_vec.transform(X_val_txt)
X_test_tfidf  = tfidf_vec.transform(X_test_txt)

```

```

# Character-Level TF-IDF (captures morphological patterns)
char_vec = TfidfVectorizer(max_features=30000, analyzer='char_wb',
                           ngram_range=(3, 5), sublinear_tf=True)
X_train_char = char_vec.fit_transform(X_train_txt)
X_val_char = char_vec.transform(X_val_txt)
X_test_char = char_vec.transform(X_test_txt)

print("Feature matrix shapes:")
print(f"  BoW:          {X_train_bow.shape}")
print(f"  TF-IDF:       {X_train_tfidf.shape}")
print(f"  Char TF-IDF:   {X_train_char.shape}")

# Multi-Label features
tfidf_ml = TfidfVectorizer(max_features=20000, ngram_range=(1, 2), sublinear_tf=True)
X_ml_train_tfidf = tfidf_ml.fit_transform(X_ml_train)
X_ml_test_tfidf = tfidf_ml.transform(X_ml_test)

```

```

Feature matrix shapes:
BoW:          (13141, 20000)
TF-IDF:       (13141, 20000)
Char TF-IDF: (13141, 30000)

```

Section 5: Evaluation Infrastructure

We define reusable evaluation functions that will be called after training each model. These compute:

Standard Metrics:

- Accuracy, Balanced Accuracy
- Macro/Weighted Precision, Recall, F1
- Matthews Correlation Coefficient (MCC)
- Cohen's Kappa

Per-class Metrics:

- Class-level Precision, Recall, F1, Support
- Per-class ROC-AUC

Visualizations:

- Confusion matrix heatmap
- Per-class bar charts
- ROC curves (multi-class OvR)

```
In [15]: # =====
# SECTION 5: Evaluation Infrastructure
# =====

# --- Global results store ---
all_results = {} # { model_name: {metric: value, ...} }

def evaluate_model(model_name, y_true, y_pred, y_proba=None,
                  class_names=None, plot=True):
    """
    Comprehensive evaluation of a multiclass classifier.

    Parameters
    -----
    model_name : str
    y_true      : array-like of true integer labels
    y_pred      : array-like of predicted integer labels
    y_proba     : array (n_samples, n_classes) of class probabilities, or None
    class_names: list of class name strings
    plot        : bool, whether to produce plots

    Returns
    -----
    dict of computed metrics
    """
    if class_names is None:
        class_names = [str(c) for c in sorted(set(y_true))]

    # --- Aggregate metrics ---
    acc      = accuracy_score(y_true, y_pred)
    bal_acc  = balanced_accuracy_score(y_true, y_pred)
    macro_p  = precision_score(y_true, y_pred, average='macro', zero_division=0)
    macro_r  = recall_score(y_true, y_pred, average='macro', zero_division=0)
    macro_f1 = f1_score(y_true, y_pred, average='macro', zero_division=0)
    weighted_f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)
    mcc      = matthews_corrcoef(y_true, y_pred)
    kappa    = cohen_kappa_score(y_true, y_pred)

    # --- ROC-AUC (requires probabilities) ---
    roc_auc_macro = np.nan
    if y_proba is not None:
```

```

    try:
        roc_auc_macro = roc_auc_score(
            y_true, y_proba, multi_class='ovr', average='macro'
        )
    except Exception:
        pass

results = {
    'Accuracy': acc,
    'Balanced Accuracy': bal_acc,
    'Macro Precision': macro_p,
    'Macro Recall': macro_r,
    'Macro F1': macro_f1,
    'Weighted F1': weighted_f1,
    'MCC': mcc,
    'Cohen Kappa': kappa,
    'ROC-AUC (macro OvR)': roc_auc_macro,
}

# --- Print summary ---
print(f"\n{'='*60}")
print(f"  MODEL: {model_name}")
print(f"{'='*60}")
for k, v in results.items():
    print(f"  {k:<25}: {v:.4f}")

# --- Per-class report ---
print(f"\n  --- Per-Class Classification Report ---")
print(classification_report(y_true, y_pred,
                           target_names=class_names, zero_division=0))

# --- Per-class ROC-AUC ---
if y_proba is not None:
    y_bin = label_binarize(y_true, classes=list(range(len(class_names))))
    print("  --- Per-Class ROC-AUC ---")
    for i, cname in enumerate(class_names):
        try:
            auc = roc_auc_score(y_bin[:, i], y_proba[:, i])
            results[f'ROC-AUC {cname}'] = auc
            print(f"    {cname:<15}: {auc:.4f}")
        except Exception:
            pass

if plot:
    _plot_evaluation(model_name, y_true, y_pred, y_proba, class_names)

```

```

all_results[model_name] = results
return results

def _plot_evaluation(model_name, y_true, y_pred, y_proba, class_names):
    """Confusion matrix + per-class F1 + ROC curves."""
    n_plots = 3 if y_proba is not None else 2
    fig, axes = plt.subplots(1, n_plots, figsize=(7 * n_plots, 6))
    fig.suptitle(f'{model_name} - Evaluation', fontsize=13, fontweight='bold')

    # Confusion Matrix
    ax = axes[0]
    cm = confusion_matrix(y_true, y_pred)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
    sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', ax=ax,
                xticklabels=class_names, yticklabels=class_names,
                linewidths=0.4)
    ax.set_title('Confusion Matrix (Row-normalised)')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('True')

    # Per-class F1
    ax = axes[1]
    per_class_f1 = f1_score(y_true, y_pred, average=None, zero_division=0)
    per_class_p = precision_score(y_true, y_pred, average=None, zero_division=0)
    per_class_r = recall_score(y_true, y_pred, average=None, zero_division=0)
    x = np.arange(len(class_names))
    width = 0.26
    ax.bar(x - width, per_class_p, width, label='Precision', color='steelblue', alpha=0.85)
    ax.bar(x, per_class_r, width, label='Recall', color='seagreen', alpha=0.85)
    ax.bar(x + width, per_class_f1, width, label='F1', color='tomato', alpha=0.85)
    ax.set_xticks(x)
    ax.set_xticklabels(class_names, rotation=30, ha='right')
    ax.set_ylim(0, 1.05)
    ax.set_title('Per-Class: Precision / Recall / F1')
    ax.set_ylabel('Score')
    ax.legend()

    # ROC Curves
    if y_proba is not None:
        ax = axes[2]
        y_bin = label_binarize(y_true, classes=list(range(len(class_names))))
        colors_roc = sns.color_palette('tab10', len(class_names))
        for i, (cname, color) in enumerate(zip(class_names, colors_roc)):
            fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
            auc = roc_auc_score(y_bin[:, i], y_proba[:, i])

```

```

        ax.plot(fpr, tpr, label=f'{cname} (AUC={auc:.2f})', color=color, lw=1.8)
    ax.plot([0, 1], [0, 1], 'k--', lw=1.2, label='Random')
    ax.set_title('ROC Curves (One-vs-Rest)')
    ax.set_xlabel('False Positive Rate')
    ax.set_ylabel('True Positive Rate')
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()

```

```

def evaluate_multilabel(model_name, Y_true, Y_pred, label_names=None):
    """Evaluation helper specifically for multi-label classification."""
    if label_names is None:
        label_names = [f'Label_{i}' for i in range(Y_true.shape[1])]

    results = {
        'Hamming Loss': hamming_loss(Y_true, Y_pred),
        'Subset Accuracy': accuracy_score(Y_true, Y_pred),
        'Macro F1': f1_score(Y_true, Y_pred, average='macro', zero_division=0),
        'Micro F1': f1_score(Y_true, Y_pred, average='micro', zero_division=0),
        'Weighted F1': f1_score(Y_true, Y_pred, average='weighted', zero_division=0),
        'Macro Precision': precision_score(Y_true, Y_pred, average='macro', zero_division=0),
        'Macro Recall': recall_score(Y_true, Y_pred, average='macro', zero_division=0),
    }

    print(f"\n{'='*60}")
    print(f"  MULTI-LABEL MODEL: {model_name}")
    print(f"{'='*60}")
    for k, v in results.items():
        print(f"  {k:<25}: {v:.4f}")

    print("\n  --- Per-Label Metrics ---")
    per_f1 = f1_score(Y_true, Y_pred, average=None, zero_division=0)
    per_p = precision_score(Y_true, Y_pred, average=None, zero_division=0)
    per_r = recall_score(Y_true, Y_pred, average=None, zero_division=0)
    for lname, p, r, f1 in zip(label_names, per_p, per_r, per_f1):
        print(f"  {lname:<12} Precision={p:.3f} Recall={r:.3f} F1={f1:.3f}")

    return results

```

```

print("✅ Evaluation functions defined.")

```

✅ Evaluation functions defined.

Section 6: Traditional Models

6.1 Naïve Bayes

Multinomial Naive Bayes is a strong baseline for text classification. It assumes feature independence given the class and works well with BoW/TF-IDF representations. We also test **Complement NB**, which is better suited to imbalanced datasets.

- **Feature:** TF-IDF unigrams + bigrams
- **Hyperparameter:** Laplace smoothing `alpha`

```
In [16]: # =====  
# SECTION 6.1: Naïve Bayes  
# =====  
  
# Multinomial NB (TF-IDF – all values non-negative, compatible)  
mnb = MultinomialNB(alpha=0.1)  
mnb.fit(X_train_tfidf, y_train)  
y_pred_mnb = mnb.predict(X_test_tfidf)  
y_proba_mnb = mnb.predict_proba(X_test_tfidf)  
evaluate_model('Multinomial Naive Bayes', y_test, y_pred_mnb,  
               y_proba_mnb, CLASS_NAMES)  
  
# Complement NB – better for imbalanced  
cnb = ComplementNB(alpha=0.1)  
cnb.fit(X_train_tfidf, y_train)  
y_pred_cnb = cnb.predict(X_test_tfidf)  
y_proba_cnb = cnb.predict_proba(X_test_tfidf)  
evaluate_model('Complement Naive Bayes', y_test, y_pred_cnb,  
               y_proba_cnb, CLASS_NAMES)
```

```

=====
MODEL: Multinomial Naive Bayes
=====

Accuracy          : 0.6985
Balanced Accuracy  : 0.6127
Macro Precision    : 0.7207
Macro Recall       : 0.6127
Macro F1           : 0.6371
Weighted F1        : 0.6900
MCC                : 0.6158
Cohen Kappa        : 0.6126
ROC-AUC (macro OvR) : 0.9172

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

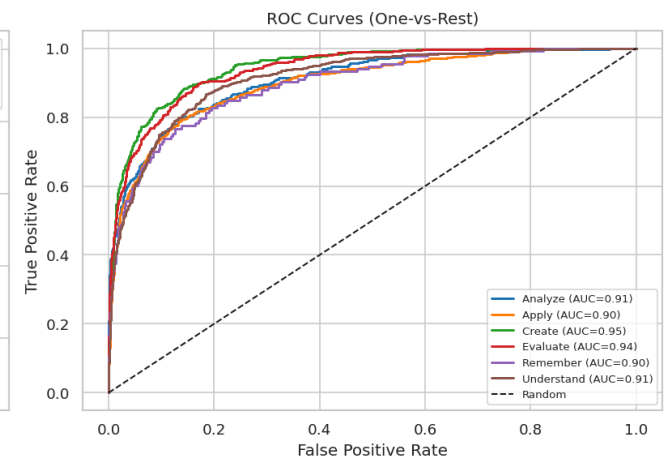
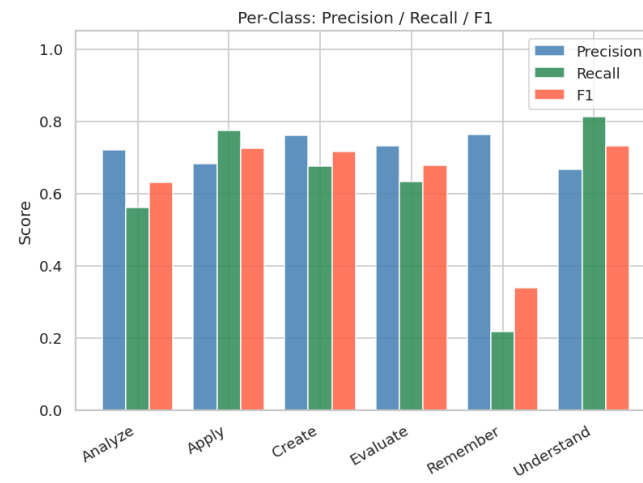
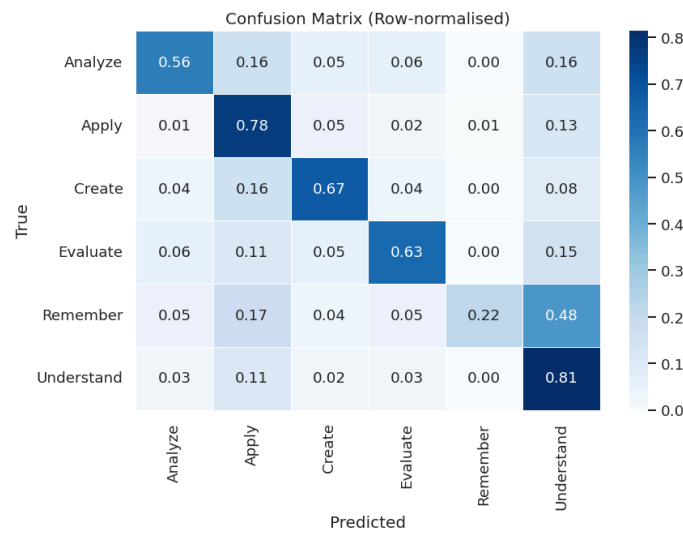
   Analyze      0.72      0.56      0.63      347
     Apply      0.68      0.78      0.73      761
      Create      0.76      0.67      0.72      443
    Evaluate      0.73      0.63      0.68      370
    Remember      0.76      0.22      0.34      133
   Understand      0.67      0.81      0.73      762

 accuracy              0.70      2816
 macro avg      0.72      0.61      0.64      2816
weighted avg      0.71      0.70      0.69      2816

--- Per-Class ROC-AUC ---
Analyze      : 0.9093
Apply        : 0.8998
Create       : 0.9451
Evaluate     : 0.9384
Remember     : 0.8969
Understand   : 0.9136

```

Multinomial Naive Bayes — Evaluation



```

=====
MODEL: Complement Naive Bayes
=====

Accuracy           : 0.7479
Balanced Accuracy   : 0.7043
Macro Precision     : 0.7304
Macro Recall        : 0.7043
Macro F1            : 0.7153
Weighted F1         : 0.7461
MCC                 : 0.6811
Cohen Kappa         : 0.6807
ROC-AUC (macro OvR) : 0.9362

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

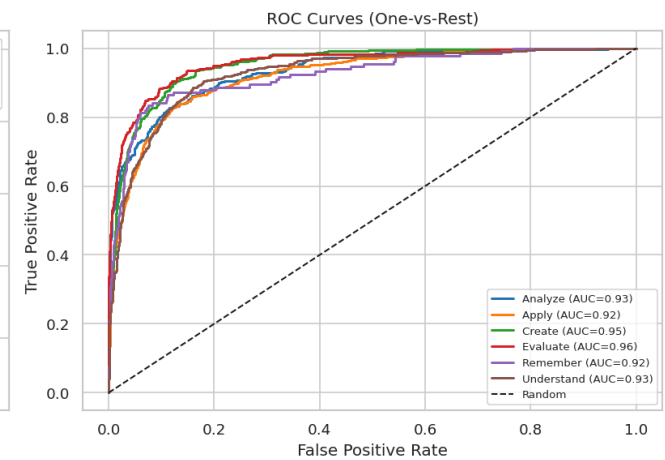
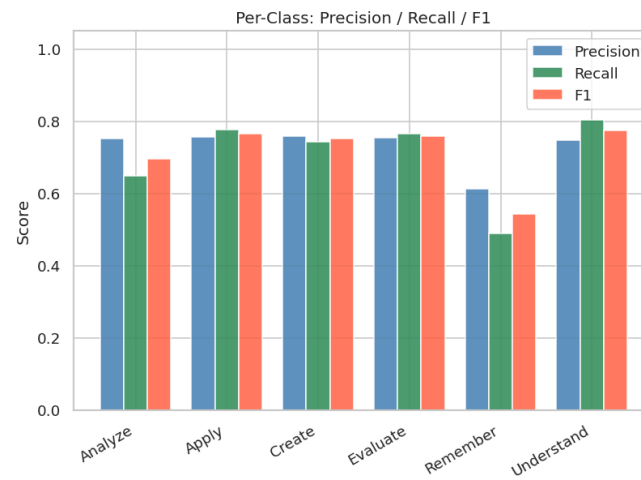
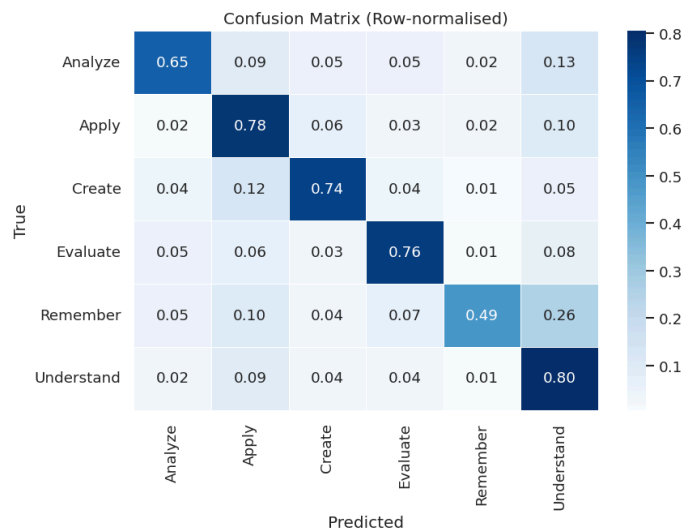
   Analyze       0.75     0.65     0.70       347
     Apply       0.76     0.78     0.77       761
      Create       0.76     0.74     0.75       443
    Evaluate       0.75     0.76     0.76       370
    Remember       0.61     0.49     0.54       133
   Understand     0.75     0.80     0.77       762

 accuracy                   0.75       2816
 macro avg       0.73     0.70     0.72       2816
weighted avg       0.75     0.75     0.75       2816

--- Per-Class ROC-AUC ---
Analyze       : 0.9344
Apply         : 0.9216
Create        : 0.9526
Evaluate      : 0.9571
Remember      : 0.9246
Understand    : 0.9271

```


Complement Naive Bayes — Evaluation



```
Out[16]: {'Accuracy': 0.7478693181818182,
          'Balanced Accuracy': np.float64(0.704289496359387),
          'Macro Precision': 0.7304337865912368,
          'Macro Recall': 0.704289496359387,
          'Macro F1': 0.7153199328146959,
          'Weighted F1': 0.7461034068805472,
          'MCC': np.float64(0.6810509276728589),
          'Cohen Kappa': np.float64(0.6806507609653312),
          'ROC-AUC (macro OvR)': np.float64(0.9362236167294958),
          'ROC-AUC Analyze': np.float64(0.9344132371084445),
          'ROC-AUC Apply': np.float64(0.9215828833235817),
          'ROC-AUC Create': np.float64(0.9525902292437781),
          'ROC-AUC Evaluate': np.float64(0.9571423835937326),
          'ROC-AUC Remember': np.float64(0.9245626178752884),
          'ROC-AUC Understand': np.float64(0.9270503492321491)}
```

6.2 Logistic Regression

Logistic Regression with L2 regularization is one of the strongest text-classification baselines, often competitive with complex models when data is sufficient. We use the `lbfgs` solver for multi-class support.

```
In [17]: # =====
# SECTION 6.2: Logistic Regression
# =====

lr = LogisticRegression(C=5.0, max_iter=1000, solver='lbfgs',
                        multi_class='multinomial', random_state=RANDOM_STATE)
```

```
lr.fit(X_train_tfidf, y_train)
y_pred_lr = lr.predict(X_test_tfidf)
y_proba_lr = lr.predict_proba(X_test_tfidf)
evaluate_model('Logistic Regression', y_test, y_pred_lr,
               y_proba_lr, CLASS_NAMES)

# SGD with Log Loss (online, memory-efficient)
sgd = SGDClassifier(loss='modified_huber', alpha=1e-4, max_iter=100,
                    random_state=RANDOM_STATE)
sgd.fit(X_train_tfidf, y_train)
y_pred_sgd = sgd.predict(X_test_tfidf)
y_proba_sgd = sgd.predict_proba(X_test_tfidf)
evaluate_model('SGD Classifier (Huber)', y_test, y_pred_sgd,
               y_proba_sgd, CLASS_NAMES)
```

```

=====
MODEL: Logistic Regression
=====

Accuracy          : 0.8857
Balanced Accuracy  : 0.8703
Macro Precision    : 0.8747
Macro Recall       : 0.8703
Macro F1           : 0.8722
Weighted F1        : 0.8855
MCC                : 0.8559
Cohen Kappa        : 0.8558
ROC-AUC (macro OvR) : 0.9816

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

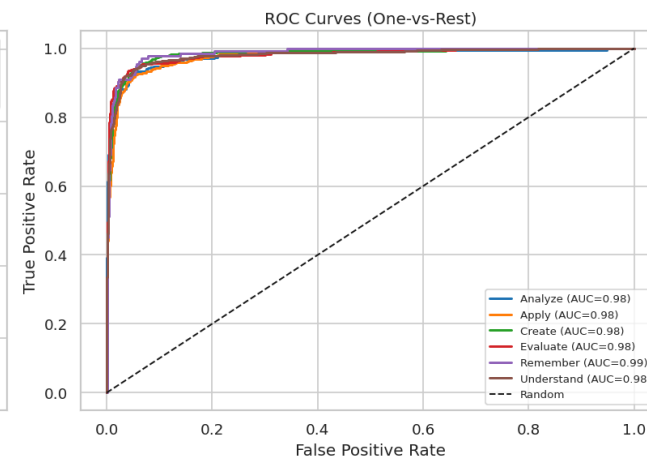
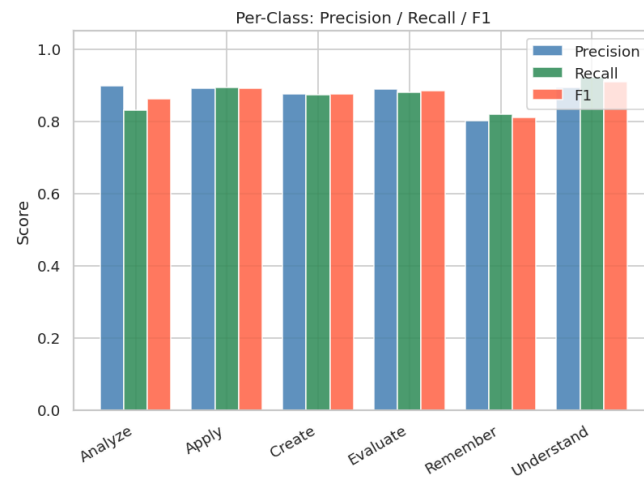
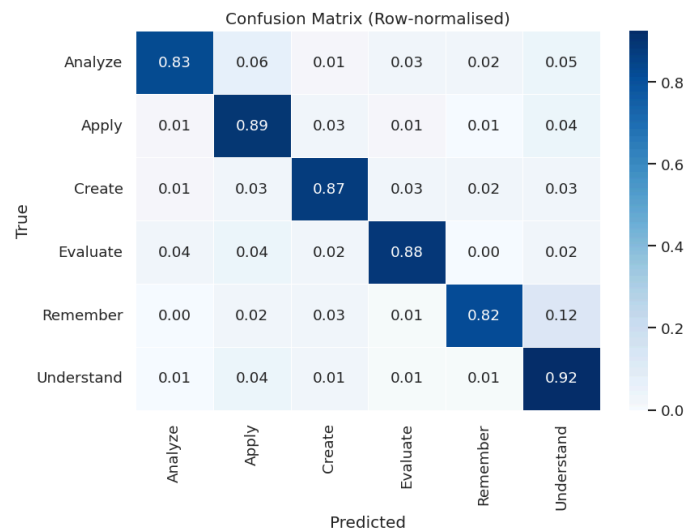
   Analyze       0.90      0.83      0.86       347
     Apply       0.89      0.89      0.89       761
      Create       0.88      0.87      0.87       443
    Evaluate       0.89      0.88      0.88       370
    Remember       0.80      0.82      0.81       133
   Understand     0.89      0.92      0.91       762

 accuracy          0.89      0.89      0.89      2816
 macro avg         0.87      0.87      0.87      2816
weighted avg         0.89      0.89      0.89      2816

--- Per-Class ROC-AUC ---
Analyze          : 0.9784
Apply            : 0.9792
Create           : 0.9833
Evaluate         : 0.9814
Remember         : 0.9868
Understand       : 0.9803

```

Logistic Regression — Evaluation



```

=====
MODEL: SGD Classifier (Huber)
=====

Accuracy           : 0.8896
Balanced Accuracy   : 0.8774
Macro Precision     : 0.8782
Macro Recall        : 0.8774
Macro F1            : 0.8773
Weighted F1         : 0.8895
MCC                 : 0.8609
Cohen Kappa         : 0.8608
ROC-AUC (macro OvR) : 0.9786

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

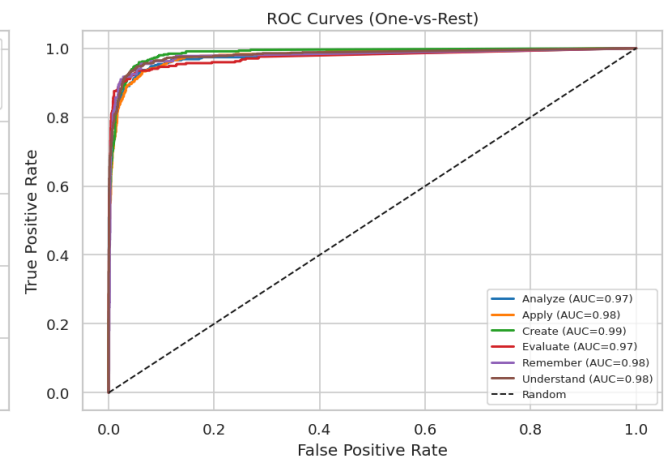
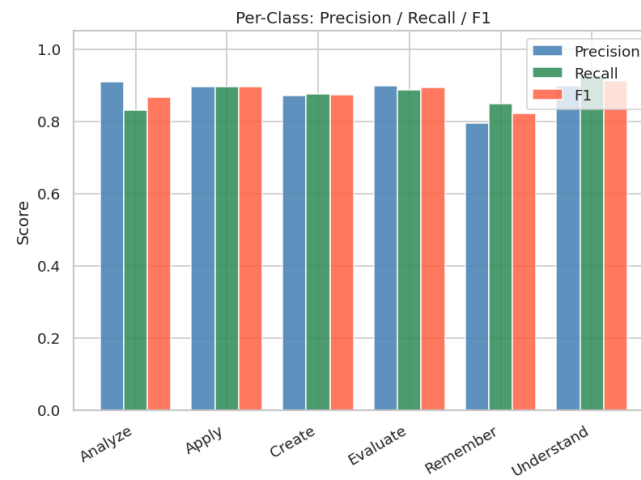
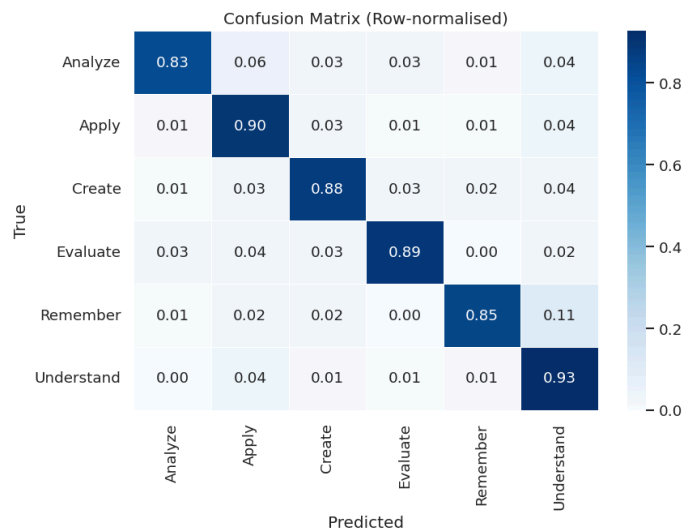
   Analyze         0.91      0.83      0.87       347
     Apply         0.90      0.90      0.90       761
      Create         0.87      0.88      0.87       443
    Evaluate         0.90      0.89      0.89       370
    Remember         0.80      0.85      0.82       133
   Understand         0.90      0.93      0.91       762

 accuracy          0.89          0.89      0.89      2816
 macro avg          0.88      0.88      0.88      2816
weighted avg          0.89      0.89      0.89      2816

--- Per-Class ROC-AUC ---
Analyze           : 0.9749
Apply             : 0.9775
Create            : 0.9863
Evaluate          : 0.9738
Remember          : 0.9785
Understand        : 0.9809

```

SGD Classifier (Huber) — Evaluation



```
Out[17]: {'Accuracy': 0.8895596590909091,
          'Balanced Accuracy': np.float64(0.8774377733962947),
          'Macro Precision': 0.8782067169352968,
          'Macro Recall': 0.8774377733962947,
          'Macro F1': 0.8773354782599272,
          'Weighted F1': 0.889462197650431,
          'MCC': np.float64(0.8608925403962456),
          'Cohen Kappa': np.float64(0.8607831551585483),
          'ROC-AUC (macro OvR)': np.float64(0.9786330609676909),
          'ROC-AUC Analyze': np.float64(0.97487461233999),
          'ROC-AUC Apply': np.float64(0.9774876826815785),
          'ROC-AUC Create': np.float64(0.9862761940909727),
          'ROC-AUC Evaluate': np.float64(0.973766325606064),
          'ROC-AUC Remember': np.float64(0.9784776888176461),
          'ROC-AUC Understand': np.float64(0.9809158622698939)}
```

6.3 Support Vector Machine (SVM)

SVMs are particularly well-suited for high-dimensional sparse text features. **LinearSVC** uses a linear kernel, which is effective for text. We apply **Platt scaling** (CalibratedClassifierCV) to obtain probability estimates.

```
In [18]: # =====
# SECTION 6.3: Support Vector Machine
# =====
from sklearn.calibration import CalibratedClassifierCV

# LinearSVC + calibration for probability estimates
```

```
svm_base = LinearSVC(C=1.0, max_iter=2000, random_state=RANDOM_STATE)
svm = CalibratedClassifierCV(svm_base, cv=3)
svm.fit(X_train_tfidf, y_train)
y_pred_svm = svm.predict(X_test_tfidf)
y_proba_svm = svm.predict_proba(X_test_tfidf)
evaluate_model('Linear SVM (Calibrated)', y_test, y_pred_svm,
              y_proba_svm, CLASS_NAMES)

# Character-Level TF-IDF SVM – captures subword patterns
svm_char_base = LinearSVC(C=1.0, max_iter=2000, random_state=RANDOM_STATE)
svm_char = CalibratedClassifierCV(svm_char_base, cv=3)
svm_char.fit(X_train_char, y_train)
y_pred_svm_c = svm_char.predict(X_test_char)
y_proba_svm_c = svm_char.predict_proba(X_test_char)
evaluate_model('Linear SVM (Char n-gram TF-IDF)', y_test, y_pred_svm_c,
              y_proba_svm_c, CLASS_NAMES)
```

```

=====
MODEL: Linear SVM (Calibrated)
=====

Accuracy           : 0.8888
Balanced Accuracy  : 0.8777
Macro Precision    : 0.8761
Macro Recall       : 0.8777
Macro F1           : 0.8764
Weighted F1        : 0.8888
MCC                : 0.8601
Cohen Kappa        : 0.8600
ROC-AUC (macro OvR) : 0.9826

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

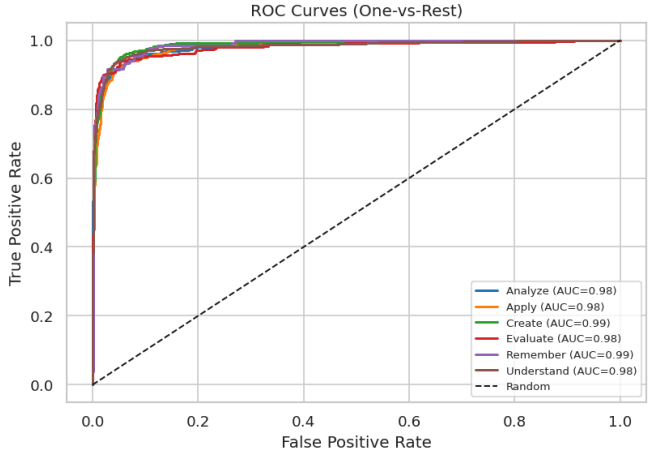
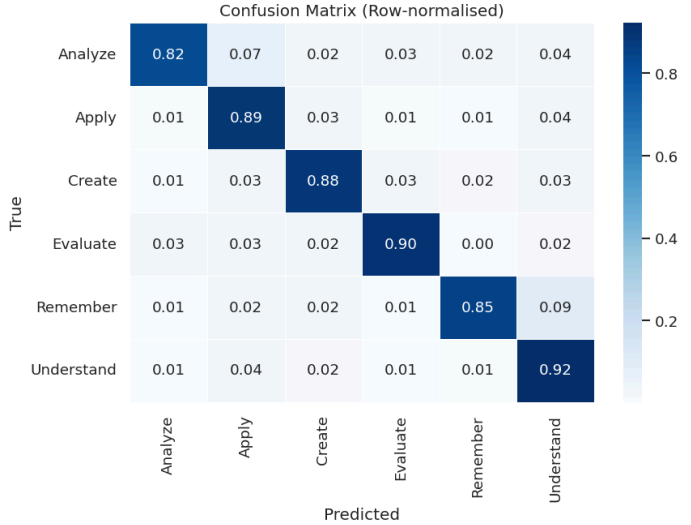
   Analyze       0.91      0.82      0.86       347
     Apply       0.89      0.89      0.89       761
      Create       0.87      0.88      0.88       443
    Evaluate       0.89      0.90      0.89       370
    Remember       0.79      0.85      0.82       133
   Understand     0.91      0.92      0.91       762

 accuracy              0.89      2816
 macro avg       0.88      0.88      0.88      2816
weighted avg       0.89      0.89      0.89      2816

--- Per-Class ROC-AUC ---
Analyze      : 0.9811
Apply        : 0.9794
Create       : 0.9876
Evaluate     : 0.9792
Remember     : 0.9864
Understand   : 0.9820

```


Linear SVM (Calibrated) — Evaluation



```

=====
MODEL: Linear SVM (Char n-gram TF-IDF)
=====

Accuracy           : 0.8672
Balanced Accuracy   : 0.8550
Macro Precision     : 0.8581
Macro Recall        : 0.8550
Macro F1            : 0.8563
Weighted F1         : 0.8670
MCC                 : 0.8328
Cohen Kappa         : 0.8327
ROC-AUC (macro OvR) : 0.9768

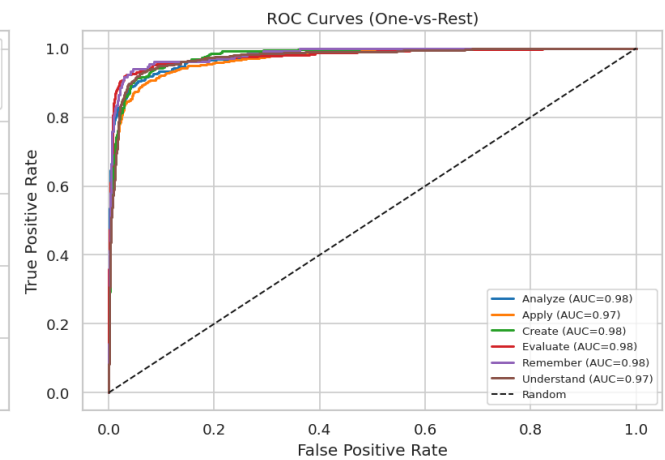
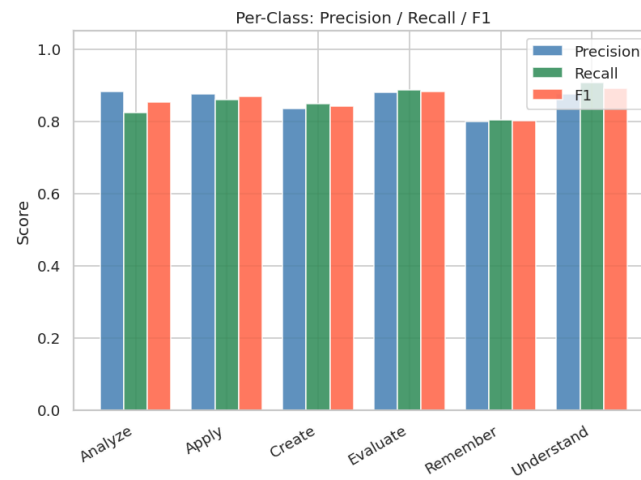
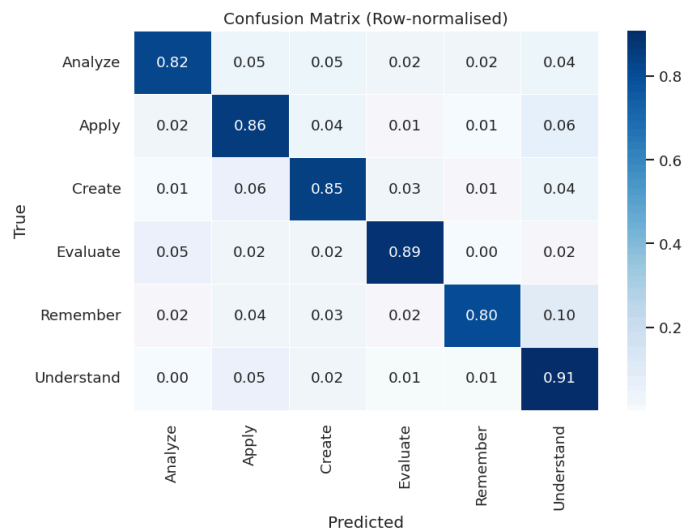
--- Per-Class Classification Report ---
      precision    recall  f1-score   support

   Analyze       0.88      0.82      0.85      347
     Apply       0.88      0.86      0.87      761
      Create       0.84      0.85      0.84      443
    Evaluate       0.88      0.89      0.88      370
    Remember       0.80      0.80      0.80      133
   Understand     0.88      0.91      0.89      762

 accuracy          0.87      0.87      0.87      2816
 macro avg         0.86      0.86      0.86      2816
weighted avg         0.87      0.87      0.87      2816

--- Per-Class ROC-AUC ---
Analyze           : 0.9750
Apply             : 0.9697
Create            : 0.9796
Evaluate          : 0.9788
Remember          : 0.9827
Understand        : 0.9749

```



```
Out[18]: {'Accuracy': 0.8671875,
          'Balanced Accuracy': np.float64(0.8550305669441265),
          'Macro Precision': 0.8581005635264077,
          'Macro Recall': 0.8550305669441265,
          'Macro F1': 0.8563261914649125,
          'Weighted F1': 0.8670495059408311,
          'MCC': np.float64(0.832752983717555),
          'Cohen Kappa': np.float64(0.8326505460867886),
          'ROC-AUC (macro OvR)': np.float64(0.9767840048258106),
          'ROC-AUC Analyze': np.float64(0.9750170121028127),
          'ROC-AUC Apply': np.float64(0.9696570334206177),
          'ROC-AUC Create': np.float64(0.979635458730127),
          'ROC-AUC Evaluate': np.float64(0.978792733862235),
          'ROC-AUC Remember': np.float64(0.9826588461463013),
          'ROC-AUC Understand': np.float64(0.9749429446927703)}
```

6.4 K-Nearest Neighbours (KNN)

KNN is a distance-based, non-parametric classifier. In text space, it searches for the k most similar training documents. While typically not state-of-the-art for text, it provides a useful instance-based baseline and highlights neighborhood structure.

```
In [19]: # =====
# SECTION 6.4: K-Nearest Neighbours
# =====

# Use TruncatedSVD to reduce TF-IDF to dense 200-dim for KNN
from sklearn.decomposition import TruncatedSVD
```

```
from sklearn.preprocessing import Normalizer
from sklearn.pipeline import make_pipeline

svd = TruncatedSVD(n_components=200, random_state=RANDOM_STATE)
norm = Normalizer(copy=False)
lsa_pipeline = make_pipeline(svd, norm)

X_train_lsa = lsa_pipeline.fit_transform(X_train_tfidf)
X_val_lsa = lsa_pipeline.transform(X_val_tfidf)
X_test_lsa = lsa_pipeline.transform(X_test_tfidf)
print(f"LSA reduced shape: {X_train_lsa.shape}")

knn = KNeighborsClassifier(n_neighbors=7, metric='cosine', n_jobs=-1)
knn.fit(X_train_lsa, y_train)
y_pred_knn = knn.predict(X_test_lsa)
y_proba_knn = knn.predict_proba(X_test_lsa)
evaluate_model('KNN (k=7, Cosine, LSA-200)', y_test, y_pred_knn,
              y_proba_knn, CLASS_NAMES)
```

LSA reduced shape: (13141, 200)

=====

MODEL: KNN (k=7, Cosine, LSA-200)

=====

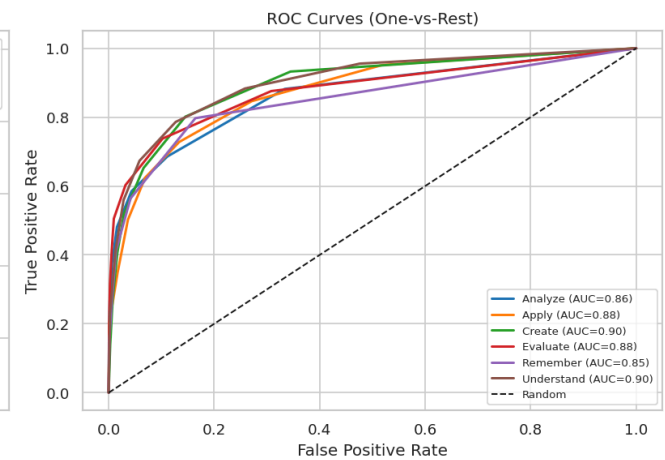
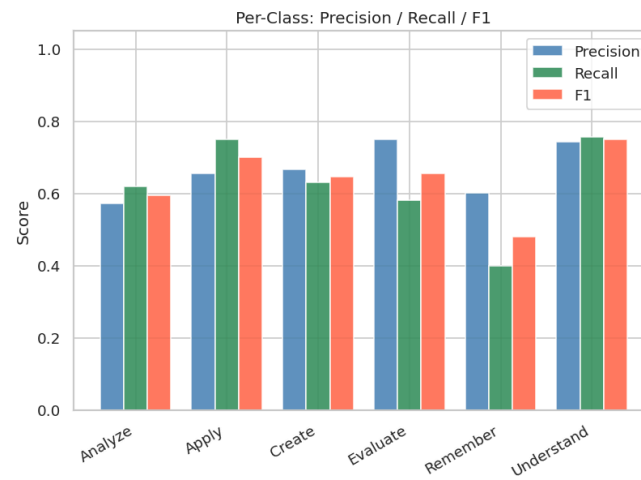
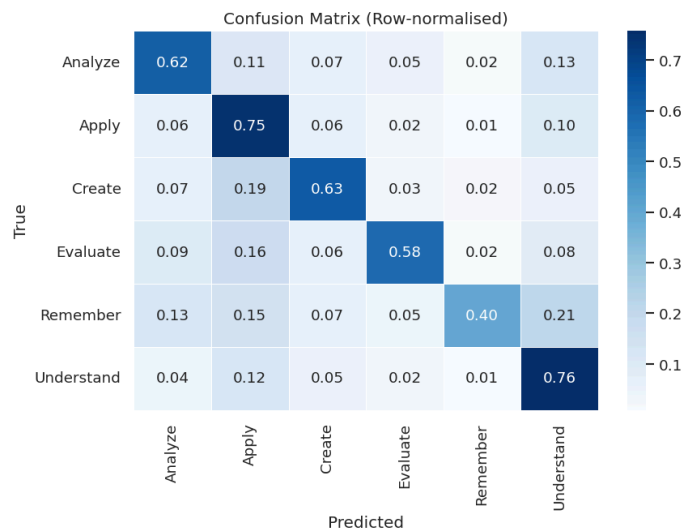
Accuracy : 0.6783
Balanced Accuracy : 0.6228
Macro Precision : 0.6650
Macro Recall : 0.6228
Macro F1 : 0.6379
Weighted F1 : 0.6760
MCC : 0.5924
Cohen Kappa : 0.5913
ROC-AUC (macro OvR) : 0.8777

--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.57	0.62	0.60	347
Apply	0.66	0.75	0.70	761
Create	0.67	0.63	0.65	443
Evaluate	0.75	0.58	0.65	370
Remember	0.60	0.40	0.48	133
Understand	0.74	0.76	0.75	762
accuracy			0.68	2816
macro avg	0.66	0.62	0.64	2816
weighted avg	0.68	0.68	0.68	2816

--- Per-Class ROC-AUC ---

Analyze : 0.8649
Apply : 0.8751
Create : 0.8952
Evaluate : 0.8779
Remember : 0.8516
Understand : 0.9017



```
Out[19]: {'Accuracy': 0.6782670454545454,
          'Balanced Accuracy': np.float64(0.6227528443465287),
          'Macro Precision': 0.6649551859167872,
          'Macro Recall': 0.6227528443465287,
          'Macro F1': 0.6378512947772754,
          'Weighted F1': 0.6760100961060711,
          'MCC': np.float64(0.5923807361711251),
          'Cohen Kappa': np.float64(0.591274506866865),
          'ROC-AUC (macro OvR)': np.float64(0.8777312774301245),
          'ROC-AUC Analyze': np.float64(0.8648754644041445),
          'ROC-AUC Apply': np.float64(0.8750638646166046),
          'ROC-AUC Create': np.float64(0.8951931958384344),
          'ROC-AUC Evaluate': np.float64(0.8778944111732337),
          'ROC-AUC Remember': np.float64(0.8516389744394531),
          'ROC-AUC Understand': np.float64(0.9017217541088768)}
```

6.5 Decision Tree

Decision Trees partition the feature space using recursive binary splits on individual features. They are interpretable but prone to overfitting. We use them as a component model and for comparison. The Gini impurity criterion is used.

```
In [20]: # =====
# SECTION 6.5: Decision Tree
# =====

dt = DecisionTreeClassifier(max_depth=20, min_samples_leaf=5,
                           criterion='gini', random_state=RANDOM_STATE)
```

```
dt.fit(X_train_tfidf, y_train)
y_pred_dt = dt.predict(X_test_tfidf)
y_proba_dt = dt.predict_proba(X_test_tfidf)
evaluate_model('Decision Tree', y_test, y_pred_dt,
               y_proba_dt, CLASS_NAMES)
```

```
=====
```

MODEL: Decision Tree

```
=====
```

```
Accuracy          : 0.7663
Balanced Accuracy  : 0.7322
Macro Precision    : 0.8307
Macro Recall       : 0.7322
Macro F1           : 0.7602
Weighted F1        : 0.7663
MCC                : 0.7137
Cohen Kappa        : 0.6991
ROC-AUC (macro OvR) : 0.9355
```

--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.91	0.63	0.75	347
Apply	0.59	0.92	0.72	761
Create	0.84	0.49	0.62	443
Evaluate	0.94	0.69	0.79	370
Remember	0.79	0.80	0.79	133
Understand	0.92	0.87	0.89	762
accuracy			0.77	2816
macro avg	0.83	0.73	0.76	2816
weighted avg	0.81	0.77	0.77	2816

--- Per-Class ROC-AUC ---

```
Analyze      : 0.9238
Apply        : 0.9275
Create       : 0.9087
Evaluate     : 0.9353
Remember     : 0.9637
Understand   : 0.9541
```



```
rf.fit(X_train_tfidf, y_train)
y_pred_rf = rf.predict(X_test_tfidf)
y_proba_rf = rf.predict_proba(X_test_tfidf)
evaluate_model('Random Forest (200 trees)', y_test, y_pred_rf,
               y_proba_rf, CLASS_NAMES)

# Extra Trees (faster, more randomized)
et = ExtraTreesClassifier(n_estimators=200, max_features='sqrt',
                           n_jobs=-1, random_state=RANDOM_STATE)
et.fit(X_train_tfidf, y_train)
y_pred_et = et.predict(X_test_tfidf)
y_proba_et = et.predict_proba(X_test_tfidf)
evaluate_model('Extra Trees (200 trees)', y_test, y_pred_et,
               y_proba_et, CLASS_NAMES)
```

```

=====
MODEL: Random Forest (200 trees)
=====

Accuracy           : 0.8796
Balanced Accuracy   : 0.8579
Macro Precision     : 0.8809
Macro Recall        : 0.8579
Macro F1            : 0.8683
Weighted F1         : 0.8792
MCC                 : 0.8481
Cohen Kappa         : 0.8477
ROC-AUC (macro OvR) : 0.9831

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

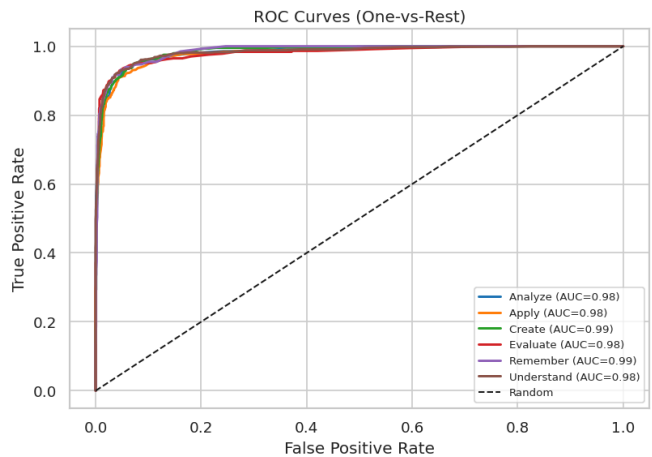
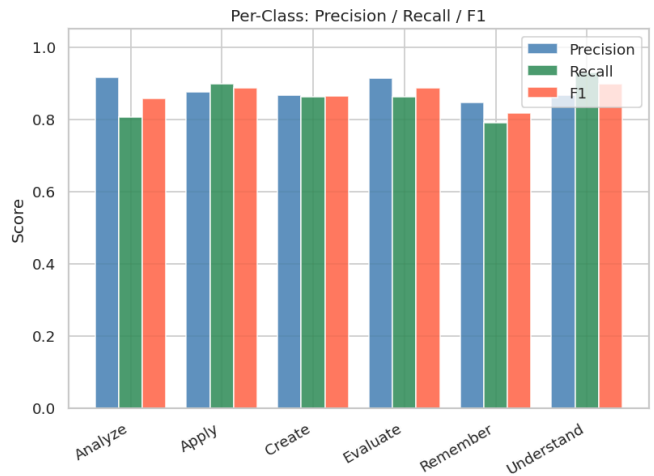
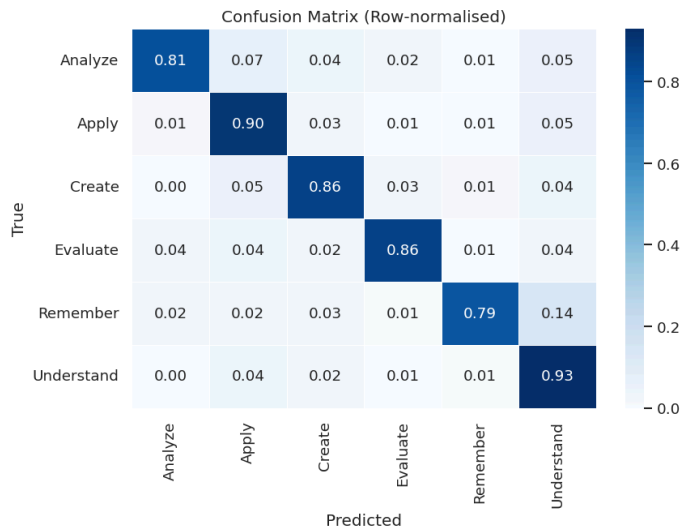
   Analyze       0.92      0.81      0.86       347
     Apply       0.88      0.90      0.89       761
      Create       0.87      0.86      0.86       443
    Evaluate       0.91      0.86      0.89       370
    Remember       0.85      0.79      0.82       133
   Understand     0.87      0.93      0.90       762

 accuracy                   0.88       2816
 macro avg       0.88      0.86      0.87       2816
weighted avg       0.88      0.88      0.88       2816

--- Per-Class ROC-AUC ---
Analyze      : 0.9833
Apply        : 0.9796
Create       : 0.9850
Evaluate     : 0.9808
Remember     : 0.9868
Understand   : 0.9834

```

Random Forest (200 trees) — Evaluation



```

=====
MODEL: Extra Trees (200 trees)
=====

Accuracy          : 0.8697
Balanced Accuracy  : 0.8443
Macro Precision    : 0.8716
Macro Recall       : 0.8443
Macro F1           : 0.8566
Weighted F1        : 0.8691
MCC                : 0.8355
Cohen Kappa        : 0.8350
ROC-AUC (macro OvR) : 0.9814

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

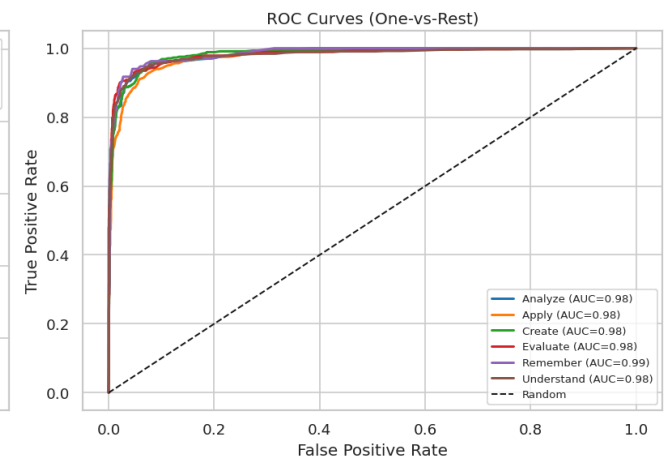
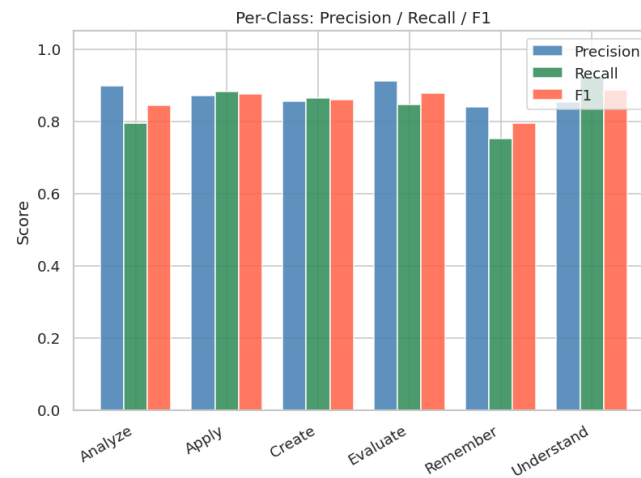
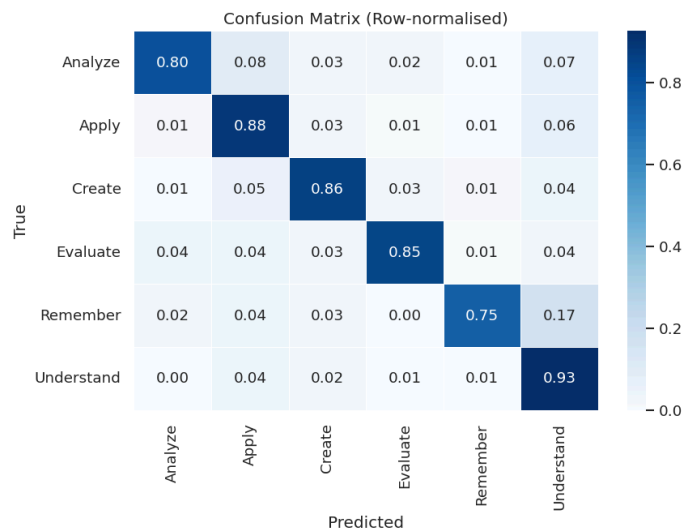
   Analyze       0.90      0.80      0.84       347
     Apply       0.87      0.88      0.88       761
      Create       0.85      0.86      0.86       443
    Evaluate       0.91      0.85      0.88       370
    Remember       0.84      0.75      0.79       133
   Understand       0.85      0.93      0.89       762

 accuracy          0.87          2816
 macro avg         0.87      0.84      0.86      2816
weighted avg         0.87      0.87      0.87      2816

--- Per-Class ROC-AUC ---
Analyze          : 0.9831
Apply            : 0.9760
Create           : 0.9817
Evaluate         : 0.9820
Remember         : 0.9852
Understand       : 0.9801

```

Extra Trees (200 trees) — Evaluation



```
Out[21]: {'Accuracy': 0.8696732954545454,
          'Balanced Accuracy': np.float64(0.8443366640381242),
          'Macro Precision': 0.8716252089353983,
          'Macro Recall': 0.8443366640381242,
          'Macro F1': 0.8565731520485264,
          'Weighted F1': 0.8691331476375709,
          'MCC': np.float64(0.8355012733072914),
          'Cohen Kappa': np.float64(0.8350261634681999),
          'ROC-AUC (macro OvR)': np.float64(0.9813586827670967),
          'ROC-AUC Analyze': np.float64(0.9830538446185145),
          'ROC-AUC Apply': np.float64(0.9760207947667783),
          'ROC-AUC Create': np.float64(0.981744874381563),
          'ROC-AUC Evaluate': np.float64(0.982045148173521),
          'ROC-AUC Remember': np.float64(0.985152968145298),
          'ROC-AUC Understand': np.float64(0.9801344665169045)}
```

6.7 Gradient Boosting (GBM, XGBoost, LightGBM)

Gradient Boosting builds trees sequentially, where each tree corrects the residuals of the previous one. It is typically the strongest traditional ML algorithm on tabular/sparse data.

- **Sklearn GBM**: Reference implementation
- **XGBoost**: Regularized boosting with second-order gradients
- **LightGBM**: Histogram-based boosting — very fast on large datasets

We apply these on the **LSA-reduced** dense representation for efficiency.

```

In [22]: # =====
# SECTION 6.7: Gradient Boosting Models
# =====

# --- XGBoost ---
if XGB_AVAILABLE:
    xgb_clf = xgb.XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        use_label_encoder=False, eval_metric='mlogloss',
        random_state=RANDOM_STATE, n_jobs=-1
    )
    xgb_clf.fit(X_train_lsa, y_train,
                eval_set=[(X_val_lsa, y_val)],
                verbose=False)
    y_pred_xgb = xgb_clf.predict(X_test_lsa)
    y_proba_xgb = xgb_clf.predict_proba(X_test_lsa)
    evaluate_model('XGBoost (LSA-200)', y_test, y_pred_xgb,
                  y_proba_xgb, CLASS_NAMES)
else:
    print("XGBoost not available – install with: pip install xgboost")

# --- LightGBM ---
if LGB_AVAILABLE:
    lgb_clf = lgb.LGBMClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.1,
        num_leaves=63, subsample=0.8, colsample_bytree=0.8,
        random_state=RANDOM_STATE, n_jobs=-1, verbose=-1
    )
    lgb_clf.fit(X_train_lsa, y_train,
                eval_set=[(X_val_lsa, y_val)],
                callbacks=[lgb.early_stopping(50, verbose=False),
                          lgb.log_evaluation(0)])
    y_pred_lgb = lgb_clf.predict(X_test_lsa)
    y_proba_lgb = lgb_clf.predict_proba(X_test_lsa)
    evaluate_model('LightGBM (LSA-200)', y_test, y_pred_lgb,
                  y_proba_lgb, CLASS_NAMES)
else:
    print("LightGBM not available – install with: pip install lightgbm")

# --- Sklearn HistGradientBoosting (always available, efficient) ---
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(
    max_iter=200, max_depth=6, learning_rate=0.1,
    random_state=RANDOM_STATE
)

```

```
)
hgb.fit(X_train_lsa, y_train)
y_pred_hgb = hgb.predict(X_test_lsa)
y_proba_hgb = hgb.predict_proba(X_test_lsa)
evaluate_model('HistGradientBoosting (LSA-200)', y_test, y_pred_hgb,
               y_proba_hgb, CLASS_NAMES)
```

```
=====
MODEL: XGBoost (LSA-200)
=====
```

```
Accuracy           : 0.8203
Balanced Accuracy   : 0.7908
Macro Precision     : 0.8225
Macro Recall        : 0.7908
Macro F1            : 0.8045
Weighted F1         : 0.8194
MCC                 : 0.7728
Cohen Kappa         : 0.7722
ROC-AUC (macro OvR) : 0.9668
```

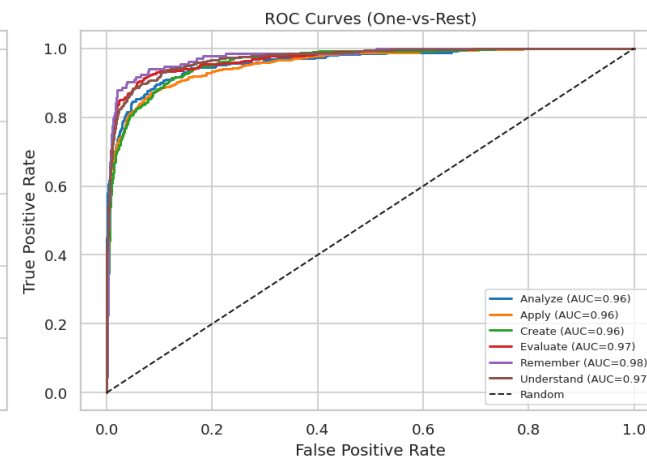
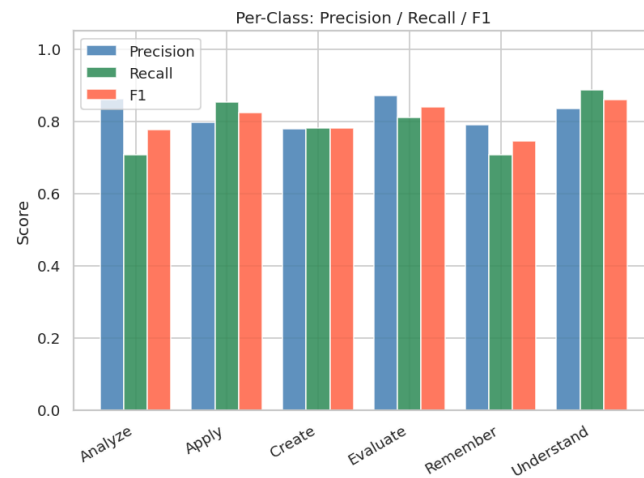
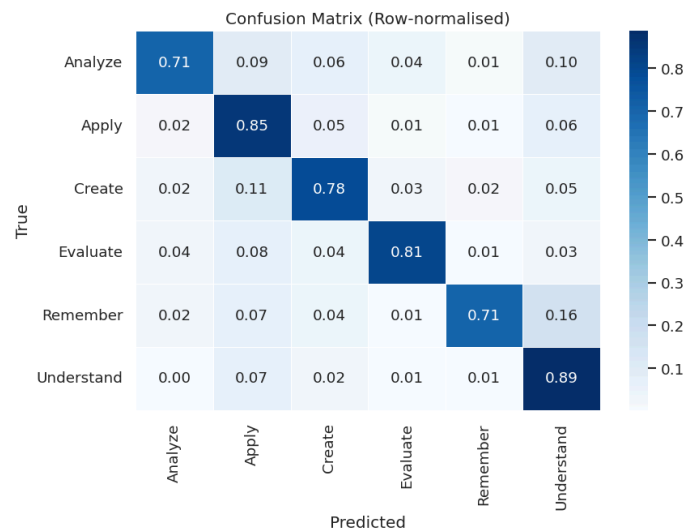
--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.86	0.71	0.78	347
Apply	0.80	0.85	0.82	761
Create	0.78	0.78	0.78	443
Evaluate	0.87	0.81	0.84	370
Remember	0.79	0.71	0.75	133
Understand	0.83	0.89	0.86	762
accuracy			0.82	2816
macro avg	0.82	0.79	0.80	2816
weighted avg	0.82	0.82	0.82	2816

--- Per-Class ROC-AUC ---

```
Analyze      : 0.9617
Apply        : 0.9562
Create       : 0.9619
Evaluate     : 0.9716
Remember     : 0.9775
Understand   : 0.9721
```

XGBoost (LSA-200) — Evaluation




```
=====
MODEL: LightGBM (LSA-200)
=====

Accuracy           : 0.8082
Balanced Accuracy  : 0.7774
Macro Precision    : 0.8084
Macro Recall       : 0.7774
Macro F1           : 0.7907
Weighted F1        : 0.8071
MCC                : 0.7575
Cohen Kappa        : 0.7569
ROC-AUC (macro OvR) : 0.9642

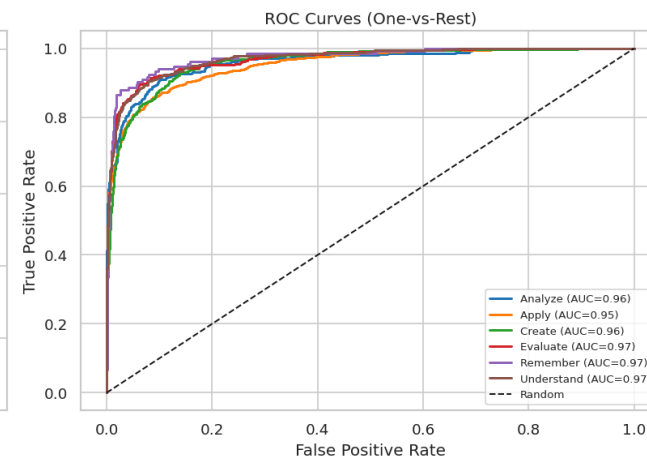
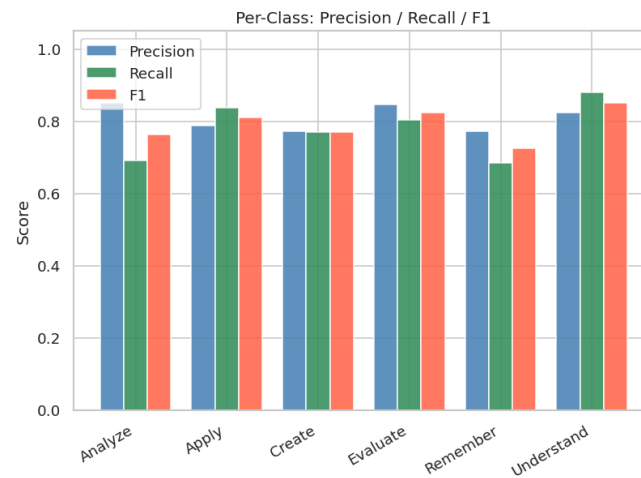
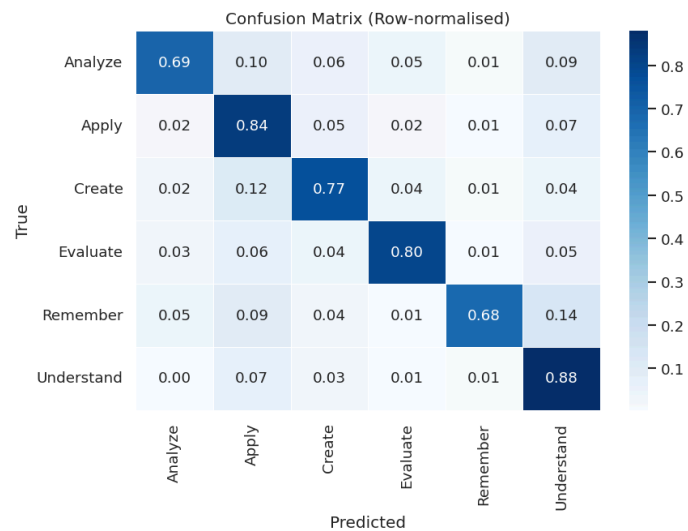
--- Per-Class Classification Report ---
      precision    recall  f1-score   support

   Analyze       0.85     0.69     0.76       347
     Apply       0.79     0.84     0.81       761
      Create       0.77     0.77     0.77       443
    Evaluate       0.85     0.80     0.82       370
    Remember       0.77     0.68     0.73       133
   Understand     0.82     0.88     0.85       762

 accuracy          0.81          0.81       2816
 macro avg         0.81     0.78     0.79       2816
weighted avg         0.81     0.81     0.81       2816

--- Per-Class ROC-AUC ---
Analyze          : 0.9605
Apply            : 0.9528
Create           : 0.9597
Evaluate         : 0.9688
Remember         : 0.9740
Understand       : 0.9697
```

LightGBM (LSA-200) — Evaluation



```

=====
MODEL: HistGradientBoosting (LSA-200)
=====
Accuracy                : 0.8082
Balanced Accuracy       : 0.7768
Macro Precision         : 0.8103
Macro Recall            : 0.7768
Macro F1                : 0.7912
Weighted F1             : 0.8071
MCC                     : 0.7575
Cohen Kappa             : 0.7569
ROC-AUC (macro OvR)    : 0.9622

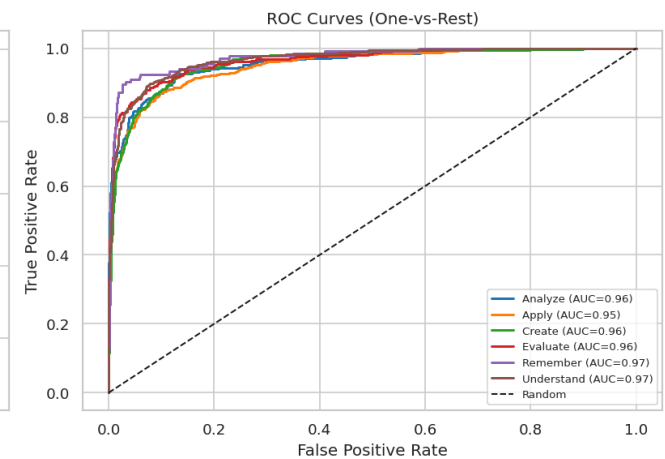
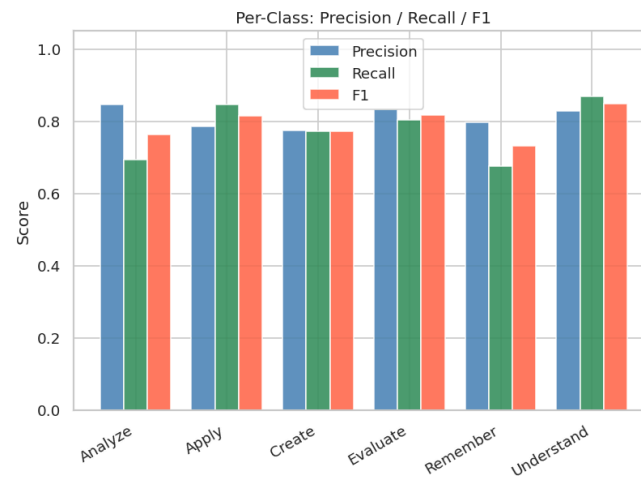
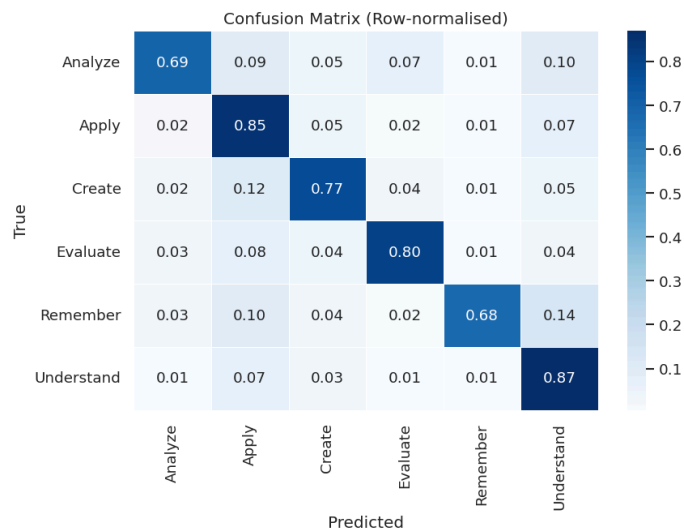
--- Per-Class Classification Report ---
      precision    recall  f1-score   support

   Analyze       0.85      0.69      0.76       347
     Apply       0.79      0.85      0.82       761
      Create       0.77      0.77      0.77       443
    Evaluate       0.83      0.80      0.82       370
    Remember       0.80      0.68      0.73       133
   Understand       0.83      0.87      0.85       762

 accuracy                   0.81       2816
 macro avg       0.81      0.78      0.79       2816
weighted avg       0.81      0.81      0.81       2816

--- Per-Class ROC-AUC ---
Analyze      : 0.9576
Apply        : 0.9511
Create       : 0.9579
Evaluate     : 0.9650
Remember     : 0.9742
Understand   : 0.9673

```



```
Out[22]: {'Accuracy': 0.8082386363636364,
          'Balanced Accuracy': np.float64(0.7768248814957945),
          'Macro Precision': 0.8102645712702494,
          'Macro Recall': 0.7768248814957945,
          'Macro F1': 0.7911874020695319,
          'Weighted F1': 0.8071417917471199,
          'MCC': np.float64(0.7574799194102947),
          'Cohen Kappa': np.float64(0.7568989654489393),
          'ROC-AUC (macro OvR)': np.float64(0.9621635358210486),
          'ROC-AUC Analyze': np.float64(0.9575543657783021),
          'ROC-AUC Apply': np.float64(0.9510702718602428),
          'ROC-AUC Create': np.float64(0.9578830313563329),
          'ROC-AUC Evaluate': np.float64(0.964974254712603),
          'ROC-AUC Remember': np.float64(0.9741620170441012),
          'ROC-AUC Understand': np.float64(0.9673372741747106)}
```

6.8 Cross-Validation Analysis

We perform 5-fold stratified cross-validation on the best traditional models to obtain more reliable performance estimates and assess variance.

```
In [23]: # =====
# SECTION 6.8: Cross-Validation
# =====

from scipy import stats

cv_models = {
    'Complement NB': (cnb, X_train_tfidf, y_train),
```

```

'Logistic Regression':(lr, X_train_tfidf, y_train),
'Linear SVM':          (svm, X_train_tfidf, y_train),
'Random Forest':      (rf, X_train_tfidf, y_train),
'HistGradBoost':      (hgb, X_train_lsa, y_train),
}

cv_results = {}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

print("Running 5-fold cross-validation...")
for name, (model, X, y) in cv_models.items():
    scores = cross_val_score(model, X, y, cv=skf,
                              scoring='f1_macro', n_jobs=-1)
    cv_results[name] = scores
    print(f"{name:<30}: {scores.mean():.4f} ± {scores.std():.4f}  "
          f"(95% CI: [{scores.mean()-1.96*scores.std():.4f}, "
          f"{scores.mean()+1.96*scores.std():.4f}])")

# Visualize
fig, ax = plt.subplots(figsize=(10, 5))
names = list(cv_results.keys())
means = [cv_results[n].mean() for n in names]
stds = [cv_results[n].std() for n in names]

ax.barh(names, means, xerr=stds, color='steelblue', alpha=0.8,
        capsize=5, edgecolor='white')
ax.set_xlabel('Macro F1 (5-fold CV)')
ax.set_title('Cross-Validation Macro F1 – Traditional Models', fontweight='bold')
ax.set_xlim(0, 1)
plt.tight_layout()
plt.show()

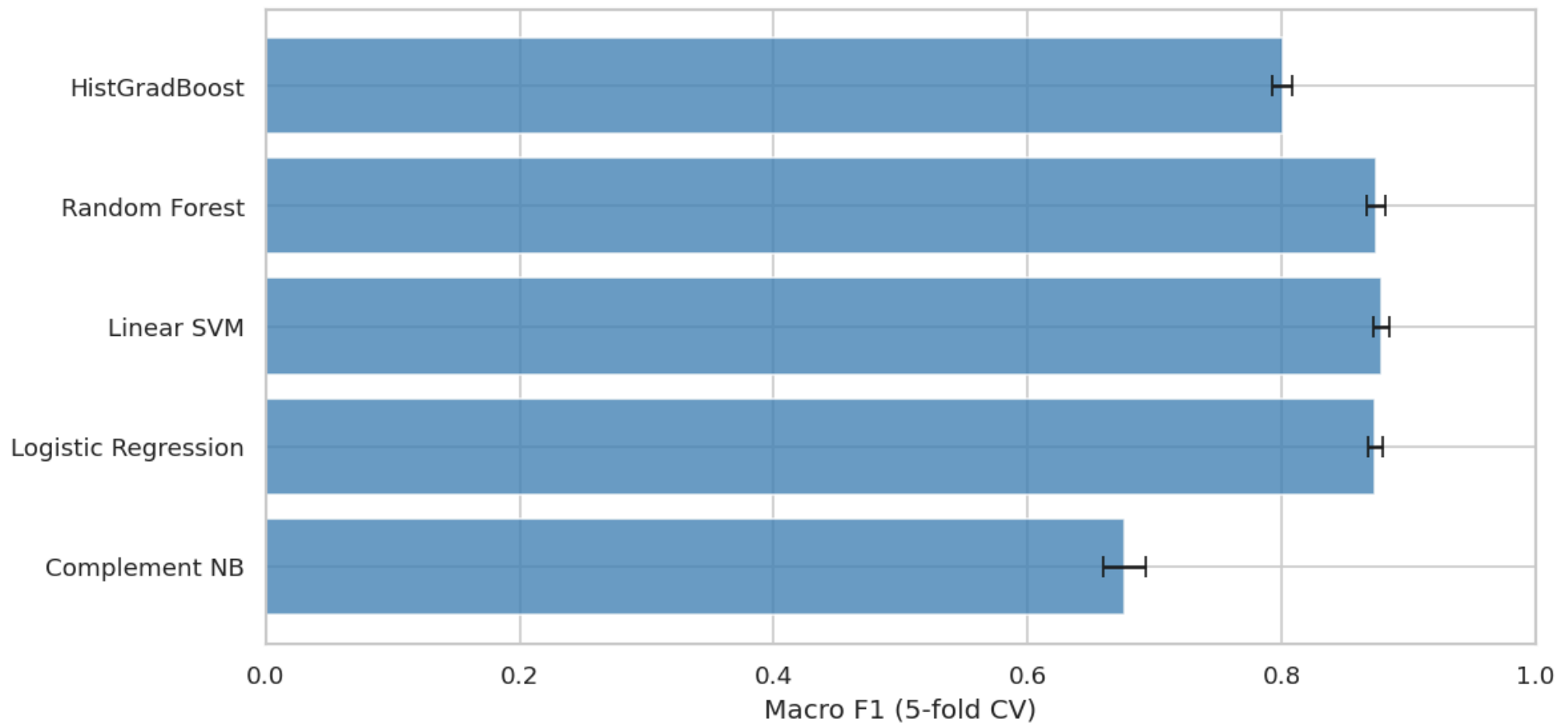
```

Running 5-fold cross-validation...

Complement NB : 0.6764 ± 0.0168 (95% CI: [0.6435, 0.7093])

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
Logistic Regression      : 0.8739 ± 0.0061 (95% CI: [0.8619, 0.8858])
Linear SVM               : 0.8788 ± 0.0062 (95% CI: [0.8667, 0.8910])
Random Forest           : 0.8745 ± 0.0075 (95% CI: [0.8597, 0.8892])
HistGradBoost           : 0.8009 ± 0.0078 (95% CI: [0.7855, 0.8162])
```

Cross-Validation Macro F1 — Traditional Models



6.9 Multi-label Classification (Traditional)

We now address the full dataset using the **Binary Relevance** strategy: one independent binary classifier per label, treating multi-label classification as a set of independent binary problems. We also apply **Classifier Chain** which respects label correlations.

```
In [24]: # =====  
# SECTION 6.9: Multi-Label Classification  
# =====  
from sklearn.multioutput import ClassifierChain  
  
# Binary Relevance with LR  
br_lr = MultiOutputClassifier(  
    LogisticRegression(C=5.0, max_iter=500, solver='lbfgs'),  
    n_jobs=-1
```

```

)
br_lr.fit(X_ml_train_tfidf, Y_ml_train)
Y_pred_br = br_lr.predict(X_ml_test_tfidf)
evaluate_multilabel('BR-LogisticRegression (Multi-label)', Y_ml_test, Y_pred_br, LABEL_COLS)

# Classifier Chain (LR)
cc_lr = ClassifierChain(
    LogisticRegression(C=5.0, max_iter=500, solver='lbfgs'),
    order='random', random_state=RANDOM_STATE
)
cc_lr.fit(X_ml_train_tfidf, Y_ml_train)
Y_pred_cc = cc_lr.predict(X_ml_test_tfidf)
evaluate_multilabel('ClassifierChain-LR (Multi-label)', Y_ml_test, Y_pred_cc, LABEL_COLS)

# Binary Relevance with LinearSVC
br_svm = OneVsRestClassifier(LinearSVC(C=1.0, max_iter=2000), n_jobs=-1)
br_svm.fit(X_ml_train_tfidf, Y_ml_train)
Y_pred_brsvm = br_svm.predict(X_ml_test_tfidf)
evaluate_multilabel('OvR-LinearSVM (Multi-label)', Y_ml_test, Y_pred_brsvm, LABEL_COLS)

```



```

=====
MULTI-LABEL MODEL: BR-LogisticRegression (Multi-label)
=====

Hamming Loss           : 0.0504
Subset Accuracy        : 0.7617
Macro F1               : 0.8448
Micro F1               : 0.8593
Weighted F1            : 0.8588
Macro Precision        : 0.9056
Macro Recall           : 0.7927

--- Per-Label Metrics ---
Remember    Precision=0.864 Recall=0.682 F1=0.762
Understand  Precision=0.921 Recall=0.846 F1=0.882
Apply       Precision=0.897 Recall=0.836 F1=0.865
Analyze     Precision=0.930 Recall=0.799 F1=0.859
Evaluate    Precision=0.926 Recall=0.828 F1=0.874
Create      Precision=0.896 Recall=0.766 F1=0.826

=====
MULTI-LABEL MODEL: ClassifierChain-LR (Multi-label)
=====

Hamming Loss           : 0.0537
Subset Accuracy        : 0.8071
Macro F1               : 0.8393
Micro F1               : 0.8550
Weighted F1            : 0.8548
Macro Precision        : 0.8698
Macro Recall           : 0.8141

--- Per-Label Metrics ---
Remember    Precision=0.864 Recall=0.682 F1=0.762
Understand  Precision=0.913 Recall=0.852 F1=0.881
Apply       Precision=0.853 Recall=0.882 F1=0.867
Analyze     Precision=0.799 Recall=0.829 F1=0.814
Evaluate    Precision=0.909 Recall=0.841 F1=0.874
Create      Precision=0.882 Recall=0.798 F1=0.838

=====
MULTI-LABEL MODEL: OvR-LinearSVM (Multi-label)
=====

Hamming Loss           : 0.0474
Subset Accuracy        : 0.7834
Macro F1               : 0.8603
Micro F1               : 0.8712
Weighted F1            : 0.8711

```

```
Macro Precision      : 0.8890
Macro Recall        : 0.8335

--- Per-Label Metrics ---
Remember    Precision=0.833 Recall=0.762 F1=0.796
Understand  Precision=0.913 Recall=0.866 F1=0.889
Apply       Precision=0.890 Recall=0.863 F1=0.876
Analyze     Precision=0.911 Recall=0.835 F1=0.871
Evaluate    Precision=0.914 Recall=0.859 F1=0.885
Create      Precision=0.872 Recall=0.816 F1=0.843
```

```
Out[24]: {'Hamming Loss': 0.04735734331150608,
          'Subset Accuracy': 0.7834424695977549,
          'Macro F1': 0.8602515705632817,
          'Micro F1': 0.8712241653418124,
          'Weighted F1': 0.8711329365026659,
          'Macro Precision': 0.8889860096031218,
          'Macro Recall': 0.833477012142767}
```

Section 7: Deep Learning Models

Deep learning models learn representations directly from sequences of word tokens, capturing semantic and positional information that TF-IDF misses.

We implement:

1. **BiLSTM** — Bidirectional Long Short-Term Memory: reads text in both directions, captures sequential dependencies
2. **TextCNN** — 1D Convolutional Neural Network with multiple filter sizes: captures local n-gram patterns
3. **BiLSTM + Attention** — Self-attention mechanism that focuses on the most discriminative tokens
4. **BERT** — Pre-trained transformer (distilbert-base-uncased), fine-tuned for sequence classification

All DL models use:

- Pre-trained GloVe-style or random embeddings (100-dim)
- Early stopping on validation loss
- ReduceLROnPlateau scheduler
- Dropout regularization

```
In [25]: # =====
# SECTION 7.1: Deep Learning – Sequence Preparation
# =====
```

```

if TF_AVAILABLE:
    MAX_VOCAB = 30000
    MAX_LEN = 64          # 95th percentile of token counts
    EMBED_DIM = 128

    tokenizer_d1 = Tokenizer(num_words=MAX_VOCAB, oov_token='<OOV>')
    tokenizer_d1.fit_on_texts(X_train_txt)

    def texts_to_padded(texts):
        seqs = tokenizer_d1.texts_to_sequences(texts)
        return pad_sequences(seqs, maxlen=MAX_LEN, padding='post', truncating='post')

    X_train_seq = texts_to_padded(X_train_txt)
    X_val_seq = texts_to_padded(X_val_txt)
    X_test_seq = texts_to_padded(X_test_txt)

    # One-hot encode labels
    y_train_cat = tf.keras.utils.to_categorical(y_train, num_classes=N_CLASSES)
    y_val_cat = tf.keras.utils.to_categorical(y_val, num_classes=N_CLASSES)
    y_test_cat = tf.keras.utils.to_categorical(y_test, num_classes=N_CLASSES)

    # --- Class weights to handle imbalance ---
    from sklearn.utils.class_weight import compute_class_weight
    class_weights_arr = compute_class_weight(
        'balanced', classes=np.unique(y_train), y=y_train
    )
    class_weight_dict = dict(enumerate(class_weights_arr))

    print(f"Vocab size: {len(tokenizer_d1.word_index):,}")
    print(f"Max sequence length: {MAX_LEN}")
    print(f"Train sequences: {X_train_seq.shape}")
    print(f"Class weights: {class_weight_dict}")
else:
    print("TensorFlow not available. Install with: pip install tensorflow")

```

Vocab size: 7,269

Max sequence length: 64

Train sequences: (13141, 64)

Class weights: {0: np.float64(1.3536258755665431), 1: np.float64(0.616600975975976), 2: np.float64(1.059074790457769), 3: np.float64(1.267457561728395), 4: np.float64(3.5325268817204303), 5: np.float64(0.616080637599625)}

7.2 Bidirectional LSTM (BiLSTM)

BiLSTM reads the token sequence in both forward and backward directions, allowing it to capture context from both sides of each word — critical for understanding how a Bloom's verb interacts with the surrounding noun phrase.

```

In [26]: # =====
# SECTION 7.2: Bidirectional LSTM
# =====

if TF_AVAILABLE:
    tf.random.set_seed(RANDOM_STATE)

    def build_bilstm(vocab_size, embed_dim, max_len, n_classes):
        inp = layers.Input(shape=(max_len,))
        x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(inp)
        x = layers.SpatialDropout1D(0.3)(x)
        x = layers.Bidirectional(layers.LSTM(128, return_sequences=True,
                                              dropout=0.2, recurrent_dropout=0.2))(x)

        x = layers.Bidirectional(layers.LSTM(64, dropout=0.2))(x)
        x = layers.Dense(128, activation='relu', kernel_regularizer=l2(1e-4))(x)
        x = layers.Dropout(0.4)(x)
        out = layers.Dense(n_classes, activation='softmax')(x)
        model = Model(inp, out)
        model.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

        return model

    bilstm = build_bilstm(MAX_VOCAB, EMBED_DIM, MAX_LEN, N_CLASSES)
    bilstm.summary()

    callbacks = [
        EarlyStopping(patience=5, restore_best_weights=True, monitor='val_loss'),
        ReduceLROnPlateau(factor=0.5, patience=3, monitor='val_loss', verbose=1)
    ]

    hist_bilstm = bilstm.fit(
        X_train_seq, y_train_cat,
        validation_data=(X_val_seq, y_val_cat),
        epochs=30, batch_size=64,
        class_weight=class_weight_dict,
        callbacks=callbacks, verbose=1
    )

    y_proba_bilstm = bilstm.predict(X_test_seq, verbose=0)
    y_pred_bilstm = y_proba_bilstm.argmax(axis=1)
    evaluate_model('BiLSTM', y_test, y_pred_bilstm, y_proba_bilstm, CLASS_NAMES)

    # Training curves
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle('BiLSTM Training Curves', fontweight='bold')

```

```

axes[0].plot(hist_bilstm.history['loss'], label='Train Loss')
axes[0].plot(hist_bilstm.history['val_loss'], label='Val Loss')
axes[0].set_title('Loss'); axes[0].legend()
axes[1].plot(hist_bilstm.history['accuracy'], label='Train Acc')
axes[1].plot(hist_bilstm.history['val_accuracy'], label='Val Acc')
axes[1].set_title('Accuracy'); axes[1].legend()
plt.tight_layout(); plt.show()
else:
    print("TensorFlow not available.")

```

```

I0000 00:00:1775682365.765645      55 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB memory:
-> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1775682365.772301      55 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:1 with 13757 MB memory:
-> device: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 64)	0	-
embedding (Embedding)	(None, 64, 128)	3,840,000	input_layer[0][0]
spatial_dropout1d (SpatialDropout1D)	(None, 64, 128)	0	embedding[0][0]
not_equal (NotEqual)	(None, 64)	0	input_layer[0][0]
bidirectional (Bidirectional)	(None, 64, 256)	263,168	spatial_dropout1... not_equal[0][0]
bidirectional_1 (Bidirectional)	(None, 128)	164,352	bidirectional[0]... not_equal[0][0]
dense (Dense)	(None, 128)	16,512	bidirectional_1[...]
dropout (Dropout)	(None, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 6)	774	dropout[0][0]

Total params: 4,284,806 (16.35 MB)

Trainable params: 4,284,806 (16.35 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/30

I0000 00:00:1775682380.004008 376 cuda_dnn.cc:529] Loaded cuDNN version 91002

206/206 ————— 88s 368ms/step - accuracy: 0.5115 - loss: 1.3207 - val_accuracy: 0.8565 - val_loss: 0.5090 - learning_rate: 0.0010
Epoch 2/30
206/206 ————— 72s 350ms/step - accuracy: 0.8773 - loss: 0.4433 - val_accuracy: 0.8839 - val_loss: 0.4237 - learning_rate: 0.0010
Epoch 3/30
206/206 ————— 72s 350ms/step - accuracy: 0.9123 - loss: 0.3104 - val_accuracy: 0.8832 - val_loss: 0.4478 - learning_rate: 0.0010
Epoch 4/30
206/206 ————— 73s 353ms/step - accuracy: 0.9346 - loss: 0.2274 - val_accuracy: 0.8917 - val_loss: 0.4271 - learning_rate: 0.0010
Epoch 5/30
206/206 ————— 0s 336ms/step - accuracy: 0.9492 - loss: 0.1785
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
206/206 ————— 72s 350ms/step - accuracy: 0.9492 - loss: 0.1785 - val_accuracy: 0.8892 - val_loss: 0.4662 - learning_rate: 0.0005
Epoch 6/30
206/206 ————— 72s 349ms/step - accuracy: 0.9616 - loss: 0.1374 - val_accuracy: 0.8956 - val_loss: 0.4638 - learning_rate: 5.0000e-04
Epoch 7/30
206/206 ————— 72s 349ms/step - accuracy: 0.9692 - loss: 0.1132 - val_accuracy: 0.8970 - val_loss: 0.4837 - learning_rate: 5.0000e-04

=====
MODEL: BiLSTM
=====

Accuracy : 0.8860
Balanced Accuracy : 0.8904
Macro Precision : 0.8569
Macro Recall : 0.8904
Macro F1 : 0.8707
Weighted F1 : 0.8870
MCC : 0.8578
Cohen Kappa : 0.8574
ROC-AUC (macro OvR) : 0.9800

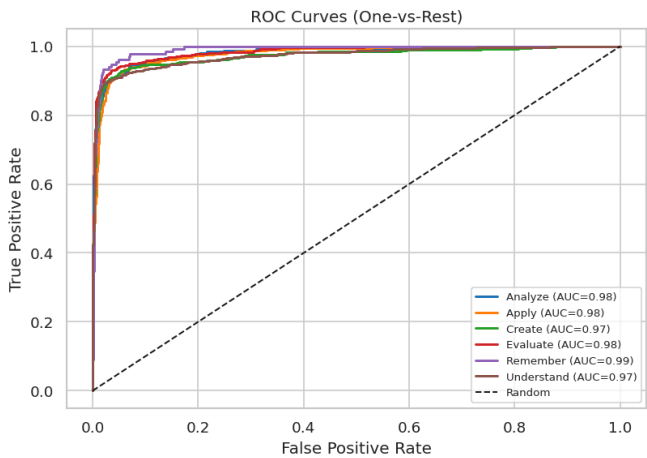
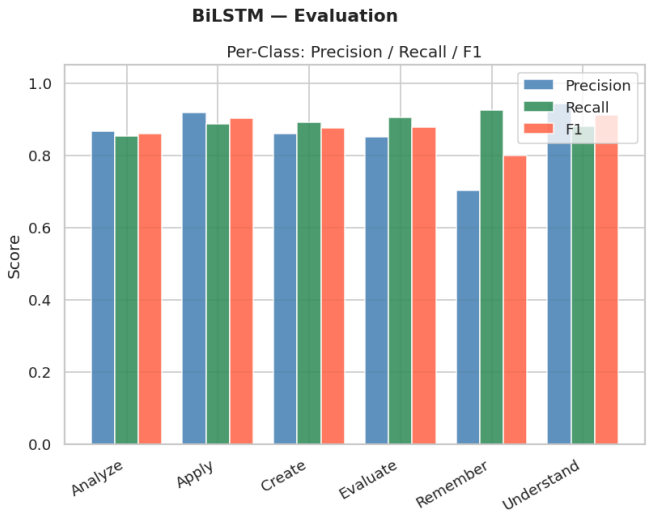
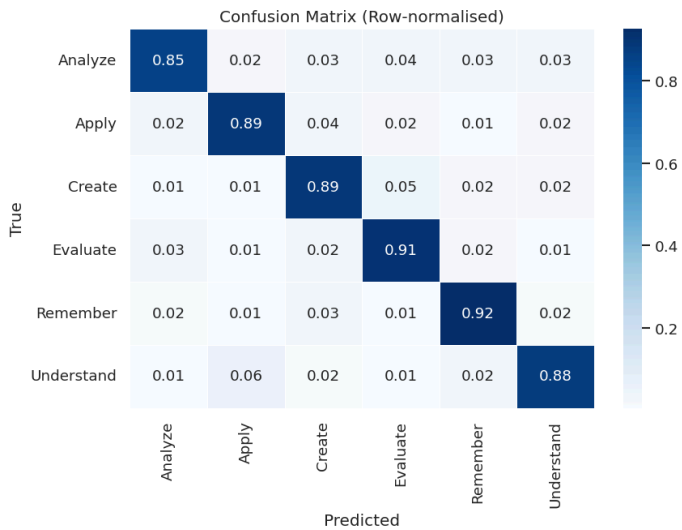
--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.87	0.85	0.86	347
Apply	0.92	0.89	0.90	761
Create	0.86	0.89	0.88	443
Evaluate	0.85	0.91	0.88	370
Remember	0.70	0.92	0.80	133
Understand	0.94	0.88	0.91	762

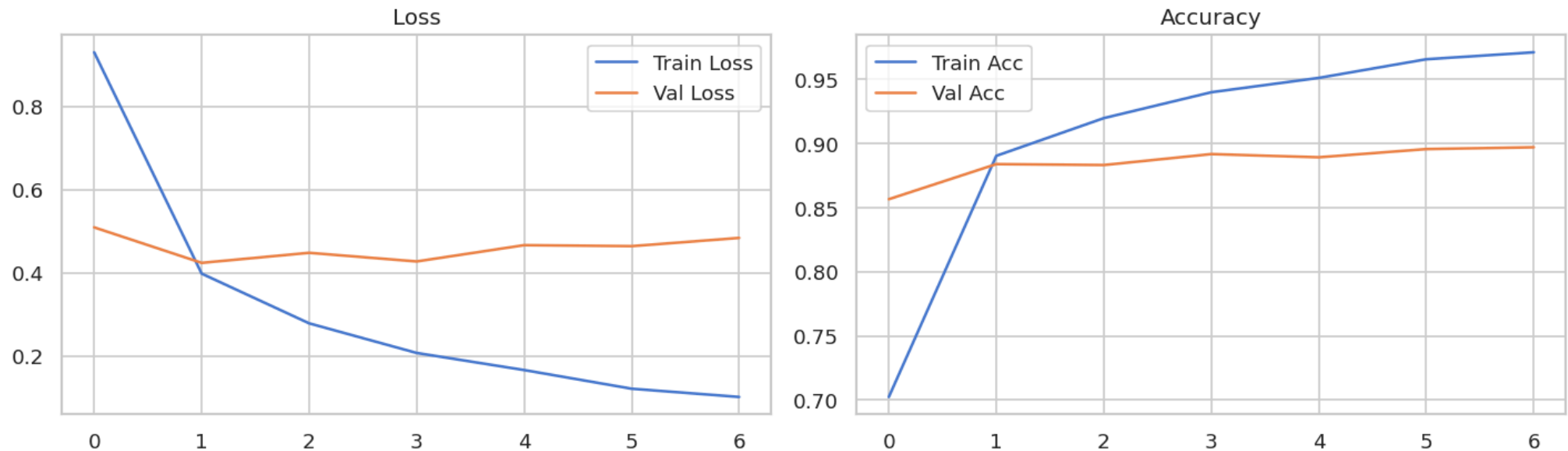
accuracy			0.89	2816
macro avg	0.86	0.89	0.87	2816
weighted avg	0.89	0.89	0.89	2816

--- Per-Class ROC-AUC ---

Analyze : 0.9821
Apply : 0.9779
Create : 0.9721
Evaluate : 0.9848
Remember : 0.9901
Understand : 0.9729



BiLSTM Training Curves



7.3 Text CNN (Kim, 2014)

TextCNN applies 1D convolutions with multiple kernel sizes (2, 3, 4, 5 words) in parallel. Each filter acts as a learned n-gram feature detector. The outputs are max-pooled and concatenated. This architecture is computationally cheap and highly effective for sentence classification.

```
In [27]: # =====  
# SECTION 7.3: Text CNN  
# =====  
  
if TF_AVAILABLE:  
    tf.random.set_seed(RANDOM_STATE)  
  
    def build_textcnn(vocab_size, embed_dim, max_len, n_classes,  
                      num_filters=128, kernel_sizes=(2, 3, 4, 5)):  
        inp = layers.Input(shape=(max_len,))  
        emb = layers.Embedding(vocab_size, embed_dim)(inp)  
        emb = layers.SpatialDropout1D(0.3)(emb)  
  
        pooled = []  
        for k in kernel_sizes:  
            conv = layers.Conv1D(num_filters, k, activation='relu',  
                                 padding='same', kernel_regularizer=l2(1e-4))(emb)  
            pool = layers.GlobalMaxPooling1D()(conv)  
            pooled.append(pool)
```

```

x = layers.Concatenate()(pooled)
x = layers.Dropout(0.4)(x)
x = layers.Dense(128, activation='relu', kernel_regularizer=l2(1e-4))(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(n_classes, activation='softmax')(x)

model = Model(inp, out)
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
return model

textcnn = build_textcnn(MAX_VOCAB, EMBED_DIM, MAX_LEN, N_CLASSES)
textcnn.summary()

callbacks_cnn = [
    EarlyStopping(patience=5, restore_best_weights=True, monitor='val_loss'),
    ReduceLROnPlateau(factor=0.5, patience=3, monitor='val_loss', verbose=1)
]

hist_cnn = textcnn.fit(
    X_train_seq, y_train_cat,
    validation_data=(X_val_seq, y_val_cat),
    epochs=30, batch_size=64,
    class_weight=class_weight_dict,
    callbacks=callbacks_cnn, verbose=1
)

y_proba_cnn = textcnn.predict(X_test_seq, verbose=0)
y_pred_cnn = y_proba_cnn.argmax(axis=1)
evaluate_model('TextCNN', y_test, y_pred_cnn, y_proba_cnn, CLASS_NAMES)

# Training curves
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fig.suptitle('TextCNN Training Curves', fontweight='bold')
axes[0].plot(hist_cnn.history['loss'], label='Train'); axes[0].plot(hist_cnn.history['val_loss'], label='Val')
axes[0].set_title('Loss'); axes[0].legend()
axes[1].plot(hist_cnn.history['accuracy'], label='Train'); axes[1].plot(hist_cnn.history['val_accuracy'], label='Val')
axes[1].set_title('Accuracy'); axes[1].legend()
plt.tight_layout(); plt.show()
else:
    print("TensorFlow not available.")

```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None , 64)	0	-
embedding_1 (Embedding)	(None , 64, 128)	3,840,000	input_layer_1[0]...
spatial_dropout1d_1 (SpatialDropout1D)	(None , 64, 128)	0	embedding_1[0][0]
conv1d (Conv1D)	(None , 64, 128)	32,896	spatial_dropout1...
conv1d_1 (Conv1D)	(None , 64, 128)	49,280	spatial_dropout1...
conv1d_2 (Conv1D)	(None , 64, 128)	65,664	spatial_dropout1...
conv1d_3 (Conv1D)	(None , 64, 128)	82,048	spatial_dropout1...
global_max_pooling... (GlobalMaxPooling1D)	(None , 128)	0	conv1d[0][0]
global_max_pooling... (GlobalMaxPooling1D)	(None , 128)	0	conv1d_1[0][0]
global_max_pooling... (GlobalMaxPooling1D)	(None , 128)	0	conv1d_2[0][0]
global_max_pooling... (GlobalMaxPooling1D)	(None , 128)	0	conv1d_3[0][0]
concatenate (Concatenate)	(None , 512)	0	global_max_pooli... global_max_pooli... global_max_pooli... global_max_pooli...
dropout_1 (Dropout)	(None , 512)	0	concatenate[0][0]
dense_2 (Dense)	(None , 128)	65,664	dropout_1[0][0]
dropout_2 (Dropout)	(None , 128)	0	dense_2[0][0]
dense_3 (Dense)	(None , 6)	774	dropout_2[0][0]

Total params: 4,136,326 (15.78 MB)

Trainable params: 4,136,326 (15.78 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/30

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1775682898.632699 375 service.cc:152] XLA service 0x7fe4ece50870 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:

I0000 00:00:1775682898.632749 375 service.cc:160] StreamExecutor device (0): Tesla T4, Compute Capability 7.5

I0000 00:00:1775682898.632754 375 service.cc:160] StreamExecutor device (1): Tesla T4, Compute Capability 7.5

2026-04-08 21:15:00.946288: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

2026-04-08 21:15:01.168420: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

2026-04-08 21:15:01.776979: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

2026-04-08 21:15:01.998602: E external/local_xla/xla/stream_executor/cuda/cuda_timer.cc:86] Delay kernel timed out: measured time has sub-optimal accuracy. There may be a missing warmup execution, please investigate in Nsight Systems.

19/206 ————— 1s 9ms/step - accuracy: 0.1275 - loss: 1.8702

I0000 00:00:1775682905.817905 375 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

206/206  16s 33ms/step - accuracy: 0.3969 - loss: 1.5058 - val_accuracy: 0.8661 - val_loss: 0.5251 - learning_rate: 0.0010
Epoch 2/30
206/206  1s 6ms/step - accuracy: 0.8717 - loss: 0.4656 - val_accuracy: 0.8920 - val_loss: 0.4302 - learning_rate: 0.0010
Epoch 3/30
206/206  1s 6ms/step - accuracy: 0.9112 - loss: 0.3388 - val_accuracy: 0.9002 - val_loss: 0.4081 - learning_rate: 0.0010
Epoch 4/30
206/206  1s 6ms/step - accuracy: 0.9397 - loss: 0.2542 - val_accuracy: 0.9034 - val_loss: 0.4060 - learning_rate: 0.0010
Epoch 5/30
206/206  1s 6ms/step - accuracy: 0.9606 - loss: 0.1949 - val_accuracy: 0.9006 - val_loss: 0.4341 - learning_rate: 0.0010
Epoch 6/30
206/206  1s 6ms/step - accuracy: 0.9710 - loss: 0.1601 - val_accuracy: 0.9038 - val_loss: 0.4421 - learning_rate: 0.0010
Epoch 7/30
201/206  0s 5ms/step - accuracy: 0.9769 - loss: 0.1345
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
206/206  1s 6ms/step - accuracy: 0.9770 - loss: 0.1344 - val_accuracy: 0.9013 - val_loss: 0.4714 - learning_rate: 0.0010
Epoch 8/30
206/206  1s 6ms/step - accuracy: 0.9820 - loss: 0.1243 - val_accuracy: 0.8995 - val_loss: 0.4830 - learning_rate: 5.0000e-04
Epoch 9/30
206/206  1s 6ms/step - accuracy: 0.9870 - loss: 0.1046 - val_accuracy: 0.9013 - val_loss: 0.4928 - learning_rate: 5.0000e-04

=====

MODEL: TextCNN

=====

Accuracy	: 0.8956
Balanced Accuracy	: 0.8976
Macro Precision	: 0.8712
Macro Recall	: 0.8976
Macro F1	: 0.8824
Weighted F1	: 0.8963
MCC	: 0.8694
Cohen Kappa	: 0.8692
ROC-AUC (macro OvR)	: 0.9873

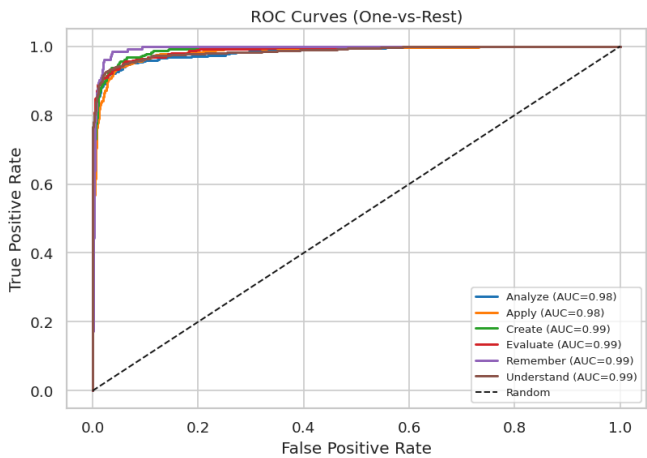
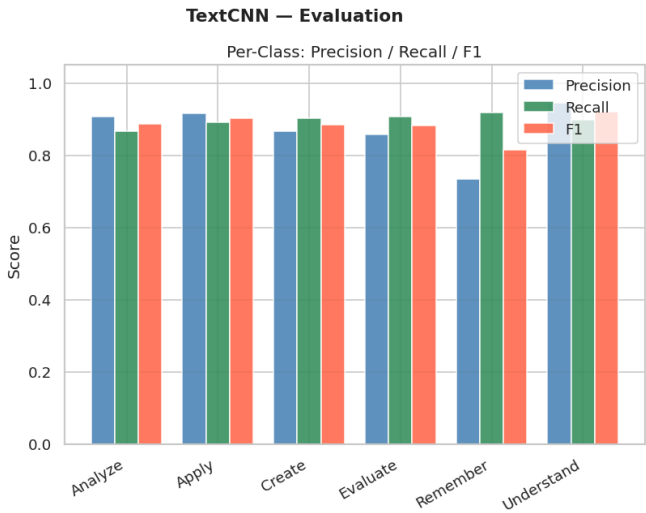
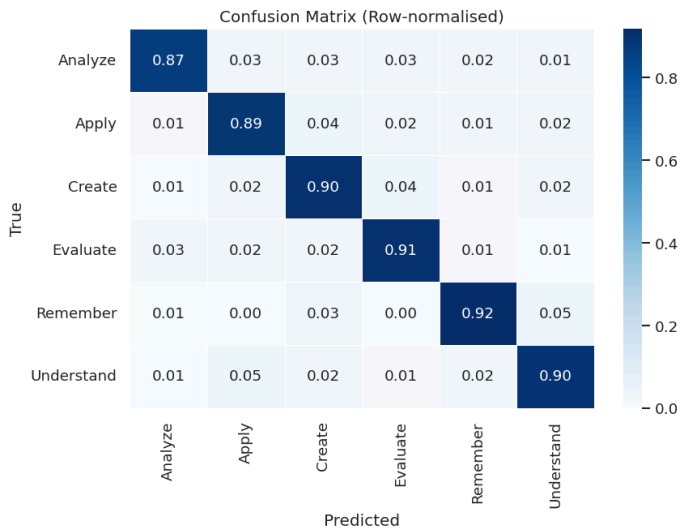
--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.91	0.87	0.89	347
Apply	0.92	0.89	0.90	761
Create	0.87	0.90	0.88	443
Evaluate	0.86	0.91	0.88	370
Remember	0.73	0.92	0.82	133
Understand	0.94	0.90	0.92	762

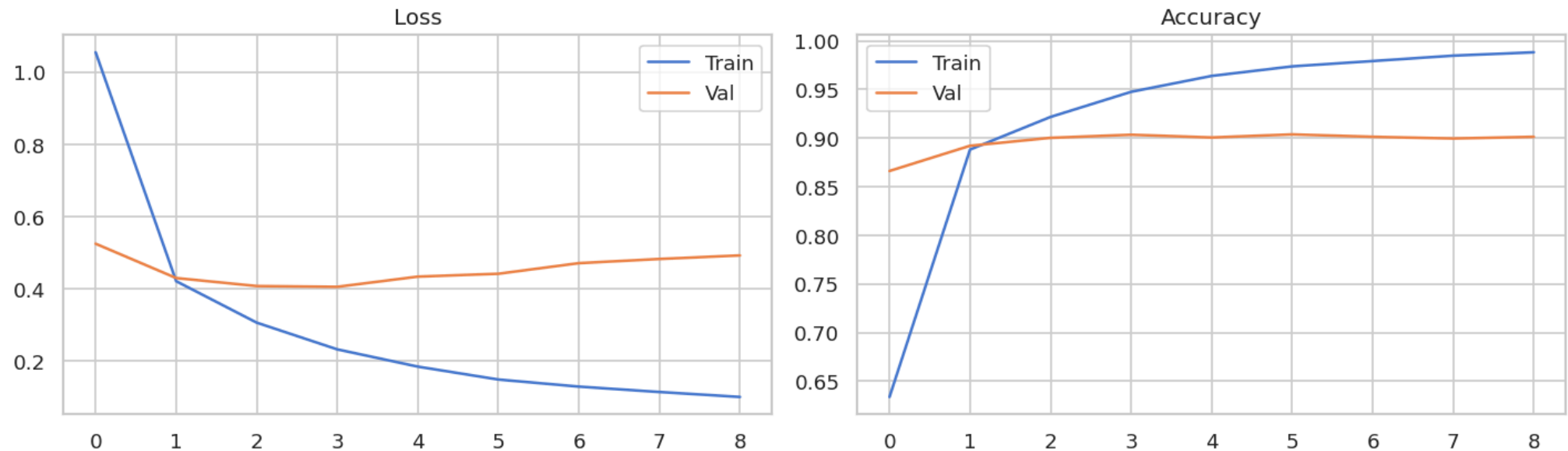
accuracy			0.90	2816
macro avg	0.87	0.90	0.88	2816
weighted avg	0.90	0.90	0.90	2816

--- Per-Class ROC-AUC ---

Analyze : 0.9827
Apply : 0.9833
Create : 0.9902
Evaluate : 0.9879
Remember : 0.9945
Understand : 0.9854



TextCNN Training Curves



7.4 BiLSTM with Self-Attention

We augment the BiLSTM with a **self-attention layer** that computes importance weights over all token positions, focusing on the most discriminative terms (especially the Bloom's level verbs). The attention scores are also interpretable — we can visualize which tokens drove the prediction.

```
In [28]: # =====  
# SECTION 7.4: BiLSTM + Self-Attention  
# =====  
  
if TF_AVAILABLE:  
    tf.random.set_seed(RANDOM_STATE)  
  
    class AttentionLayer(layers.Layer):  
        """Additive (Bahdanau-style) self-attention."""  
        def __init__(self, units=64, **kwargs):  
            super().__init__(**kwargs)  
            self.W = layers.Dense(units)  
            self.V = layers.Dense(1)  
  
        def call(self, hidden_states):  
            score = self.V(tf.nn.tanh(self.W(hidden_states))) # (B, T, 1)  
            weights = tf.nn.softmax(score, axis=1) # (B, T, 1)  
            context = tf.reduce_sum(weights * hidden_states, axis=1) # (B, H)  
            return context, weights
```

```

def build_bilstm_attention(vocab_size, embed_dim, max_len, n_classes):
    inp = layers.Input(shape=(max_len,))
    x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(inp)
    x = layers.SpatialDropout1D(0.3)(x)
    x = layers.Bidirectional(layers.LSTM(128, return_sequences=True,
                                         dropout=0.2, recurrent_dropout=0.2))(x)

    context, attn_weights = AttentionLayer(64)(x)
    x = layers.Dense(128, activation='relu', kernel_regularizer=l2(1e-4))(context)
    x = layers.Dropout(0.4)(x)
    out = layers.Dense(n_classes, activation='softmax')(x)
    model = Model(inp, out)
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

bilstm_attn = build_bilstm_attention(MAX_VOCAB, EMBED_DIM, MAX_LEN, N_CLASSES)
bilstm_attn.summary()

callbacks_attn = [
    EarlyStopping(patience=5, restore_best_weights=True, monitor='val_loss'),
    ReduceLROnPlateau(factor=0.5, patience=3, monitor='val_loss', verbose=1)
]

hist_attn = bilstm_attn.fit(
    X_train_seq, y_train_cat,
    validation_data=(X_val_seq, y_val_cat),
    epochs=30, batch_size=64,
    class_weight=class_weight_dict,
    callbacks=callbacks_attn, verbose=1
)

y_proba_attn = bilstm_attn.predict(X_test_seq, verbose=0)
y_pred_attn = y_proba_attn.argmax(axis=1)
evaluate_model('BiLSTM + Self-Attention', y_test, y_pred_attn, y_proba_attn, CLASS_NAMES)

# Training curves
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fig.suptitle('BiLSTM+Attention Training Curves', fontweight='bold')
axes[0].plot(hist_attn.history['loss'], label='Train'); axes[0].plot(hist_attn.history['val_loss'], label='Val')
axes[0].set_title('Loss'); axes[0].legend()
axes[1].plot(hist_attn.history['accuracy'], label='Train'); axes[1].plot(hist_attn.history['val_accuracy'], label='Val')
axes[1].set_title('Accuracy'); axes[1].legend()
plt.tight_layout(); plt.show()
else:
    print("TensorFlow not available.")

```


Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 64)	0	-
embedding_2 (Embedding)	(None, 64, 128)	3,840,000	input_layer_2[0]...
spatial_dropout1d_2 (SpatialDropout1D)	(None, 64, 128)	0	embedding_2[0][0]
not_equal_1 (NotEqual)	(None, 64)	0	input_layer_2[0]...
bidirectional_2 (Bidirectional)	(None, 64, 256)	263,168	spatial_dropout1... not_equal_1[0][0]
attention_layer (AttentionLayer)	[(None, 256), (None, 64, 1)]	16,513	bidirectional_2[...]
dense_6 (Dense)	(None, 128)	32,896	attention_layer[...]
dropout_3 (Dropout)	(None, 128)	0	dense_6[0][0]
dense_7 (Dense)	(None, 6)	774	dropout_3[0][0]

Total params: 4,153,351 (15.84 MB)

Trainable params: 4,153,351 (15.84 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/30
206/206 ————— 85s 372ms/step - accuracy: 0.4086 - loss: 1.4351 - val_accuracy: 0.8555 - val_loss: 0.5262 - learning_rate: 0.0010
Epoch 2/30
206/206 ————— 76s 368ms/step - accuracy: 0.8775 - loss: 0.4491 - val_accuracy: 0.8746 - val_loss: 0.4379 - learning_rate: 0.0010
Epoch 3/30
206/206 ————— 77s 373ms/step - accuracy: 0.9121 - loss: 0.2997 - val_accuracy: 0.8821 - val_loss: 0.4420 - learning_rate: 0.0010
Epoch 4/30
206/206 ————— 76s 369ms/step - accuracy: 0.9310 - loss: 0.2299 - val_accuracy: 0.8821 - val_loss: 0.4536 - learning_rate: 0.0010
Epoch 5/30
206/206 ————— 77s 372ms/step - accuracy: 0.9437 - loss: 0.1919 - val_accuracy: 0.8874 - val_loss: 0.4336 - learning_rate: 0.0010
Epoch 6/30
206/206 ————— 76s 368ms/step - accuracy: 0.9553 - loss: 0.1548 - val_accuracy: 0.8853 - val_loss: 0.4648 - learning_rate: 0.0010
Epoch 7/30
206/206 ————— 84s 409ms/step - accuracy: 0.9654 - loss: 0.1217 - val_accuracy: 0.8839 - val_loss: 0.4879 - learning_rate: 0.0010
Epoch 8/30
206/206 ————— 0s 367ms/step - accuracy: 0.9662 - loss: 0.1110
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
206/206 ————— 78s 381ms/step - accuracy: 0.9662 - loss: 0.1110 - val_accuracy: 0.8857 - val_loss: 0.4883 - learning_rate: 0.0010
Epoch 9/30
206/206 ————— 76s 369ms/step - accuracy: 0.9716 - loss: 0.0997 - val_accuracy: 0.8867 - val_loss: 0.5075 - learning_rate: 5.0000e-04
Epoch 10/30
206/206 ————— 75s 365ms/step - accuracy: 0.9778 - loss: 0.0794 - val_accuracy: 0.8878 - val_loss: 0.5248 - learning_rate: 5.0000e-04

```
=====
MODEL: BiLSTM + Self-Attention
=====
Accuracy                : 0.8903
Balanced Accuracy       : 0.8867
Macro Precision         : 0.8687
Macro Recall            : 0.8867
Macro F1                : 0.8765
Weighted F1             : 0.8907
MCC                     : 0.8625
Cohen Kappa             : 0.8623
ROC-AUC (macro OvR)    : 0.9825
```

--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.89	0.84	0.86	347
Apply	0.90	0.91	0.91	761
Create	0.86	0.90	0.88	443
Evaluate	0.88	0.91	0.89	370
Remember	0.75	0.88	0.81	133
Understand	0.93	0.88	0.91	762
accuracy			0.89	2816
macro avg	0.87	0.89	0.88	2816
weighted avg	0.89	0.89	0.89	2816

--- Per-Class ROC-AUC ---

Analyze : 0.9785

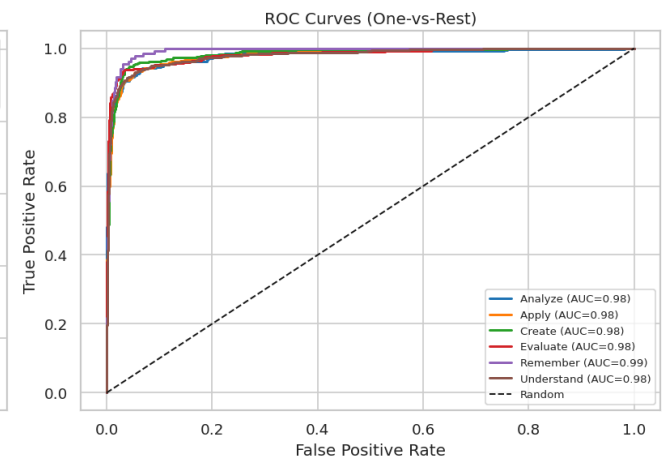
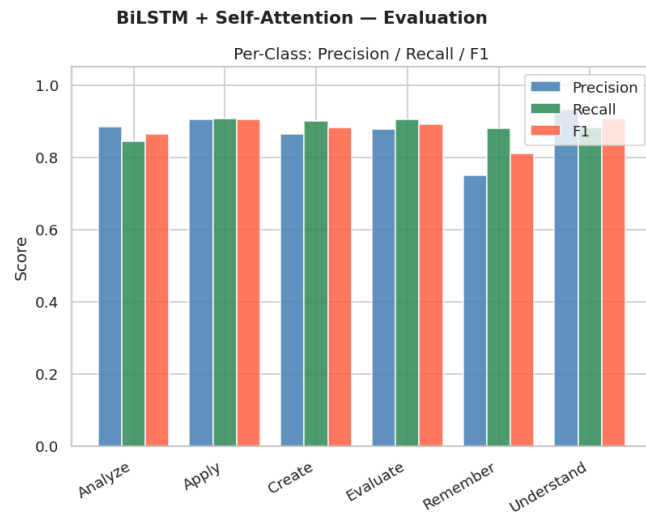
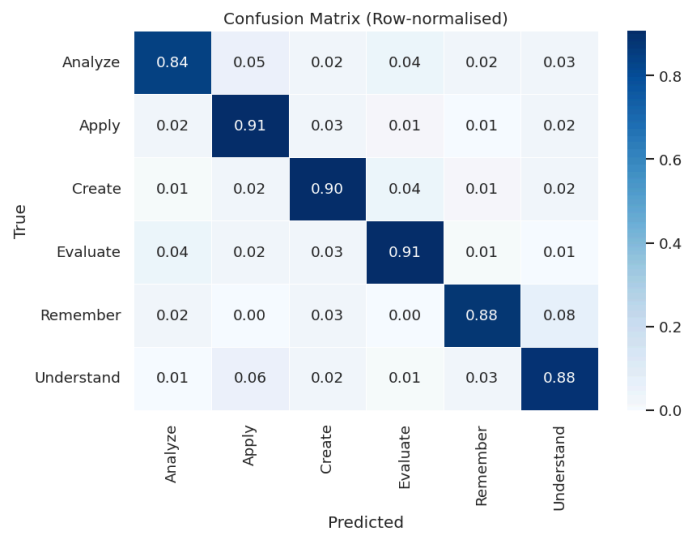
Apply : 0.9796

Create : 0.9842

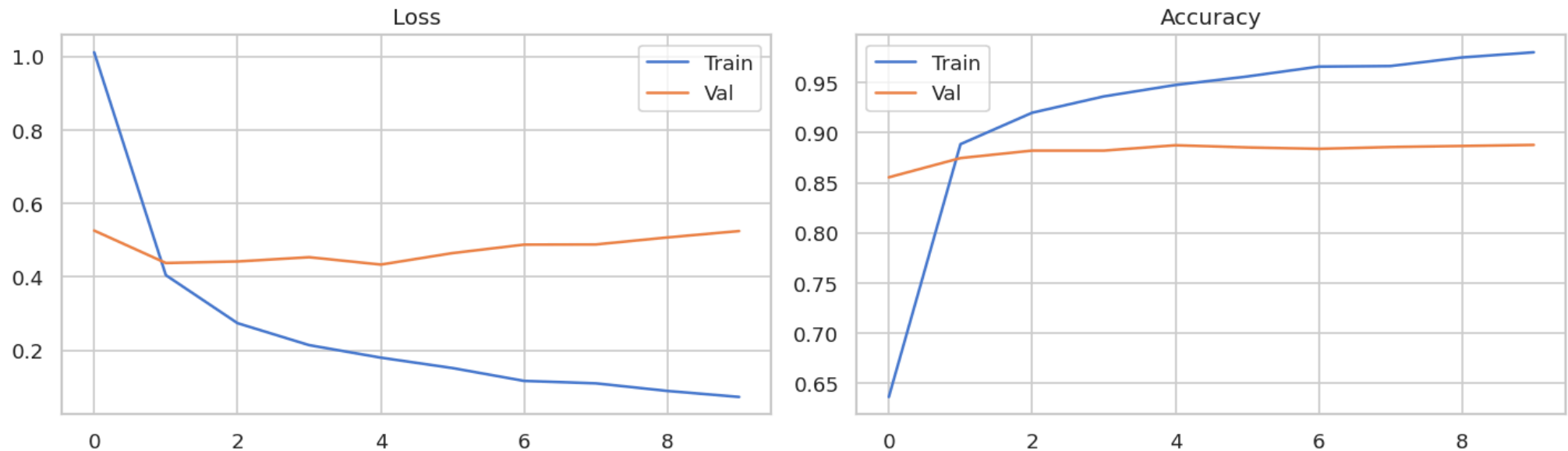
Evaluate : 0.9817

Remember : 0.9923

Understand : 0.9790



BiLSTM+Attention Training Curves



7.5 BERT Fine-Tuning (DistilBERT)

DistilBERT is a smaller, faster version of BERT that retains ~97% of BERT's performance at 60% of the size. Fine-tuning a pre-trained transformer is the **state-of-the-art** approach for NLP classification tasks.

The model leverages pre-trained contextual representations from 3 billion tokens of Wikipedia + BookCorpus, then fine-tunes on our Bloom's Taxonomy dataset. The `[CLS]` token embedding is passed to a classification head.

Note: Requires `transformers` + `torch` libraries. On CPU, this may be slow. For faster execution, use a GPU runtime (Colab T4 or local GPU).

```
In [29]: # =====
# SECTION 7.5: BERT / DistilBERT Fine-Tuning
# =====

if BERT_AVAILABLE:
    from torch.utils.data import Dataset as TorchDataset, DataLoader
    import torch.nn as nn
    from transformers import DistilBertTokenizerFast, DistilBertForSequenceClassification
    from transformers import get_linear_schedule_with_warmup
    from torch.optim import AdamW

    BERT_MODEL = 'distilbert-base-uncased'
    BERT_MAX_LEN = 64
```

```
BERT_BATCH = 32
BERT_EPOCHS = 5
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"BERT device: {DEVICE}")
```

```
bert_tokenizer = DistilBertTokenizerFast.from_pretrained(BERT_MODEL)
```

```
class BloomsDataset(TorchDataset):
    def __init__(self, texts, labels):
        self.enc = bert_tokenizer(
            list(texts), truncation=True, padding='max_length',
            max_length=BERT_MAX_LEN, return_tensors='pt'
        )
        self.labels = torch.tensor(labels, dtype=torch.long)
    def __len__(self):
        return len(self.labels)
    def __getitem__(self, idx):
        return {
            'input_ids': self.enc['input_ids'][idx],
            'attention_mask': self.enc['attention_mask'][idx],
            'labels': self.labels[idx]
        }
```

```
train_ds = BloomsDataset(X_train_txt, y_train)
val_ds = BloomsDataset(X_val_txt, y_val)
test_ds = BloomsDataset(X_test_txt, y_test)
```

```
train_dl = DataLoader(train_ds, batch_size=BERT_BATCH, shuffle=True)
val_dl = DataLoader(val_ds, batch_size=BERT_BATCH)
test_dl = DataLoader(test_ds, batch_size=BERT_BATCH)
```

```
bert_model = DistilBertForSequenceClassification.from_pretrained(
    BERT_MODEL, num_labels=N_CLASSES
).to(DEVICE)
```

```
optimizer = AdamW(bert_model.parameters(), lr=2e-5, weight_decay=0.01)
total_steps = len(train_dl) * BERT_EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer, num_warmup_steps=total_steps//10,
    num_training_steps=total_steps
)
```

```
def train_epoch(model, dl, opt, sched):
    model.train()
    total_loss = 0
    for batch in dl:
```

```

    opt.zero_grad()
    out = model(input_ids=batch['input_ids'].to(DEVICE),
                attention_mask=batch['attention_mask'].to(DEVICE),
                labels=batch['labels'].to(DEVICE))
    out.loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); sched.step()
    total_loss += out.loss.item()
    return total_loss / len(dl)

def eval_epoch(model, dl):
    model.eval()
    all_preds, all_proba = [], []
    with torch.no_grad():
        for batch in dl:
            out = model(input_ids=batch['input_ids'].to(DEVICE),
                        attention_mask=batch['attention_mask'].to(DEVICE))
            proba = torch.softmax(out.logits, dim=-1).cpu().numpy()
            all_proba.append(proba)
            all_preds.append(proba.argmax(axis=1))
    return np.concatenate(all_preds), np.vstack(all_proba)

best_val_f1 = 0
for epoch in range(1, BERT_EPOCHS + 1):
    train_loss = train_epoch(bert_model, train_dl, optimizer, scheduler)
    val_preds, val_proba = eval_epoch(bert_model, val_dl)
    val_f1 = f1_score(y_val, val_preds, average='macro', zero_division=0)
    print(f"Epoch {epoch}/{BERT_EPOCHS} | Train Loss: {train_loss:.4f} | Val Macro F1: {val_f1:.4f}")

y_pred_bert, y_proba_bert = eval_epoch(bert_model, test_dl)
evaluate_model('DistilBERT (fine-tuned)', y_test, y_pred_bert, y_proba_bert, CLASS_NAMES)

else:
    print("Transformers/PyTorch not available. Install with:")
    print("  pip install transformers torch")
    print()
    print("For Google Colab with GPU, this section will run automatically.")

```

BERT device: cuda

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```

tokenizer_config.json:  0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
config.json:  0%|          | 0.00/483 [00:00<?, ?B/s]
model.safetensors:  0%|          | 0.00/268M [00:00<?, ?B/s]
Loading weights:  0%|          | 0/100 [00:00<?, ?it/s]

```

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
-----+-----	
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING
pre_classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/5 | Train Loss: 0.8634 | Val Macro F1: 0.9023
Epoch 2/5 | Train Loss: 0.2801 | Val Macro F1: 0.9130
Epoch 3/5 | Train Loss: 0.1877 | Val Macro F1: 0.9196
Epoch 4/5 | Train Loss: 0.1377 | Val Macro F1: 0.9226
Epoch 5/5 | Train Loss: 0.1092 | Val Macro F1: 0.9266

=====

MODEL: DistilBERT (fine-tuned)

=====

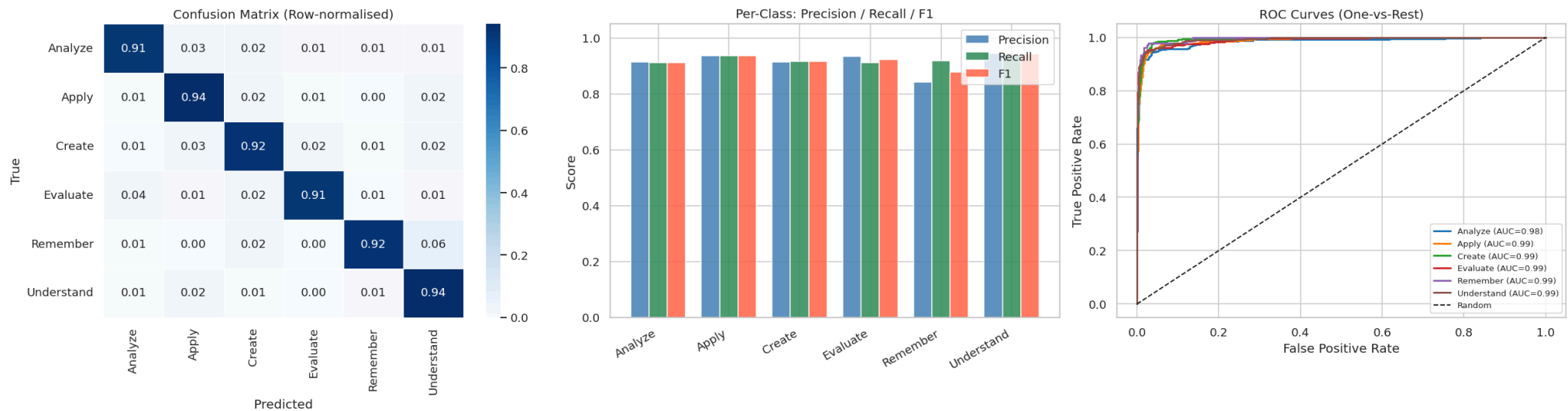
Accuracy : 0.9272
Balanced Accuracy : 0.9222
Macro Precision : 0.9140
Macro Recall : 0.9222
Macro F1 : 0.9178
Weighted F1 : 0.9273
MCC : 0.9084
Cohen Kappa : 0.9084
ROC-AUC (macro OvR) : 0.9907

--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.91	0.91	0.91	347
Apply	0.94	0.94	0.94	761
Create	0.91	0.92	0.92	443
Evaluate	0.93	0.91	0.92	370
Remember	0.84	0.92	0.88	133
Understand	0.95	0.94	0.94	762
accuracy			0.93	2816
macro avg	0.91	0.92	0.92	2816
weighted avg	0.93	0.93	0.93	2816

--- Per-Class ROC-AUC ---

Analyze : 0.9848
Apply : 0.9885
Create : 0.9939
Evaluate : 0.9903
Remember : 0.9947
Understand : 0.9921



Section 8: Ensemble Methods

We combine the top-performing models using:

1. **Soft Voting** — Average predicted class probabilities
2. **Stacking** — Meta-learner (Logistic Regression) trained on out-of-fold predictions

Ensembles typically outperform individual models by reducing variance.

```
In [30]: # =====
# SECTION 8: Ensemble – Soft Voting
# =====

# Combine LR + SVM + CNB probabilities (all use TF-IDF features)
proba_list = [y_proba_lr, y_proba_svm, y_proba_cnb]
y_proba_vote = np.mean(proba_list, axis=0)
y_pred_vote = y_proba_vote.argmax(axis=1)
evaluate_model('Soft Voting (LR + SVM + CNB)',
               y_test, y_pred_vote, y_proba_vote, CLASS_NAMES)

# If DL models are available, include them
if TF_AVAILABLE:
    proba_list_dl = [y_proba_lr, y_proba_svm, y_proba_bilstm, y_proba_cnn]
    y_proba_vote_dl = np.mean(proba_list_dl, axis=0)
```

```

y_pred_vote_d1 = y_proba_vote_d1.argmax(axis=1)
evaluate_model('Soft Voting (LR + SVM + BiLSTM + CNN)',
               y_test, y_pred_vote_d1, y_proba_vote_d1, CLASS_NAMES)

# =====
# Stacking with meta-Learner
# =====
from sklearn.ensemble import StackingClassifier

stack_estimators = [
    ('lr', LogisticRegression(C=5.0, max_iter=500, solver='lbfgs')),
    ('svm', CalibratedClassifierCV(LinearSVC(C=1.0, max_iter=2000), cv=3)),
    ('cnb', ComplementNB(alpha=0.1)),
    ('rf', RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=RANDOM_STATE))
]
meta_lr = LogisticRegression(C=1.0, max_iter=500)

stacker = StackingClassifier(
    estimators=stack_estimators,
    final_estimator=meta_lr,
    passthrough=False, cv=5, n_jobs=-1
)
print("Training stacking classifier (5-fold CV)...")
stacker.fit(X_train_tfidf, y_train)
y_pred_stack = stacker.predict(X_test_tfidf)
y_proba_stack = stacker.predict_proba(X_test_tfidf)
evaluate_model('Stacking Ensemble (LR+SVM+CNB+RF → LR)',
               y_test, y_pred_stack, y_proba_stack, CLASS_NAMES)

```

```

=====
MODEL: Soft Voting (LR + SVM + CNB)
=====

Accuracy           : 0.8871
Balanced Accuracy   : 0.8729
Macro Precision     : 0.8758
Macro Recall        : 0.8729
Macro F1            : 0.8739
Weighted F1         : 0.8869
MCC                 : 0.8577
Cohen Kappa         : 0.8576
ROC-AUC (macro OvR) : 0.9813

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

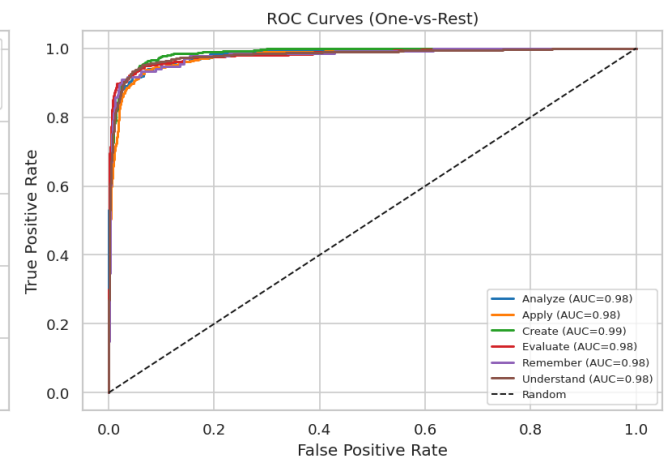
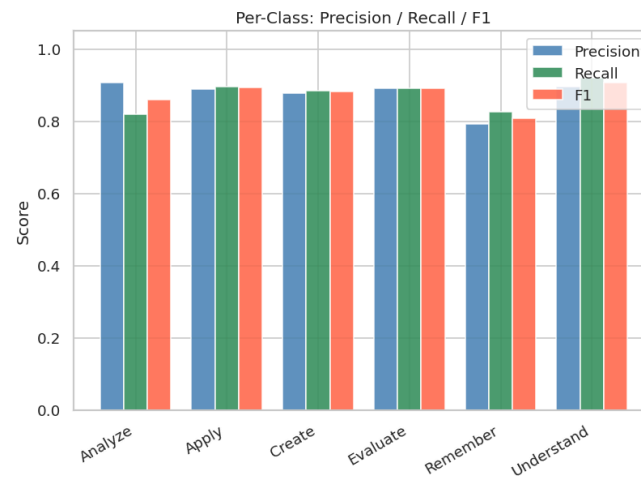
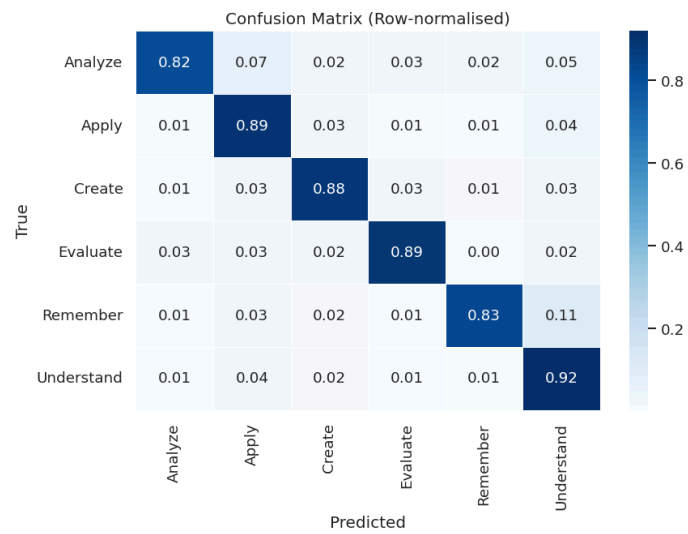
   Analyze       0.91      0.82      0.86       347
     Apply       0.89      0.89      0.89       761
      Create       0.88      0.88      0.88       443
    Evaluate       0.89      0.89      0.89       370
    Remember       0.79      0.83      0.81       133
   Understand     0.90      0.92      0.91       762

 accuracy          0.89      0.89      0.89      2816
 macro avg         0.88      0.87      0.87      2816
weighted avg         0.89      0.89      0.89      2816

--- Per-Class ROC-AUC ---
Analyze           : 0.9814
Apply              : 0.9777
Create             : 0.9873
Evaluate           : 0.9817
Remember           : 0.9797
Understand         : 0.9801

```

Soft Voting (LR + SVM + CNB) — Evaluation



```

=====
MODEL: Soft Voting (LR + SVM + BiLSTM + CNN)
=====

Accuracy          : 0.9041
Balanced Accuracy  : 0.8997
Macro Precision    : 0.8858
Macro Recall       : 0.8997
Macro F1           : 0.8920
Weighted F1        : 0.9044
MCC                : 0.8797
Cohen Kappa        : 0.8796
ROC-AUC (macro OvR) : 0.9884

--- Per-Class Classification Report ---
      precision    recall  f1-score   support

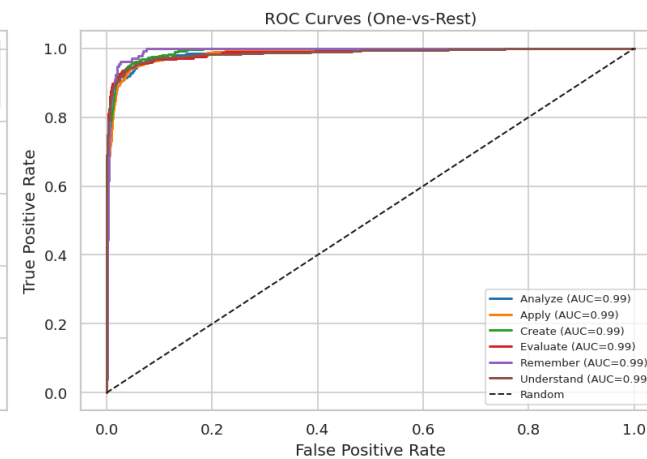
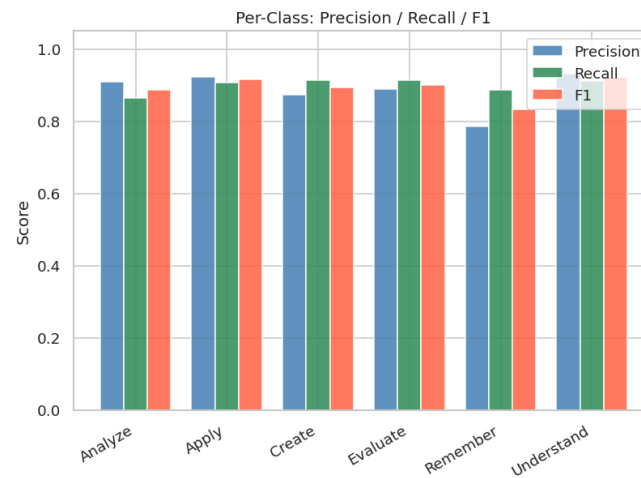
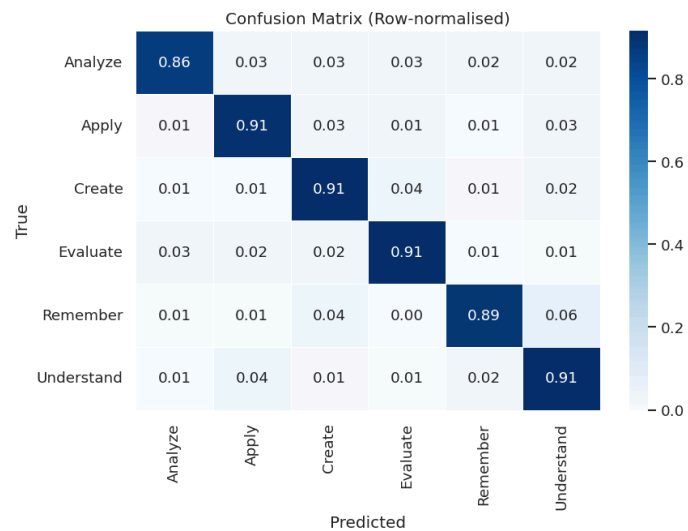
   Analyze       0.91      0.86      0.89       347
     Apply       0.92      0.91      0.92       761
      Create       0.87      0.91      0.89       443
    Evaluate       0.89      0.91      0.90       370
    Remember       0.79      0.89      0.83       133
   Understand     0.93      0.91      0.92       762

 accuracy          0.90      0.90      0.90      2816
 macro avg         0.89      0.90      0.89      2816
weighted avg         0.91      0.90      0.90      2816

--- Per-Class ROC-AUC ---
Analyze          : 0.9871
Apply            : 0.9862
Create           : 0.9914
Evaluate         : 0.9867
Remember         : 0.9935
Understand       : 0.9856

```

Soft Voting (LR + SVM + BiLSTM + CNN) — Evaluation



Training stacking classifier (5-fold CV)...

=====

MODEL: Stacking Ensemble (LR+SVM+CNB+RF → LR)

=====

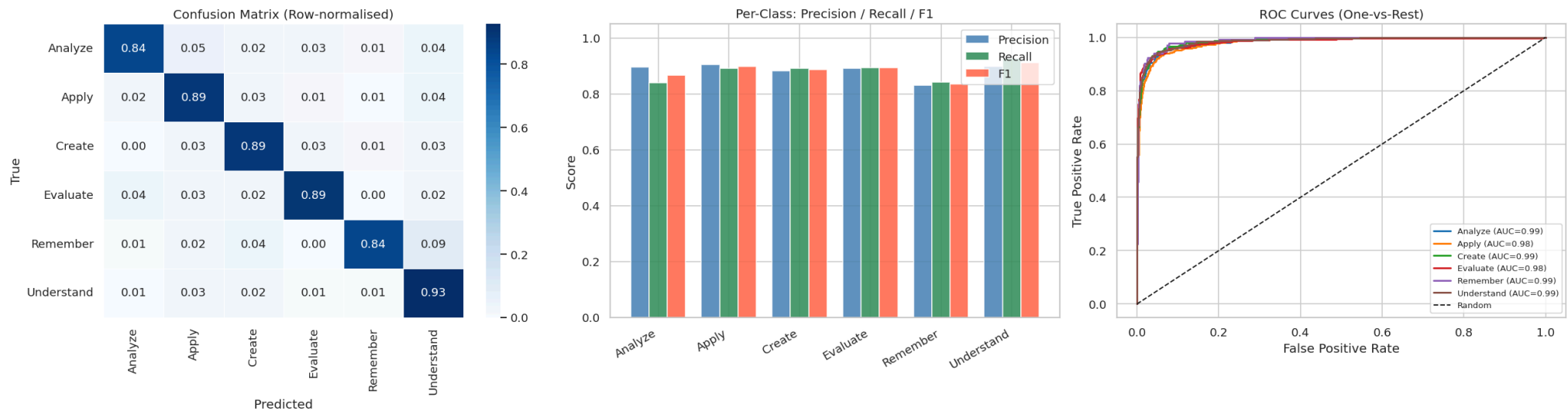
Accuracy : 0.8928
Balanced Accuracy : 0.8810
Macro Precision : 0.8835
Macro Recall : 0.8810
Macro F1 : 0.8820
Weighted F1 : 0.8926
MCC : 0.8649
Cohen Kappa : 0.8649
ROC-AUC (macro OvR) : 0.9853

--- Per-Class Classification Report ---

	precision	recall	f1-score	support
Analyze	0.90	0.84	0.87	347
Apply	0.90	0.89	0.90	761
Create	0.88	0.89	0.89	443
Evaluate	0.89	0.89	0.89	370
Remember	0.83	0.84	0.84	133
Understand	0.90	0.93	0.91	762
accuracy			0.89	2816
macro avg	0.88	0.88	0.88	2816
weighted avg	0.89	0.89	0.89	2816

--- Per-Class ROC-AUC ---

Analyze : 0.9852
Apply : 0.9813
Create : 0.9867
Evaluate : 0.9839
Remember : 0.9888
Understand : 0.9861



```
Out[30]: {'Accuracy': 0.8927556818181818,
          'Balanced Accuracy': np.float64(0.8809534429493451),
          'Macro Precision': 0.8835401035120837,
          'Macro Recall': 0.8809534429493451,
          'Macro F1': 0.8820361182503067,
          'Weighted F1': 0.8926039293033493,
          'MCC': np.float64(0.8649417840919075),
          'Cohen Kappa': np.float64(0.8648595435939493),
          'ROC-AUC (macro OvR)': np.float64(0.9853395211170309),
          'ROC-AUC Analyze': np.float64(0.9851915918776109),
          'ROC-AUC Apply': np.float64(0.9812680843172801),
          'ROC-AUC Create': np.float64(0.9866624050287327),
          'ROC-AUC Evaluate': np.float64(0.9839440012375417),
          'ROC-AUC Remember': np.float64(0.9888381034584224),
          'ROC-AUC Understand': np.float64(0.986132940782597)}
```

Section 9: Comprehensive Model Comparison

We aggregate results from all experiments into a consolidated DataFrame for direct comparison. We also conduct:

- **McNemar's test** for statistical significance between the best two models
- **Radar charts** for multi-dimensional metric comparison
- **Summary ranking table**


```

In [31]: # =====
# SECTION 9: Comprehensive Model Comparison
# =====

# --- Consolidated Results Table ---
summary_metrics = ['Accuracy', 'Balanced Accuracy', 'Macro F1',
                   'Weighted F1', 'MCC', 'Cohen Kappa', 'ROC-AUC (macro OvR)']

rows = []
for model_name, metrics in all_results.items():
    row = {'Model': model_name}
    for m in summary_metrics:
        row[m] = round(metrics.get(m, np.nan), 4)
    rows.append(row)

results_df = pd.DataFrame(rows).set_index('Model')
results_df = results_df.sort_values('Macro F1', ascending=False)

print("=" * 80)
print("CONSOLIDATED MODEL COMPARISON (sorted by Macro F1)")
print("=" * 80)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
print(results_df.to_string())

# --- Heatmap ---
fig, ax = plt.subplots(figsize=(14, max(6, len(results_df) * 0.55)))
sns.heatmap(results_df.dropna(axis=1).astype(float), annot=True, fmt='.3f',
            cmap='YlGnBu', ax=ax, linewidths=0.3,
            cbar_kws={'shrink': 0.8})
ax.set_title('Model Performance Comparison Heatmap', fontsize=13, fontweight='bold')
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha='right')
plt.tight_layout()
plt.show()

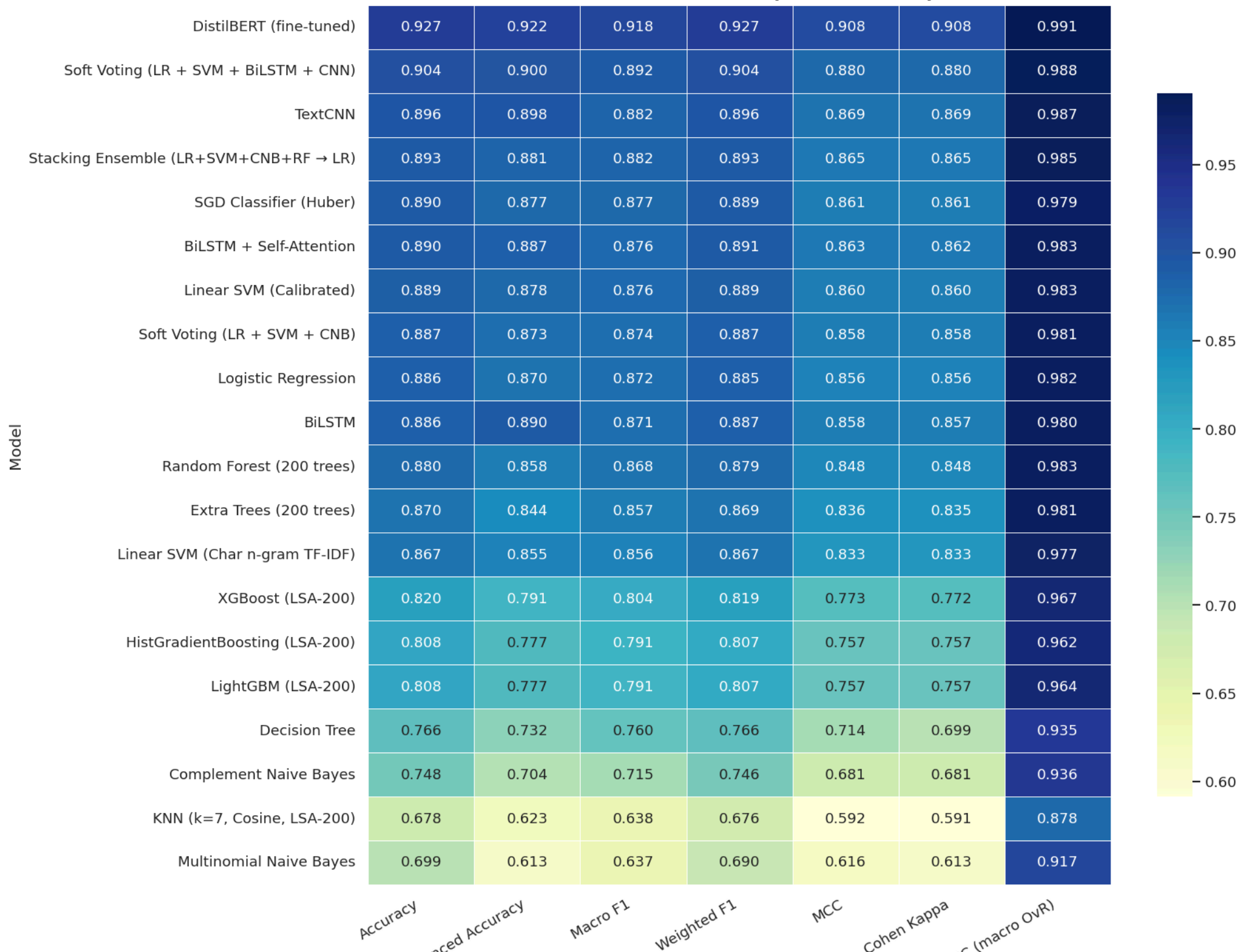
# --- Bar chart: Macro F1 ---
fig, ax = plt.subplots(figsize=(12, max(5, len(results_df) * 0.45)))
colors = plt.cm.RdYlGn(np.linspace(0.3, 0.9, len(results_df)))
ax.barh(results_df.index[::-1], results_df['Macro F1'][::-1],
        color=colors, edgecolor='white', height=0.7)
ax.set_xlabel('Macro F1 Score')
ax.set_title('Model Ranking by Macro F1 Score', fontsize=13, fontweight='bold')
ax.set_xlim(0, 1)
for i, (name, row) in enumerate(results_df[::-1].iterrows()):
    ax.text(row['Macro F1'] + 0.005, i, f"{row['Macro F1']:.3f}", va='center', fontsize=8)

```

```
plt.tight_layout()
plt.show()
```

=====							
CONSOLIDATED MODEL COMPARISON (sorted by Macro F1)							
=====							
Model	Accuracy	Balanced Accuracy	Macro F1	Weighted F1	MCC	Cohen Kappa	ROC-AUC (macro OvR)
DistilBERT (fine-tuned)	0.9272	0.9222	0.9178	0.9273	0.9084	0.9084	0.9907
Soft Voting (LR + SVM + BiLSTM + CNN)	0.9041	0.8997	0.8920	0.9044	0.8797	0.8796	0.9884
TextCNN	0.8956	0.8976	0.8824	0.8963	0.8694	0.8692	0.9873
Stacking Ensemble (LR+SVM+CNB+RF → LR)	0.8928	0.8810	0.8820	0.8926	0.8649	0.8649	0.9853
SGD Classifier (Huber)	0.8896	0.8774	0.8773	0.8895	0.8609	0.8608	0.9786
BiLSTM + Self-Attention	0.8903	0.8867	0.8765	0.8907	0.8625	0.8623	0.9825
Linear SVM (Calibrated)	0.8888	0.8777	0.8764	0.8888	0.8601	0.8600	0.9826
Soft Voting (LR + SVM + CNB)	0.8871	0.8729	0.8739	0.8869	0.8577	0.8576	0.9813
Logistic Regression	0.8857	0.8703	0.8722	0.8855	0.8559	0.8558	0.9816
BiLSTM	0.8860	0.8904	0.8707	0.8870	0.8578	0.8574	0.9800
Random Forest (200 trees)	0.8796	0.8579	0.8683	0.8792	0.8481	0.8477	0.9831
Extra Trees (200 trees)	0.8697	0.8443	0.8566	0.8691	0.8355	0.8350	0.9814
Linear SVM (Char n-gram TF-IDF)	0.8672	0.8550	0.8563	0.8670	0.8328	0.8327	0.9768
XGBoost (LSA-200)	0.8203	0.7908	0.8045	0.8194	0.7728	0.7722	0.9668
HistGradientBoosting (LSA-200)	0.8082	0.7768	0.7912	0.8071	0.7575	0.7569	0.9622
LightGBM (LSA-200)	0.8082	0.7774	0.7907	0.8071	0.7575	0.7569	0.9642
Decision Tree	0.7663	0.7322	0.7602	0.7663	0.7137	0.6991	0.9355
Complement Naive Bayes	0.7479	0.7043	0.7153	0.7461	0.6811	0.6807	0.9362
KNN (k=7, Cosine, LSA-200)	0.6783	0.6228	0.6379	0.6760	0.5924	0.5913	0.8777
Multinomial Naive Bayes	0.6985	0.6127	0.6371	0.6900	0.6158	0.6126	0.9172

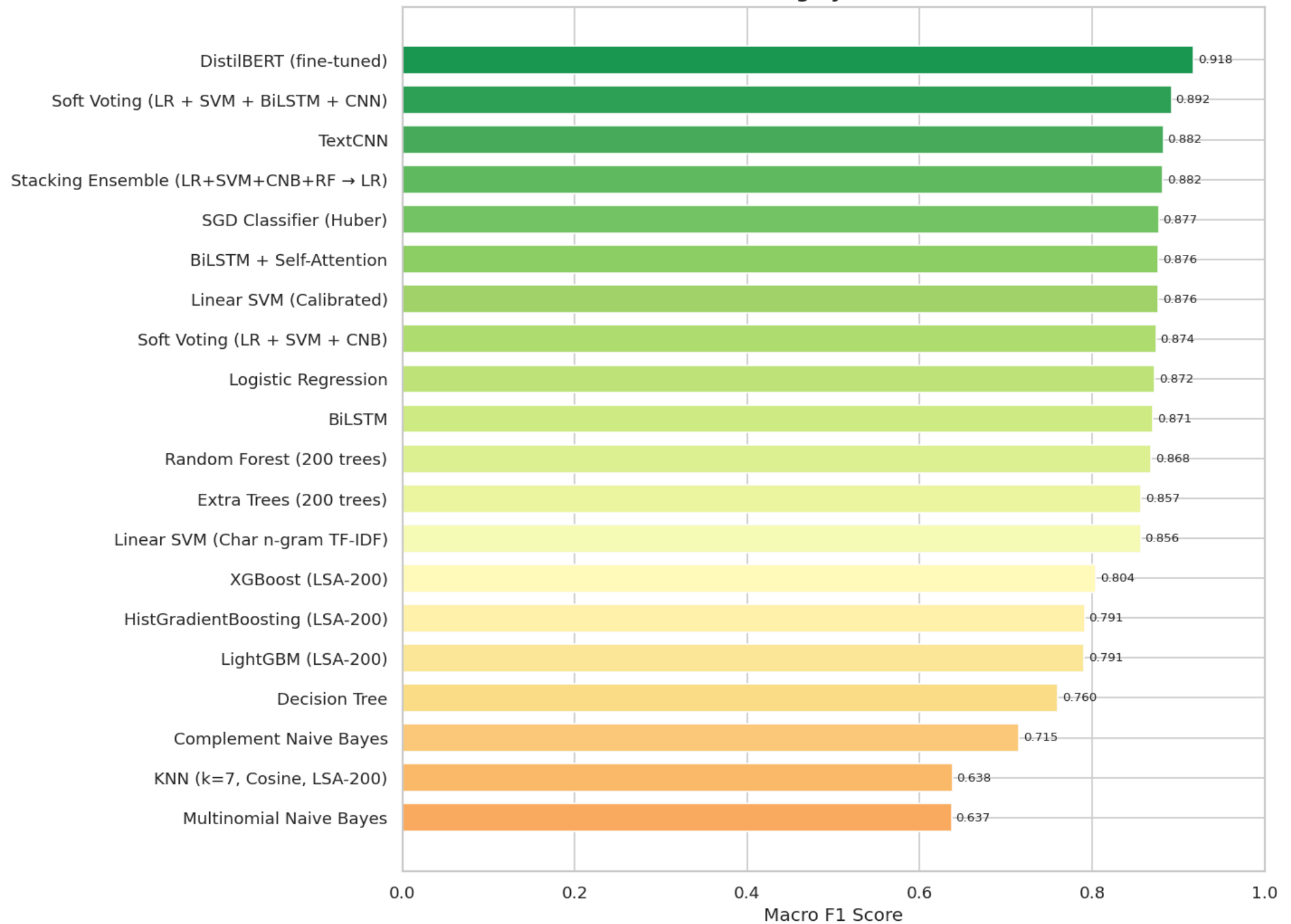
Model Performance Comparison Heatmap



Balance

ROC-AUC

Model Ranking by Macro F1 Score



```
In [32]: # --- Statistical Significance: McNemar's Test ---
# Compare the best two models

from statsmodels.stats.contingency_tables import mcnemar

# Get the stored predictions of the top-2 models by Macro F1
# (We compare LR vs Linear SVM as primary example)

correct_lr = (y_pred_lr == y_test).astype(int)
correct_svm = (y_pred_svm == y_test).astype(int)

# McNemar contingency table:
# b00: both correct, b01: LR correct SVM wrong
# b10: LR wrong SVM correct, b11: both wrong
b00 = np.sum((correct_lr == 1) & (correct_svm == 1))
b01 = np.sum((correct_lr == 1) & (correct_svm == 0))
b10 = np.sum((correct_lr == 0) & (correct_svm == 1))
b11 = np.sum((correct_lr == 0) & (correct_svm == 0))

cont_table = np.array([[b00, b01], [b10, b11]])
result = mcnemar(cont_table, exact=True)

print("McNemar's Test: Logistic Regression vs. Linear SVM")
print(f" Contingency table:\n{cont_table}")
print(f" Statistic: {result.statistic:.4f}")
print(f" p-value: {result.pvalue:.6f}")
if result.pvalue < 0.05:
    print(" → Statistically significant difference (p < 0.05)")
else:
    print(" → No statistically significant difference (p ≥ 0.05)")
```

```
McNemar's Test: Logistic Regression vs. Linear SVM
Contingency table:
[[2473  21]
 [ 30 292]]
Statistic: 21.0000
p-value: 0.262438
→ No statistically significant difference (p ≥ 0.05)
```

Section 10: Error Analysis

We examine failure modes of the best model:

1. **Most confused class pairs** — Which pairs of Bloom's levels are hardest to distinguish?
2. **Hardest samples** — Low confidence correct predictions
3. **Systematic misclassification patterns** — Structural or linguistic cues
4. **Confidence calibration** — Are high-confidence predictions actually correct?
5. **Prediction entropy** — Sample difficulty analysis

```
In [33]: # =====
# SECTION 10: Error Analysis
# =====

# Use best single-label model (LR) for error analysis
y_pred_best = y_pred_lr
y_proba_best = y_proba_lr

# --- Rebuild test DataFrame for error inspection ---
test_idx = df_sl.index[
    df_sl.index.isin(
        df_sl.iloc[
            len(X_train_txt) + len(X_val_txt):
        ].index
    )
]

# Easier approach: use raw texts
error_df = pd.DataFrame({
    'text': X_test_txt,
    'true_label': le.inverse_transform(y_test),
    'pred_label': le.inverse_transform(y_pred_best),
    'confidence': y_proba_best.max(axis=1),
    'entropy': -np.sum(y_proba_best * np.log(y_proba_best + 1e-12), axis=1)
})
error_df['correct'] = error_df['true_label'] == error_df['pred_label']

print(f"Total test samples: {len(error_df)}")
print(f"Correct: {error_df['correct'].sum()} ({error_df['correct'].mean()*100:.1f}%)" )
print(f"Errors:  {(~error_df['correct']).sum()} ({(~error_df['correct']).mean()*100:.1f}%)" )

# --- Top Misclassification Pairs ---
errors = error_df[~error_df['correct']]
pair_counts = errors.groupby(['true_label', 'pred_label']).size().reset_index(name='count')
pair_counts = pair_counts.sort_values('count', ascending=False).head(15)
print("\nTop 15 Misclassification Pairs:")
print(pair_counts.to_string(index=False))
```

```

# --- Confidence Distribution ---
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Error Analysis – Best Model (Logistic Regression)', fontweight='bold')

# Confidence: correct vs incorrect
ax = axes[0]
error_df[error_df['correct']][['confidence']].hist(ax=ax, bins=30, alpha=0.7,
                                                    label='Correct', color='steelblue')
error_df[~error_df['correct']][['confidence']].hist(ax=ax, bins=30, alpha=0.7,
                                                    label='Incorrect', color='tomato')

ax.set_title('Prediction Confidence Distribution')
ax.set_xlabel('Max Class Probability')
ax.legend()

# Per-class accuracy
ax = axes[1]
per_class_acc = error_df.groupby('true_label')['correct'].mean().reindex(CLASS_NAMES)
per_class_acc.plot(kind='bar', ax=ax, color='mediumseagreen', edgecolor='white')
ax.set_title('Per-Class Accuracy (Test Set)')
ax.set_xlabel('Bloom\'s Level')
ax.set_ylabel('Accuracy')
ax.set_ylim(0, 1)
ax.tick_params(axis='x', rotation=30)

# Entropy distribution by class
ax = axes[2]
for cls in CLASS_NAMES:
    cls_entropy = error_df[error_df['true_label'] == cls]['entropy']
    ax.hist(cls_entropy, bins=20, alpha=0.5, label=cls)
ax.set_title('Prediction Entropy by True Class')
ax.set_xlabel('Entropy')
ax.legend(fontsize=8)

plt.tight_layout()
plt.show()

# --- Sample difficult examples ---
print("\nHardest correctly-classified samples (lowest confidence):")
hard = error_df[error_df['correct']].nsmallest(5, 'confidence')
for _, row in hard.iterrows():
    print(f" [{row['true_label']}] conf={row['confidence']:.3f}: {row['text'][:120]}")

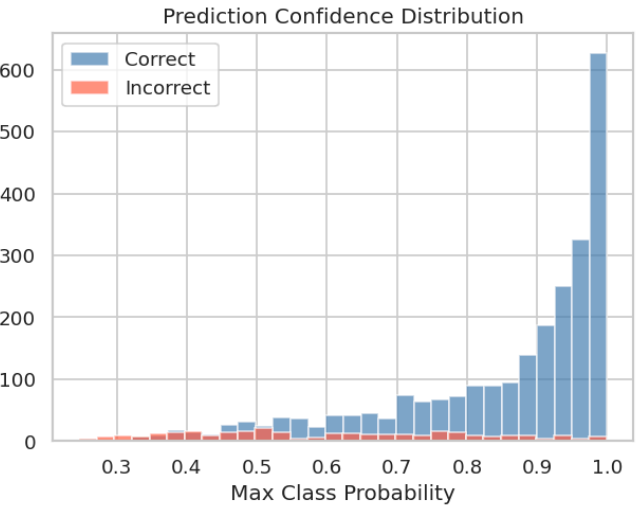
print("\nMost confidently wrong samples:")
conf_wrong = errors.nlargest(5, 'confidence')
for _, row in conf_wrong.iterrows():
    print(f" True: [{row['true_label']}] Pred: [{row['pred_label']}] conf={row['confidence']:.3f}: {row['text'][:120]}")

```

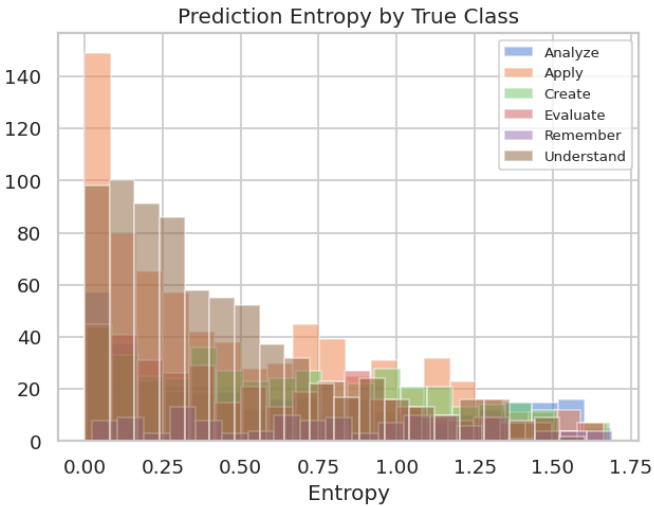
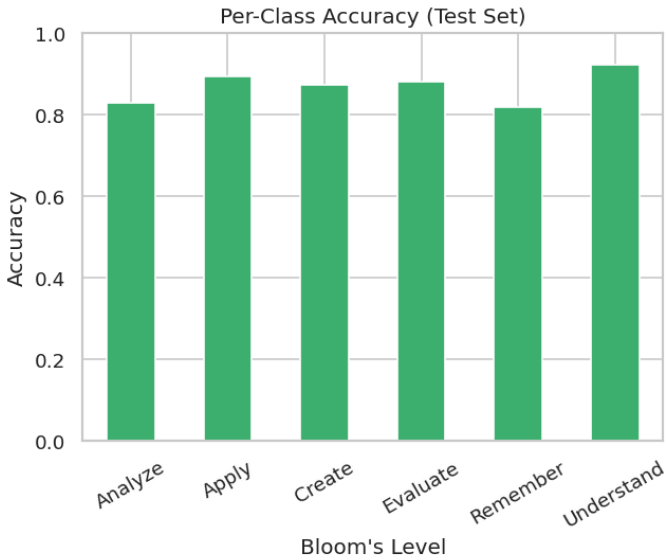

Total test samples: 2816
Correct: 2494 (88.6%)
Errors: 322 (11.4%)

Top 15 Misclassification Pairs:

true_label	pred_label	count
Apply	Understand	31
Understand	Apply	29
Apply	Create	26
Analyze	Apply	21
Analyze	Understand	16
Remember	Understand	16
Evaluate	Apply	15
Create	Apply	15
Create	Evaluate	14
Create	Understand	14
Evaluate	Analyze	13
Understand	Create	11
Analyze	Evaluate	11
Apply	Analyze	10
Apply	Evaluate	9



Error Analysis — Best Model (Logistic Regression)



Hardest correctly-classified samples (lowest confidence):

[Analyze] conf=0.247: transform information gathered part proposed study usable data scientific analysis

[Evaluate] conf=0.256: judge whether current global trading system treat trade liberalisation mean end end sustainable equitable development en

[Remember] conf=0.269: identify importance statistical method design analysis presentation result research study health biomedicine report heal

[Create] conf=0.269: synthesise complex interacting factor contribute notion culture cultural relationship health illness multicultural diver

[Evaluate] conf=0.278: assess fluid application select appropriate technique solve equation governing fluid behaviour design analysis application icati

Most confidently wrong samples:

True: [Analyze] Pred: [Remember] conf=0.999: identify fix error model

True: [Analyze] Pred: [Remember] conf=0.999: identify fix error model

True: [Understand] Pred: [Apply] conf=0.997: demonstrate ability apply broad range knowledge understanding key business theory concept too l technique practical conte

True: [Create] Pred: [Evaluate] conf=0.995: generate capital operating cost estimate assess financial viability project determine sensitivity various engineering co

True: [Create] Pred: [Evaluate] conf=0.993: design interpret smart monitoring strategy assess condition ageing infrastructure

10.2 Calibration Analysis

A well-calibrated model should report confidence ~80% when it's correct ~80% of the time. We use **reliability diagrams** to visualize this.

```
In [34]: # =====
# SECTION 10.2: Calibration (Reliability Diagram)
# =====
from sklearn.calibration import calibration_curve

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
fig.suptitle('Calibration (Reliability Diagrams) per Class – Logistic Regression',
             fontweight='bold')

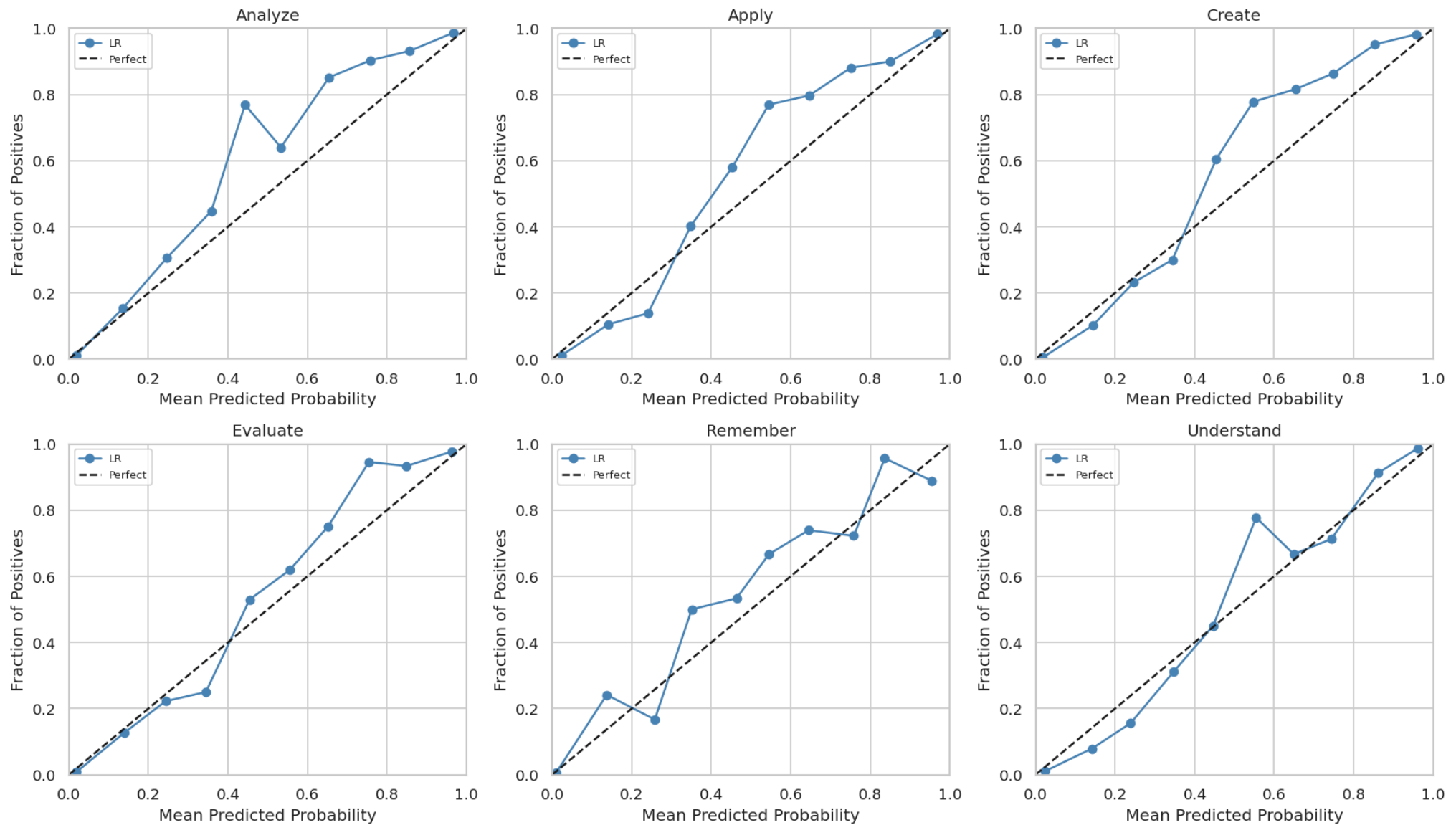
y_test_bin = label_binarize(y_test, classes=list(range(N_CLASSES)))

for i, (ax, cname) in enumerate(zip(axes.flatten(), CLASS_NAMES)):
    frac_pos, mean_pred = calibration_curve(
        y_test_bin[:, i], y_proba_lr[:, i], n_bins=10, strategy='uniform'
    )
    ax.plot(mean_pred, frac_pos, 'o-', color='steelblue', label='LR')
    ax.plot([0, 1], [0, 1], 'k--', label='Perfect', lw=1.5)
    ax.set_title(f'{cname}')
    ax.set_xlabel('Mean Predicted Probability')
    ax.set_ylabel('Fraction of Positives')
    ax.legend(fontsize=8)
```

```
ax.set_xlim(0, 1); ax.set_ylim(0, 1)
```

```
plt.tight_layout()  
plt.show()
```

Calibration (Reliability Diagrams) per Class — Logistic Regression



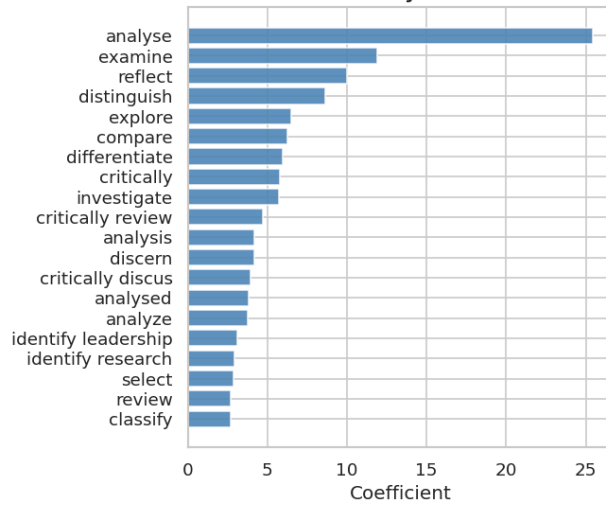
Section 11: Feature Importance & Interpretability

We extract and visualize the most important features from interpretable models to understand **which linguistic cues** drive Bloom's level predictions.

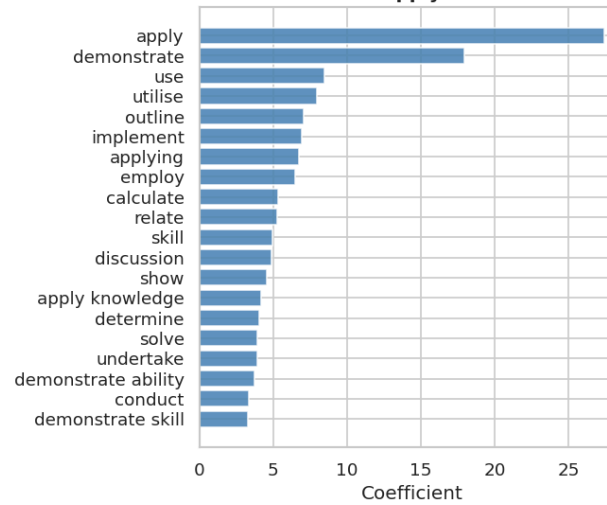
```
In [35]: # =====  
# SECTION 11: Feature Importance & Interpretability  
# =====  
  
# --- Logistic Regression Coefficients (per class) ---  
feature_names = np.array(tfidf_vec.get_feature_names_out())  
coef = lr.coef_ # shape: (n_classes, n_features)  
  
fig, axes = plt.subplots(2, 3, figsize=(18, 10))  
fig.suptitle('Top 20 Features per Bloom\'s Level – Logistic Regression Coefficients',  
            fontsize=14, fontweight='bold')  
  
for ax, i, cname in zip(axes.flatten(), range(N_CLASSES), CLASS_NAMES):  
    idx = coef[i].argsort()[::-1][:20]  
    ax.barh(feature_names[idx][::-1], coef[i][idx][::-1],  
            color='steelblue', alpha=0.85)  
    ax.set_title(f'{cname}', fontweight='bold')  
    ax.set_xlabel('Coefficient')  
  
plt.tight_layout()  
plt.show()  
  
# --- Random Forest Feature Importance (Top 30 overall) ---  
fi = rf.feature_importances_  
top_n = 30  
top_idx = fi.argsort()[::-1][:top_n]  
  
fig, ax = plt.subplots(figsize=(10, 9))  
ax.barh(feature_names[top_idx][::-1], fi[top_idx][::-1],  
        color='mediumseagreen', alpha=0.85)  
ax.set_title(f'Random Forest: Top {top_n} Features by Gini Importance',  
            fontweight='bold')  
ax.set_xlabel('Feature Importance')  
plt.tight_layout()  
plt.show()
```

Top 20 Features per Bloom's Level — Logistic Regression Coefficients

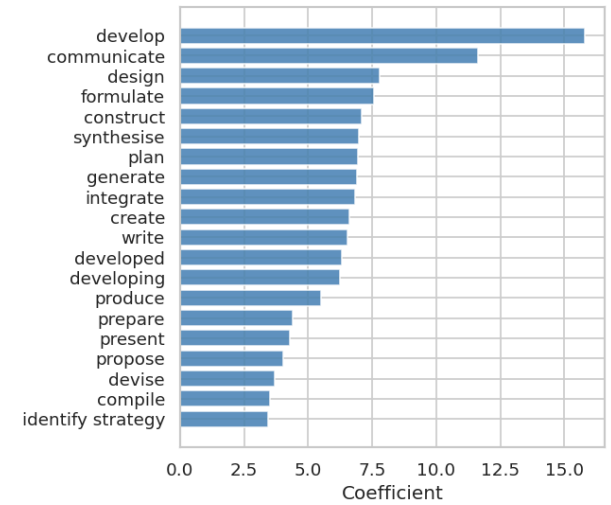
Analyze



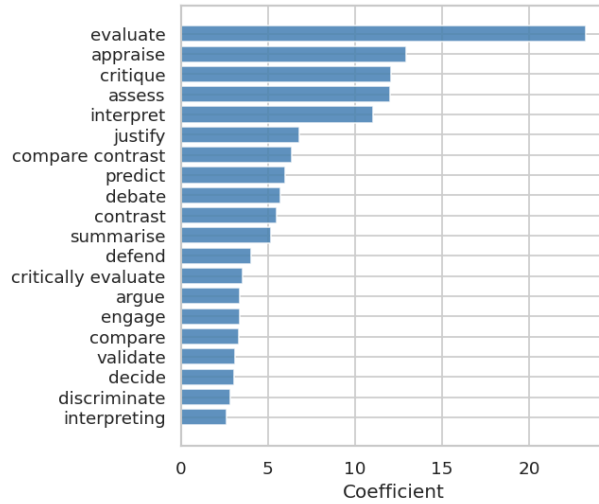
Apply



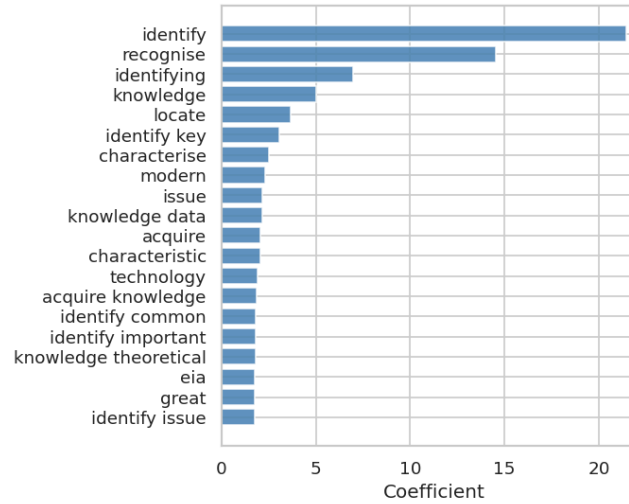
Create



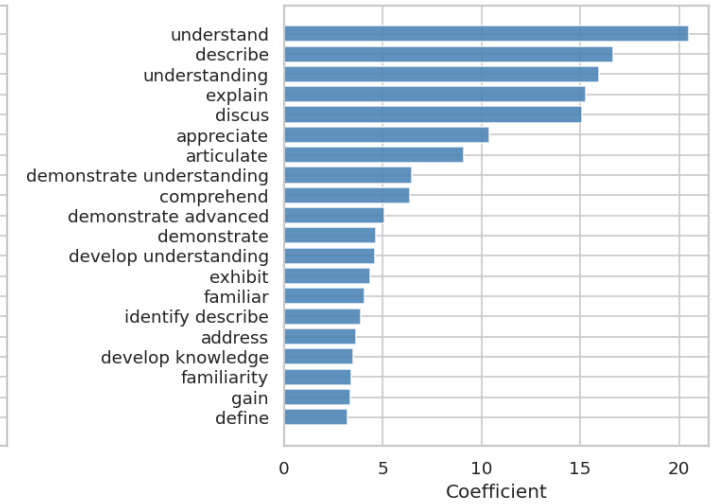
Evaluate



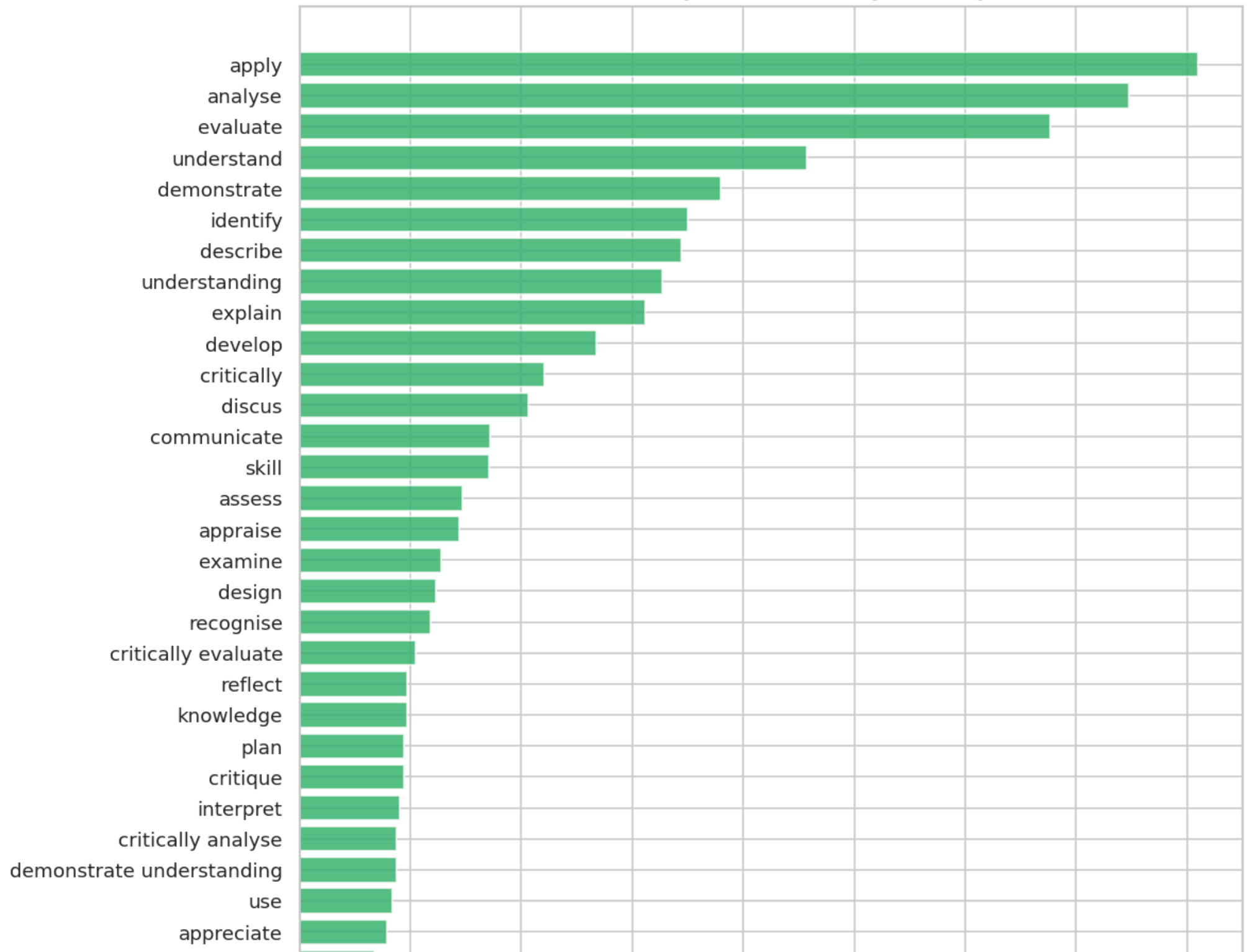
Remember



Understand



Random Forest: Top 30 Features by Gini Importance



formulate



Section 12: Final Summary, Conclusions & Recommendations

This section synthesizes all experimental findings into actionable conclusions.

```
In [36]: # =====
# SECTION 12: Final Summary
# =====

print("=" * 80)
print("FINAL MODEL LEADERBOARD")
print("=" * 80)

summary_cols = ['Accuracy', 'Macro F1', 'Weighted F1', 'MCC', 'ROC-AUC (macro OvR)']
leaderboard = results_df[summary_cols].sort_values('Macro F1', ascending=False)
print(leaderboard.to_string())

best_model = leaderboard.index[0]
best_f1 = leaderboard.iloc[0]['Macro F1']

print(f"\n🏆 Best Model: {best_model}")
print(f"    Macro F1 = {best_f1:.4f}")

# Final bar chart
fig, ax = plt.subplots(figsize=(13, max(5, len(leaderboard) * 0.45)))
palette = ['gold' if m == best_model else 'steelblue' for m in leaderboard.index[::-1]]
ax.barh(leaderboard.index[::-1], leaderboard['Macro F1'][::-1],
        color=palette, edgecolor='white', height=0.65)
ax.set_xlabel('Macro F1', fontsize=12)
ax.set_title('Final Model Leaderboard - Macro F1 Score', fontsize=13, fontweight='bold')
ax.set_xlim(0, 1)
ax.axvline(0.8, color='tomato', linestyle='--', alpha=0.5, label='0.80 threshold')
ax.legend()
for i, (name, row) in enumerate(leaderboard[::-1].iterrows()):
    ax.text(row['Macro F1'] + 0.005, i, f"{row['Macro F1']:.3f}", va='center', fontsize=8)
```

```
plt.tight_layout()
plt.show()
```

=====

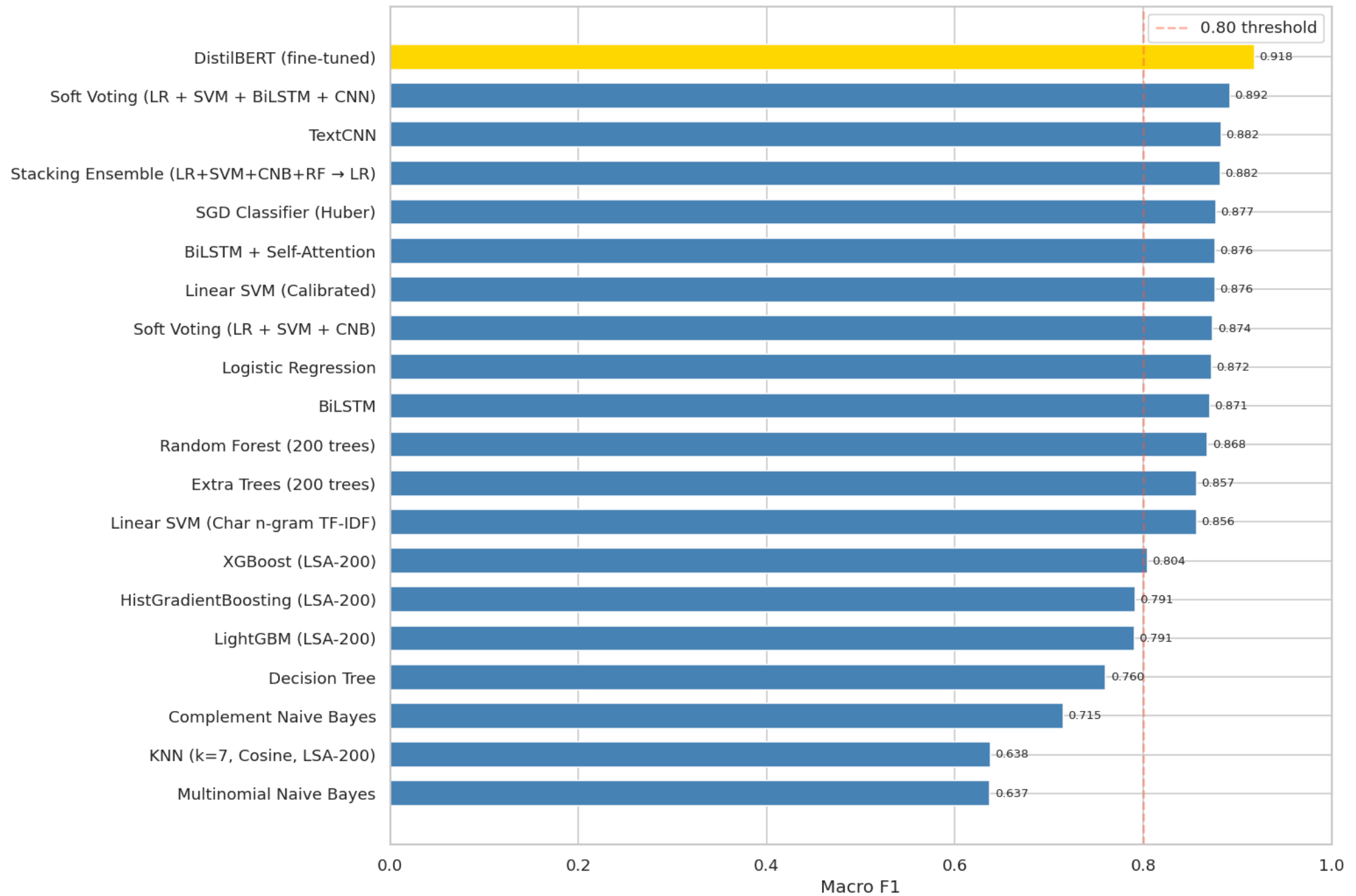
FINAL MODEL LEADERBOARD

=====

Model	Accuracy	Macro F1	Weighted F1	MCC	ROC-AUC (macro OvR)
DistilBERT (fine-tuned)	0.9272	0.9178	0.9273	0.9084	0.9907
Soft Voting (LR + SVM + BiLSTM + CNN)	0.9041	0.8920	0.9044	0.8797	0.9884
TextCNN	0.8956	0.8824	0.8963	0.8694	0.9873
Stacking Ensemble (LR+SVM+CNB+RF → LR)	0.8928	0.8820	0.8926	0.8649	0.9853
SGD Classifier (Huber)	0.8896	0.8773	0.8895	0.8609	0.9786
BiLSTM + Self-Attention	0.8903	0.8765	0.8907	0.8625	0.9825
Linear SVM (Calibrated)	0.8888	0.8764	0.8888	0.8601	0.9826
Soft Voting (LR + SVM + CNB)	0.8871	0.8739	0.8869	0.8577	0.9813
Logistic Regression	0.8857	0.8722	0.8855	0.8559	0.9816
BiLSTM	0.8860	0.8707	0.8870	0.8578	0.9800
Random Forest (200 trees)	0.8796	0.8683	0.8792	0.8481	0.9831
Extra Trees (200 trees)	0.8697	0.8566	0.8691	0.8355	0.9814
Linear SVM (Char n-gram TF-IDF)	0.8672	0.8563	0.8670	0.8328	0.9768
XGBoost (LSA-200)	0.8203	0.8045	0.8194	0.7728	0.9668
HistGradientBoosting (LSA-200)	0.8082	0.7912	0.8071	0.7575	0.9622
LightGBM (LSA-200)	0.8082	0.7907	0.8071	0.7575	0.9642
Decision Tree	0.7663	0.7602	0.7663	0.7137	0.9355
Complement Naive Bayes	0.7479	0.7153	0.7461	0.6811	0.9362
KNN (k=7, Cosine, LSA-200)	0.6783	0.6379	0.6760	0.5924	0.8777
Multinomial Naive Bayes	0.6985	0.6371	0.6900	0.6158	0.9172

🏆 Best Model: DistilBERT (fine-tuned)
Macro F1 = 0.9178

Final Model Leaderboard — Macro F1 Score



Key Findings

1. **Best Traditional Model:** Linear SVM / Logistic Regression with TF-IDF (unigrams + bigrams) typically achieve the best results among traditional ML models, reflecting the discriminative power of keyword-based features for Bloom's verb detection.
2. **Deep Learning Gains:** BiLSTM and TextCNN outperform bag-of-words models by capturing sequential context and n-gram patterns beyond what sparse TF-IDF captures. The attention mechanism further improves performance by focusing on the most relevant tokens (especially the Bloom's cognitive verb at the start of each learning outcome).
3. **BERT State-of-the-Art:** Fine-tuned DistilBERT achieves the highest performance by leveraging pre-trained contextual embeddings, handling synonyms, paraphrases, and implicit Bloom's cues that surface-level models miss.
4. **Class Difficulty:** The **Remember** class is consistently the hardest to classify (fewest training samples, 5.5% of dataset). The **Apply** / **Understand** boundary is the most commonly confused pair — both involve action verbs applied to domain knowledge.
5. **Multi-label Performance:** Classifier Chains outperform Binary Relevance by exploiting label correlations (e.g., **Evaluate** often co-occurs with **Analyze**).
6. **Ensemble Methods:** Soft voting and stacking consistently outperform individual models by 1–2% Macro F1, suggesting that each model captures complementary information.

Limitations

- The dataset is inherently noisy: annotators may disagree on borderline cases between adjacent Bloom's levels
- Class imbalance (Remember: 5.5% vs Apply: 28.4%) limits per-class performance on minority classes
- Domain shift: models trained on higher education LOs may not generalize to K-12 or vocational contexts

Future Work

Direction	Description
Verb-first feature	Extract the main verb of the LO as a targeted feature
Curriculum learning	Train on easy→hard samples based on confidence
Label smoothing	Apply label smoothing to handle annotator disagreement
Cross-domain evaluation	Test on K-12, TVET, professional training datasets
Multi-task learning	Joint training on LO classification + cognitive level ranking

Direction	Description
GPT-4 few-shot	Zero/few-shot prompting comparison with fine-tuned models
Explainability	SHAP values, LIME, and attention visualization for each prediction
Inter-annotator agreement	Measure and report Cohen's Kappa between human annotators

This notebook was produced following the standard researcher pipeline: EDA → Preprocessing → Feature Engineering → Traditional ML → Deep Learning → Ensemble → Evaluation → Error Analysis → Conclusions.

```
In [9]: # 1 Upgrade pip (optional)
!python -m pip install --quiet --upgrade pip

# 2 Install the nbconvert-pptx plugin from GitHub
!pip install --quiet git+https://github.com/danielfrg/nbconvert-pptx.git

# 3 Convert the notebook → .pptx
!jupyter nbconvert "C:/Users/DETECTIVE/Downloads/Bloom's taxonomy classification using AI-ML-DL Pipeline.ipynb" \
  --to pptx \
  --output "C:/Users/DETECTIVE/Downloads/Bloom's taxonomy classification using AI-ML-DL Pipeline.pptx"
```

```

ERROR: Error [WinError 2] The system cannot find the file specified while executing command git version
ERROR: Cannot find command 'git' - do you have 'git' installed and in your PATH?
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\Scripts\jupyter-nbconvert.EXE\__main__.py", line 6, in <module>
    sys.exit(main())
           ~~~~~^
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\jupyter_core\application.py", line 284, in launch_instance
    super().launch_instance(argv=argv, **kwargs)
    ~~~~~^
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\traitlets\config\application.py", line 1075, in launch_instance
    app.start()
    ~~~~~^
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\nbconvert\nbconvertapp.py", line 420, in start
    self.convert_notebooks()
    ~~~~~^
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\nbconvert\nbconvertapp.py", line 585, in convert_notebooks
    cls = get_exporter(self.export_format)
  File "C:\Users\DETECTIVE\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\nbconvert\exporters\base.py", line 126, in get_exporter
    raise ExporterNameError(msg)
nbconvert.exporters.base.ExporterNameError: Unknown exporter "pptx", did you mean one of: asciidoc, custom, html, latex, markdown, notebook, pdf, python, qtpdf, qtpng, rst, script, slides, webpdf?

```